

Proyecto Final

Implementación de una Aplicación de Gestión de Eventos como un CRUD utilizando DevOps y Spring Boot

Integrantes:

Jesús Seiler
Gustavo Andrade
Gabriel Muñoz
Daniela Gallardo
Erasto Borthomierth
Cristopher Vergara
Sixto Guerra

Índice

1. Planificación 4
1.1. Objetivo del Proyecto 4
1.2. Justificación Tecnológica 4
1.3 Fase de Definición de Requerimientos 5
1.4 Control de Versiones y Automatización 5
1.4.1 GitHub 5
1.4.2 Jenkins 5

2. Cronograma de Desarrollo 6
2.1. Visión General del Proyecto 6
2.2. Estructura del Proyecto en Scrum 6
2.3. Roles Principales 6
2.4. Product Backlog 6
2.5. Roadmap del Proyecto 7
2.6. Sprint Planning 7
2.7. Desarrollo en Sprints 7
2.8. Ceremonias Scrum 8
2.9. Cronograma General en Sprints 8
2.10. Herramientas de Monitoreo y Seguimiento 9
2.11. Tiempo Total Estimado 9
2.12. Beneficios del Enfoque Scrum 9
2.13. Resúmen del Proyecto 9
2.14 Entrega continua 9

3. Desarrollo 10
3.1. Implementación de versionamiento para trabajo en equipo 12

4. Pruebas 13
4.2 Mejora de la calidad 13
4.3 Seguridad 13
4.4 Funcionalidad 13
4.5 Usabilidad 13
4.6 Mantenimiento 13
4.7 Satisfacción del cliente 13

5. Despliegue 16
5.1.1 Consistencia entre Entornos 16
5.1.2 Tiempo de Despliegue Rápido 16
5.1.3 Aislamiento de Aplicaciones 16
5.2 Secciones y Componentes del Diagrama 17
5.3 Flujo del Pipeline de Jenkins 18

5.4 Interpretación Global del Flujo 18
5.5 Instalación de Ngrok y Jenkins 18
5.6. Generar un token en Jenkin para permitir conexiones entrantes 21
5.7. Generación de un Jenkinsfile 22
5.8. Configuración de GitHub 23
5.9. Configurar Jenkins 27
5.10. Crear un Pipeline en Jenkins 30
5.11. Crear un Push en el repositorio de GitHub y accionar el Pipeline 33
5.12. Integrar SonarQube 34
5.13. Integrar notificaciones con Slack 47
5.14. Integrar Nexus Sonatype 55

6. Monitoreo 65
6.1 Elasticsearch-Kibana-Logstash (ELK) 65
6.1.1 Definir contenedores en docker-compose.yml 65
6.1.2 Definir configuraciones de logstash 66
6.1.3 Agregar dependencia de logstash en pom.xml 67
6.1.4 Crear y configurar archivo logback-spring.xml en carpeta de resources 67
6.1.5 Verificar/Agregar logs en controladores REST 68
6.1.6 Docker-compose up y verificar creacion del index 68
6.1.7 Creación de Dataview de Kibana 70
6.1.8 Resultado final y visualización de Logs 71
6.2 Graylog,Fluentd,Grafana (GFG) 71

7. Conclusión 76

1. Planificación

1.1. Objetivo del Proyecto

El objetivo del proyecto es implementar una aplicación web para la gestión y arriendo de salones, donde se llevarán a cabo eventos de diversos tipos, el sistema deberá ser escalable, resiliente y estar enfocado en la calidad y la automatización.

Tecnologías principales:

- **Lenguaje**: Java
- **Framework**: Spring Boot
- **CI/CD**: Jenkins
- **Control de versiones**: GitHub
- **Pruebas**: Herramientas de testing automatizado integradas en el pipeline de Jenkins.
- **Observabilidad**: Implementación de herramientas de monitoreo, para garantizar la estabilidad y rendimiento del sistema.

1.2. Justificación Tecnológica

- **Lenguaje**: Se ha seleccionado **Java,** debido a su confiabilidad, seguridad y amplio ecosistema de herramientas de soporte.
- **Framework**: **Spring Boot**, este ha sido elegido por su capacidad de ofrecer aplicaciones empresariales escalables y su integración fluida con herramientas de terceros para manejo de dependencias, seguridad y bases de datos.
- **Automatización**: Se utilizará **Jenkins** para gestionar el pipeline de integración continua y despliegue continuo (CI/CD), permitiendo ciclos de desarrollo ágiles y automatizados.
- **Versionamiento**: El código será manejado en **GitHub**, utilizando flujos de trabajo de ramas para facilitar la colaboración y el control de versiones.
- **Observabilidad**: Se implementarán herramientas de observabilidad para monitorear la aplicación y garantizar un rendimiento óptimo y la capacidad de recuperación ante fallos.

Para garantizar la **resiliencia** y **calidad** de la aplicación, utilizaremos una serie de herramientas para la automatización de pruebas y validaciones, el proceso completo será automatizado utilizando **Jenkins**, con el objetivo de estandarizar el flujo de despliegue y simplificar la gestión a largo plazo, además, se implementarán herramientas de **observabilidad** y **control de versiones** utilizando **GitHub**.

1.3 Fase de Definición de Requerimientos

- **Duración**: 2 días
- **Actividades**:
    - Reunión con stakeholders para definir los requerimientos funcionales y no funcionales del sistema.
    - Identificación de los casos de uso clave (arriendo de salones, gestión de disponibilidad, creación de eventos, etc.).
    - Definición de historias de usuario prioritarias y aceptación de criterios.

Entregables:

- Documento de especificaciones técnicas y funcionales.
- Lista inicial de historias de usuario en el **Product Backlog**.

1.4 Control de Versiones y Automatización

Para mantener un control estricto sobre el progreso del desarrollo y la calidad del código, usaremos GitHub para versionamiento y Jenkins para automatización:

1.4.1 GitHub

- **Branching Strategy**: Se implementará Git Flow para gestionar ramas y versiones de forma clara.
    - **Master**: Rama de producción.
    - **Develop**: Rama principal de desarrollo.
    - **Feature Branches**: Para el desarrollo de nuevas funcionalidades.
    - **Hotfix Branches**: Para correcciones urgentes.

1.4.2 Jenkins

- Se configurará un pipeline en Jenkins que, al realizar un push a GitHub, ejecute automáticamente:
    - Pruebas unitarias y de integración.
    - Validación del código con herramientas de análisis estático (e.g., SonarQube).
    - Despliegue a entornos de pruebas y producción.

2. Cronograma de Desarrollo

A continuación, se presenta una estimación del tiempo necesario para el desarrollo y despliegue de los diferentes componentes de la aplicación. Cabe destacar que el sistema es relativamente sencillo, lo que permite tiempos de desarrollo razonables:

2.1. Visión General del Proyecto

El proyecto consiste en el desarrollo de una aplicación para la gestión de eventos y salones. El sistema debe contar con controladores, entidades, repositorios y servicios, que permitan realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) y gestionar la lógica de negocio asociada. Además, se incluye la implementación de pruebas unitarias e integración, y el despliegue en un entorno de pruebas.

2.2. Estructura del Proyecto en Scrum

En Scrum, el trabajo se organiza en sprints cortos y repetitivos que permiten la entrega incremental de valor. Cada sprint tiene una duración definida (generalmente entre 1 y 2 semanas) y finaliza con un incremento de producto funcional.

2.3. Roles Principales

- Product Owner: Encargado de definir las prioridades del proyecto, gestionar el backlog y asegurar que el equipo esté alineado con los objetivos de negocio.
- Scrum Master: Facilita las ceremonias de Scrum, ayuda a remover impedimentos y asegura que el equipo siga los principios ágiles.
- Equipo de Desarrollo: Conformado por desarrolladores, testers y otros profesionales técnicos que implementan las historias de usuario.

2.4. Product Backlog

El Product Backlog contiene todas las funcionalidades necesarias para completar el proyecto. Estas funcionalidades se describen como historias de usuario, cada una con sus criterios de aceptación, las cuales serán implementadas en sprints.

Historias de Usuario

HU01: Como organizador de eventos, quiero poder crear, leer, actualizar y eliminar eventos, para gestionar las actividades de manera eficiente.

Criterios de Aceptación: CRUD completo implementado para EventoController.
HU02: Como administrador del sistema, quiero gestionar los salones disponibles, para asignarlos correctamente a los eventos.

Criterios de Aceptación: CRUD completo implementado para SalonController.

HU03: Como usuario del sistema, quiero poder visualizar la lista de eventos y su disponibilidad, para planificar la ocupación de los salones.

Criterios de Aceptación: Vista de eventos y salones disponible en la interfaz de usuario.

2.5. Roadmap del Proyecto

El roadmap inicial, propone un ciclo de trabajo de 3 sprints, con una duración de 1-2 semanas cada uno.

2.6. Sprint Planning

Cada sprint comienza con una reunión de planificación, donde el equipo selecciona las historias de usuario que pueden completar durante el sprint, las tareas seleccionadas se desglosan en subtareas técnicas que permiten su implementación.

2.7. Desarrollo en Sprints

Sprint 1 (Duración: 2 semanas)

Objetivo del Sprint: Desarrollo de las entidades y controladores iniciales para eventos y salones.

Historias de Usuario:

- Implementar la entidad `Evento` (HU01).
- Implementar la entidad `Salon` (HU02).
- CRUD completo para `EventoController` (HU01).
- CRUD completo para `SalonController` (HU02).
- Desarrollo de los repositorios (`EventoRepositorio` y `SalonRepositorio`).

Entregables:

- Controladores y entidades implementados con operaciones básicas de CRUD.
- Repositorios configurados con JPA para la persistencia de datos.

Sprint 2 (Duración: 2 semanas)

Objetivo del Sprint: Implementación de la lógica de negocio y servicios.

Historias de Usuario:

- Desarrollar `EventoServicio` para gestionar la lógica de negocio de eventos (HU01).
- Desarrollar `SalonServicio` para gestionar la lógica de negocio de salones (HU02).

- Validación de reglas de negocio (e.g., no permitir eventos en salones ocupados).

Entregables:

- Servicios implementados y probados.
- Reglas de negocio validadas.

Sprint 3 (Duración: 1-2 semanas)

Objetivo del Sprint: Pruebas de integración y despliegue en entorno de pruebas.

Historias de Usuario:

- Implementación de pruebas unitarias y de integración.
- Despliegue en entorno de pruebas.
- Validación funcional y ajuste de errores.

Entregables:

- Sistema funcional con pruebas de integración.
- Despliegue en entorno de pruebas listo para producción.

2.8. Ceremonias Scrum

Daily Scrum

Reunión diaria de 15 minutos en la que el equipo de desarrollo discute el estado actual del trabajo, posibles impedimentos y las actividades del día.

Sprint Review

Al final de cada sprint, el equipo presenta el incremento de producto desarrollado al **Product Owner** y otros stakeholders. El objetivo es mostrar los avances, recibir feedback y ajustar el plan para el siguiente sprint si es necesario.

Sprint Retrospective

Después de cada **Sprint Review**, el equipo realiza una retrospectiva para discutir lo que funcionó bien, lo que se puede mejorar, y cómo optimizar la colaboración y el flujo de trabajo para futuros sprints.

2.9. Cronograma General en Sprints

- **Sprint 1**: Desarrollo de entidades, controladores y repositorios (5-8 días).

- **Sprint 2**: Desarrollo de servicios y reglas de negocio (7-9 días).
- **Sprint 3**: Pruebas, ajustes y despliegue en entorno de pruebas (5-7 días).

2.10. Herramientas de Monitoreo y Seguimiento

Burndown Chart

El **Burndown Chart** será utilizado para monitorear el progreso del sprint en tiempo real, visualizando las tareas completadas y las que aún faltan por desarrollar.

Product Backlog Refinement

El Product Backlog se revisa continuamente para ajustar prioridades, agregar nuevas historias de usuario, y refinar las ya existentes, de modo que el equipo siempre trabaje en lo más relevante para el proyecto.

2.11. Tiempo Total Estimado

- **Duración total del proyecto**: 17-26 días, distribuidos en 3 sprints.

2.12. Beneficios del Enfoque Scrum

- **Flexibilidad**: Permite adaptarse a cambios y ajustes en el producto.
- **Entrega continua**: Cada sprint entrega un incremento funcional que puede ser utilizado o revisado por los stakeholders.
- **Colaboración**: El equipo trabaja de forma colaborativa, recibiendo feedback constante y mejorando el producto en cada ciclo.

2.13. Resúmen del Proyecto

- **Planificación**: Definición de requerimientos y diseño de arquitectura.
- **Desarrollo**: Implementación de controladores, servicios y lógica de negocio.
- **Pruebas y despliegue**: Automatización con Jenkins y monitoreo con herramientas de observabilidad.

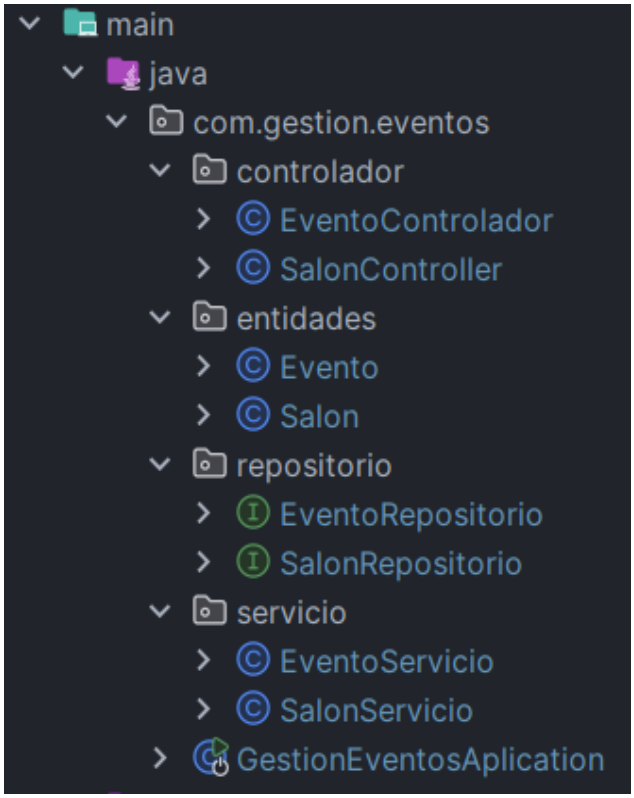
2.14 Entrega continua

Integración continua y entregas incrementales cada sprint.

3. Desarrollo

Se escogió Spring Boot por su excelente escalabilidad y rendimiento, además siguiendo las pautas para estructurar el código recomendada por el framework produciremos un resultado ordenado y fácil de entender.

El código del proyecto se ordena de la siguiente manera



Hay un package central llamado *com.gestion.eventos* que aloja a los demás los cuales son:

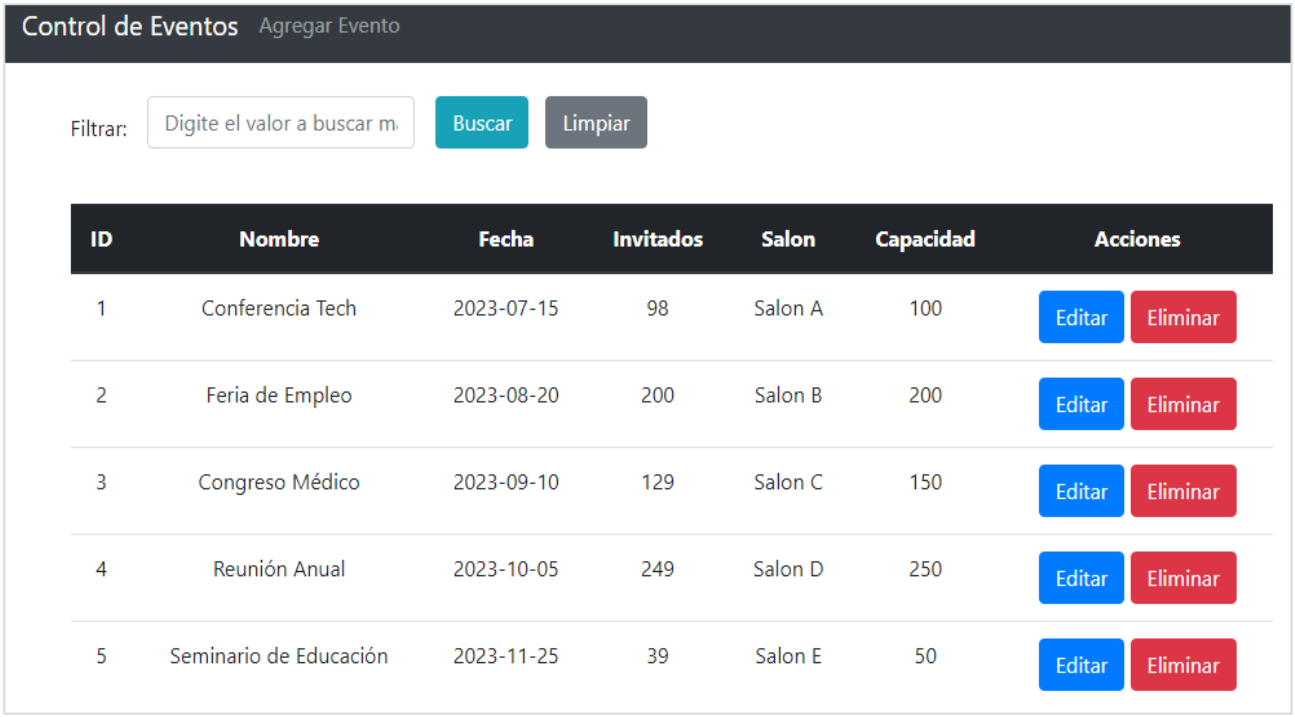
controlador: es la capa de exposición de nuestra aplicación, en esta se encuentran los endpoints que nos permiten interactuar con la app

entidades: las entidades son abstracciones de elementos físicos hacia código, acá tenemos dos de estas, las cuales son Evento y Salón. Como sus nombres lo indican poseen las propiedades de los eventos y los salones con los que vamos a interactuar en nuestra aplicación.

repositorio: es la capa la cual nos permite interactuar con la base de datos para almacenar y extraer nuestros eventos.

servicio: es la capa la cual almacena la lógica de negocio de nuestra aplicación, aquí podremos registrar eventos, asignarlos a un salón, asignarles una fecha, números de invitados, editarlos posteriormente, asignarles un precio etc.

En la siguiente captura podemos ver un poco de la funcionalidad de la web en donde listamos los eventos a efectuarse en próximas fechas, además posee un botón para buscar y cada evento se puede editar o eliminar



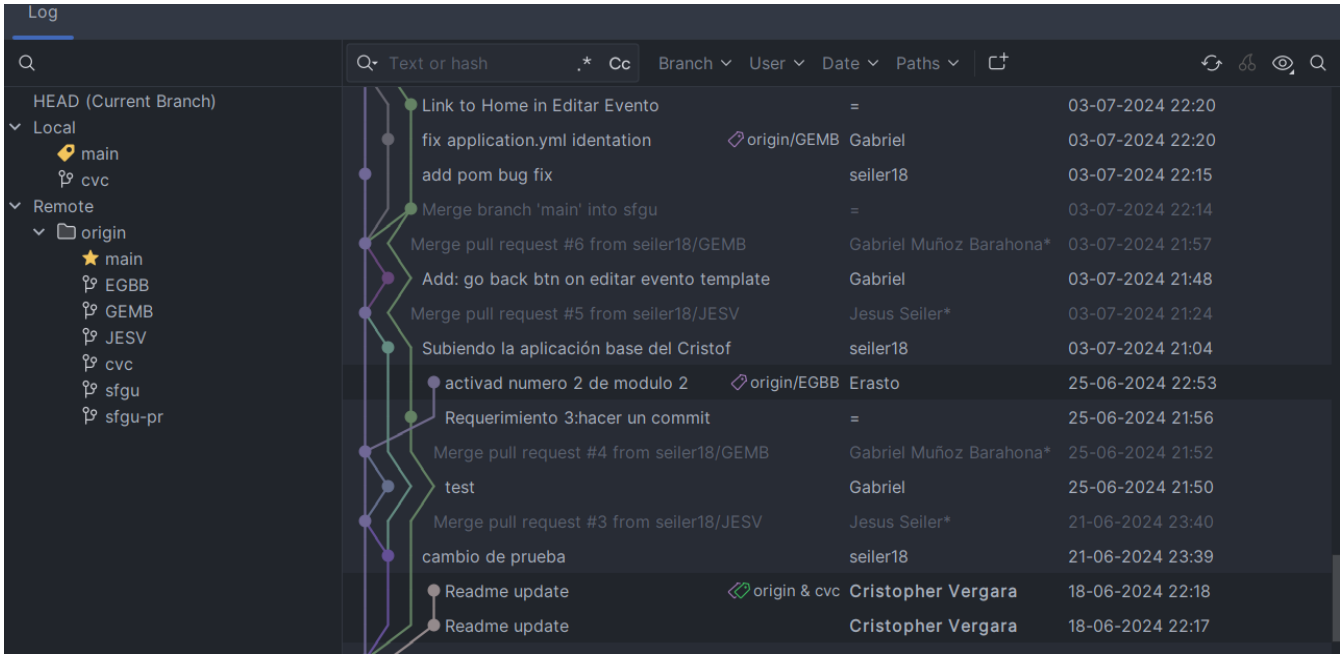
En la siguiente captura podemos ver el formulario de registro del evento en donde se nos solicita un nombre para identificarlo, la fecha a efectuarse, la cantidad de invitados y la asignación de algún salón para llevarse a cabo.



3.1. Implementación de versionamiento para trabajo en equipo.

Para el correcto control de versiones usaremos GitHub, en esta plataforma podemos almacenar, compartir y trabajar junto con otros usuarios para escribir código. Guardar nuestro código en un "repositorio" en GitHub nos permite: Presentar o compartir el trabajo. Seguir y administrar los cambios en el código a lo largo del tiempo.

Una vez subido nuestro código a GitHub creamos varias ramas o también llamadas branchs, cada branch se asigna a un miembro del equipo para que este pueda desarrollar las características que estime que ayuden a mejorar el proyecto.



4. Pruebas

Una vez codificado el software, crearemos pruebas que nos aseguren que nuestras funcionalidades y lleven a cabo las tareas para las cuales fueron implementadas.

Además, de asegurar la funcionalidad el testeo en nuestro código, que conlleva beneficios entre los cuales están:

4.1 Detección de errores: Ayudan a identificar y corregir fallos antes de que el software sea implementado, lo que reduce costos a largo plazo.

4.2 Mejora de la calidad: Aseguran que el software cumpla con los estándares de calidad y las expectativas del usuario.

4.3 Seguridad: Permiten identificar vulnerabilidades que podrían ser explotadas, garantizando que la aplicación sea segura.

4.4 Funcionalidad: Verifican que todas las funciones y características del software operen correctamente.

4.5 Usabilidad: Ayudan a asegurar que la interfaz de usuario sea intuitiva y fácil de usar.

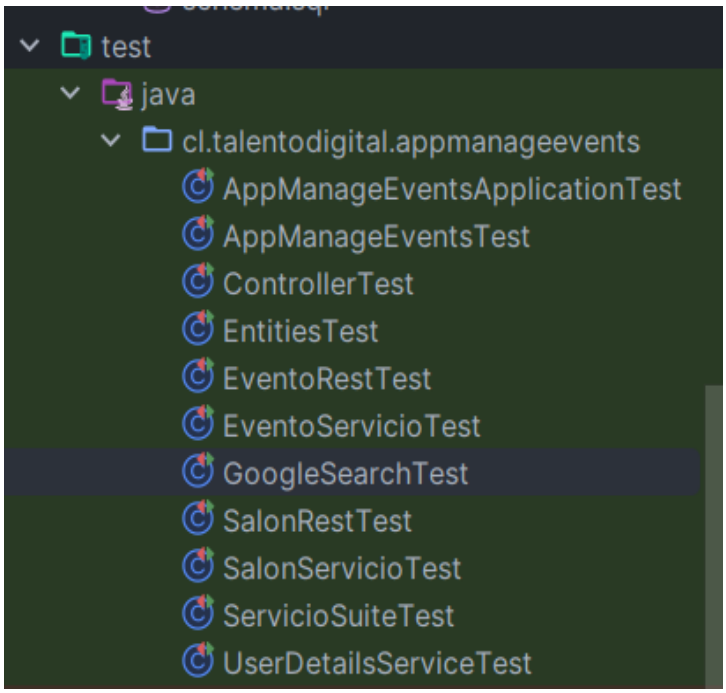
4.6 Mantenimiento: Facilitan el mantenimiento futuro del software al proporcionar una base sólida de pruebas que pueden ser reutilizadas.

4.7 Satisfacción del cliente: Un software bien probado genera confianza en los usuarios y mejora la satisfacción general.

Realizar pruebas es, por tanto, una inversión que contribuye a un producto final más robusto y fiable.

Para las Pruebas Unitarias, se eligió a JUnit como framework ya que posee una alta integración con Spring Boot y hay disponible una gran cantidad de documentación para este.

Estas pruebas quedan alojadas en el mismo proyecto para una fácil visibilidad para el equipo de desarrollo y posterior ejecución por herramientas CI/CD



Además de codificar los tests necesarios incluimos los plugins de Jacoco y Surefire los cuales generan un informe automático de fácil acceso y visualización para todos los miembros del equipo.

El comando para ejecutar los tests y ver sus informes es mvn test.

```
~\Documents\Basic_Crud_Thymeleaf git:[main]
mvn test
[INFO] Scanning for projects...
[INFO]
[INFO] ---------------< com.gestion.productos:gestion-eventos >---------------
[INFO] Building gestion-eventos 0.0.1-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- jacoco:0.8.8:prepare-agent (prepare-agent) @ gestion-eventos ---
[INFO] argLine set to -javaagent:C:\\Users\\crist\\.m2\\repository\\org\\jacoco\\org.jac
 .jar=destfile=C:\\Users\\crist\\Documents\\Basic_Crud_Thymeleaf\\target\\jacoco.exec
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ gestion-eventos ---
[INFO] Copying 1 resource from src\main\resources to target\classes
[INFO] Copying 7 resources from src\main\resources to target\classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ gestion-eventos ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ gestion-eventos ---
[INFO] skip non existing resourceDirectory C:\Users\crist\Documents\Basic_Crud_Thymeleaf
[INFO]
```

Y aquí podemos ver un ejemplo básico de un informe generado con Surefire Report indicando la cantidad de tests que se ejecutaron con un su tiempo de tasa de éxito y el tiempo tomado.

Publicado el: 2024-10-13 | Versión: 0.0.1-SNAPSHOT

Generado por Maven

Surefire Report

Summary

[Summary] [Package List] [Test Cases]

Tests	Errors	Failures	Skipped	Success Rate	Time
2	0	0	0	100%	11,126

Note: failures are anticipated and checked for with assertions while errors are unanticipated.

Package List

[Summary] [Package List] [Test Cases]

Package	Tests	Errors	Failures	Skipped	Success Rate	Time
com.gestion.eventos	2	0	0	0	100%	11,126

Note: package statistics are not computed recursively, they only sum up all of its testsuites numbers.

com.gestion.eventos

	Class	Tests	Errors	Failures	Skipped	Success Rate	Time
	EventoControllerTest	1	0	0	0	100%	5,578
	GestionProductosApplicationTests	1	0	0	0	100%	5,548

En este otro ejemplo podemos ver un informe generado por el plugin de Jacoco

appmanageevents

appmanageevents

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
cl.talentodigital.appmanageevents.controllers		79 %		75 %	10	33	23	107	9	31	0	6
cl.talentodigital.appmanageevents.services		88 %		100 %	3	18	3	32	3	17	0	3
cl.talentodigital.appmanageevents		37 %		n/a	1	2	2	3	1	2	0	1
cl.talentodigital.appmanageevents.config		98 %		n/a	1	11	1	23	1	11	0	2
cl.talentodigital.appmanageevents.entities		100 %		100 %	0	5	0	20	0	4	0	3
Total	106 of 735	85 %	1 of 8	87 %	15	69	29	185	14	65	0	15

5. Despliegue

Debido a la naturaleza colaborativa de nuestro proyecto y que cada persona puede tener una variedad de condiciones, en sus equipos las cuales, pueden inducir errores o variaciones inesperadas. En nuestro despliegue, hemos decidido homologar el entorno CI/CD con Docker. Esto nos permite, tener un entorno de trabajo común, independiente del sistema operativo o equipo que posea la persona.

Una explicación más detallada con este y otros beneficios puede ser encontrada a continuación:

5.1.1 Consistencia entre Entornos

- **"Funciona en mi máquina"** deja de ser un problema porque Docker, garantiza que el entorno de desarrollo, pruebas y producción sea el mismo.
- Los contenedores, contienen todas las dependencias necesarias, asegurando que la aplicación funcione exactamente igual sin importar el entorno donde se ejecute.

5.1.2 Tiempo de Despliegue Rápido

- Los contenedores, son **livianos y rápidos** de iniciar, permitiendo desplegar aplicaciones y servicios en cuestión de segundos.

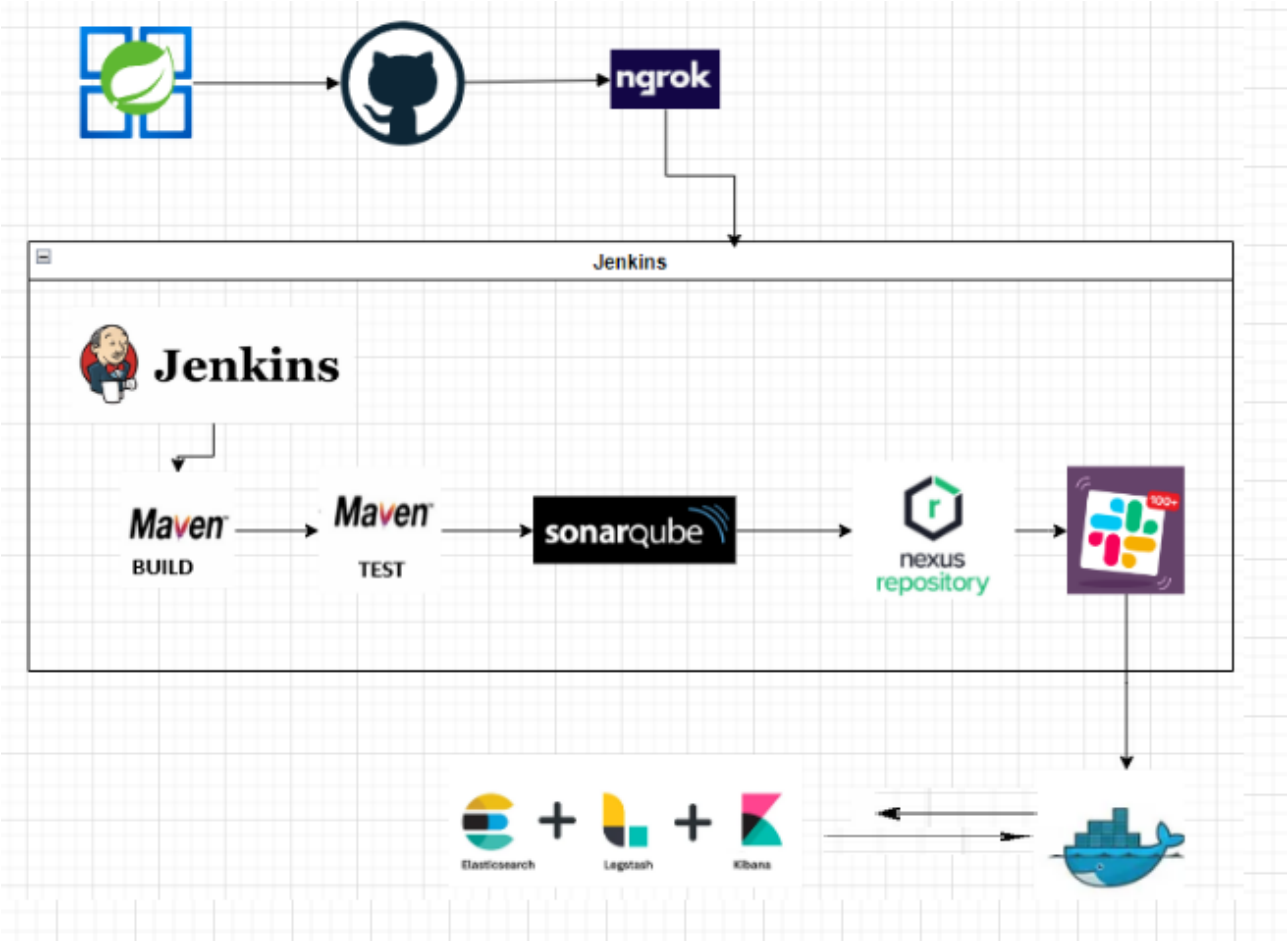
- Facilita despliegues automatizados y frecuentes, esenciales en entornos de CI/CD.

5.1.3 Aislamiento de Aplicaciones

- Cada aplicación o microservicio, se ejecuta en su propio contenedor, aislado del sistema operativo y de otras aplicaciones.
- Esto reduce conflictos de dependencias y facilita la **ejecución paralela** de diferentes versiones, de la misma aplicación para pruebas A/B.

Si bien, tenemos la posibilidad de ejecutar nuestras pruebas manualmente, junto con desplegar nuestra aplicación, es una buena práctica automatizar nuestras tareas.

Este diagrama ilustra un flujo de CI/CD (Integración Continua y Despliegue Continuo) utilizando varias herramientas como Jenkins, Maven, SonarQube, Nexus, Slack, entre otros. A continuación, explicamos cada componente y cómo interactúan:



5.2 Secciones y Componentes del Diagrama

Aplicación Spring Boot (Esquina superior izquierda)

- Esta es la aplicación que se está desarrollando y desplegando.
- La aplicación se encuentra en un repositorio de GitHub, donde el código fuente se mantiene versionado.

Ngrok (exposición del entorno de desarrollo o pruebas)

- **Ngrok** se utiliza para exponer localmente tu aplicación (por ejemplo, una aplicación Spring Boot) a través de una URL pública temporal, lo que facilita pruebas o integraciones remotas.

Jenkins (Servidor de Integración Continua)

- **Jenkins** orquesta las diferentes tareas en este flujo. Cada vez que se realiza un commit en GitHub o un cambio en el repositorio, Jenkins ejecuta un pipeline para automatizar los siguientes pasos.

5.3 Flujo del Pipeline de Jenkins.

1. **Compilación con Maven (Maven BUILD)**
    - Jenkins usa Maven para compilar el proyecto. Se descargan dependencias y se generan los archivos .jar
2. **Pruebas con Maven (Maven TEST)**
    - Se ejecutan las pruebas unitarias o de integración para validar que el código funciona correctamente.
3. **Análisis de Calidad de Código con SonarQube**
    - Jenkins envía el código a SonarQube, para realizar un análisis estático. SonarQube verifica problemas de calidad, deuda técnica y vulnerabilidades.
4. **Publicación en Nexus Repository**
    - Si la compilación y las pruebas son exitosas, el artefacto (por ejemplo, un .jar o .war) se almacena en Nexus Repository, que actúa como un repositorio privado para los artefactos generados.
5. **Notificación en Slack**
    - Una vez completada la ejecución del pipeline, Jenkins envía una notificación al canal de **Slack** para informar al equipo sobre el estado del pipeline (éxito o error).
6. **Despliegue con Docker**
    - Al final del pipeline, el artefacto se puede desplegar utilizando **Docker** para ejecutarlo en un contenedor.

5.4 Interpretación Global del Flujo.

- **Automatización**: Este pipeline, muestra cómo un cambio en el repositorio de GitHub puede desencadenar automáticamente un flujo de trabajo, desde la compilación hasta las pruebas y el despliegue.
- **Control de Calidad**: La integración de **SonarQube**, garantiza que el código mantenga buenos estándares de calidad y se detecten problemas antes de llegar a producción.
- **Artefactos Centralizados**: **Nexus Repository** almacena los artefactos generados, lo que facilita la gestión de versiones y despliegues.
- **Notificaciones en Tiempo Real**: La integración con **Slack,** mantiene al equipo informado sobre el progreso de las tareas del pipeline.
- **Docker para Despliegue**: El uso de **Docker,** permite empaquetar la aplicación con todas sus dependencias, asegurando que funcione de manera consistente en cualquier entorno.

5.5 Instalación de Ngrok y Jenkins.

El servicio de ngrok, nos provee una manera rápida de exponer nuestro entorno de pruebas local. Para ejecutarlo, de manera segura independiente de la plataforma usaremos Docker. Además, gracias a la funcionalidad de Docker podremos ir poniendo todos nuestros servicios en un archivo único llamado docker-compose.

Primero creamos un Dockerfile, que contendrá Jenkins junto con Maven y Java 17

```
FROM jenkins/jenkins:lts
USER root
# Instalar Maven, OpenJDK 17 y wget
RUN apt-get update && \
    apt-get install -y maven openjdk-17-jdk wget && \
    apt-get clean
# Establecer Java 17 como la versión por defecto
RUN update-alternatives --install /usr/bin/java java /usr/lib/jvm/java-17-openjdk-amd64/bin/java 1 && \
    update-alternatives --set java /usr/lib/jvm/java-17-openjdk-amd64/bin/java
USER jenkins
```

Luego creamos un docker-compose, para invocar el servicio que acabamos de crear junto con lo ngrok.

```
version: '3'


services:
  jenkins:
    build:
      context: .
      dockerfile: Dockerfile
    container_name: jenkins
    volumes:
      - jenkins_data:/var/jenkins_home

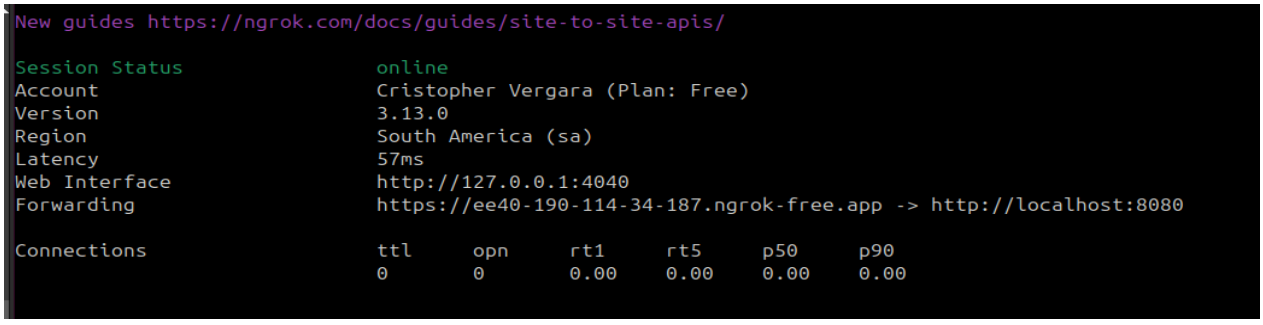

    networks:
      - sonarnet
```

```
  ports:
    - '8080:8080'
    - '50000:50000'
  deploy:
    resources:
      limits:
        memory: 4g
        cpus: '4.0'
 ngrok:
  image: ngrok/ngrok:latest
  container_name: ngrok
  network_mode: "host"  # Usar la red del host
  environment:
    - NGROK_AUTHTOKEN=2jTbuO37kjLJg7PGCCDOHtjEGaC_6KSrg2eTYxj6n4t8CeJZA
  command: http --domain=mammal-ethical-albacore.ngrok-free.app 8080  # dominio fijo gratis de ngrock , expondra el puerto 8080
  depends_on:
    - jenkins  # Espera a que Jenkins esté disponible
```

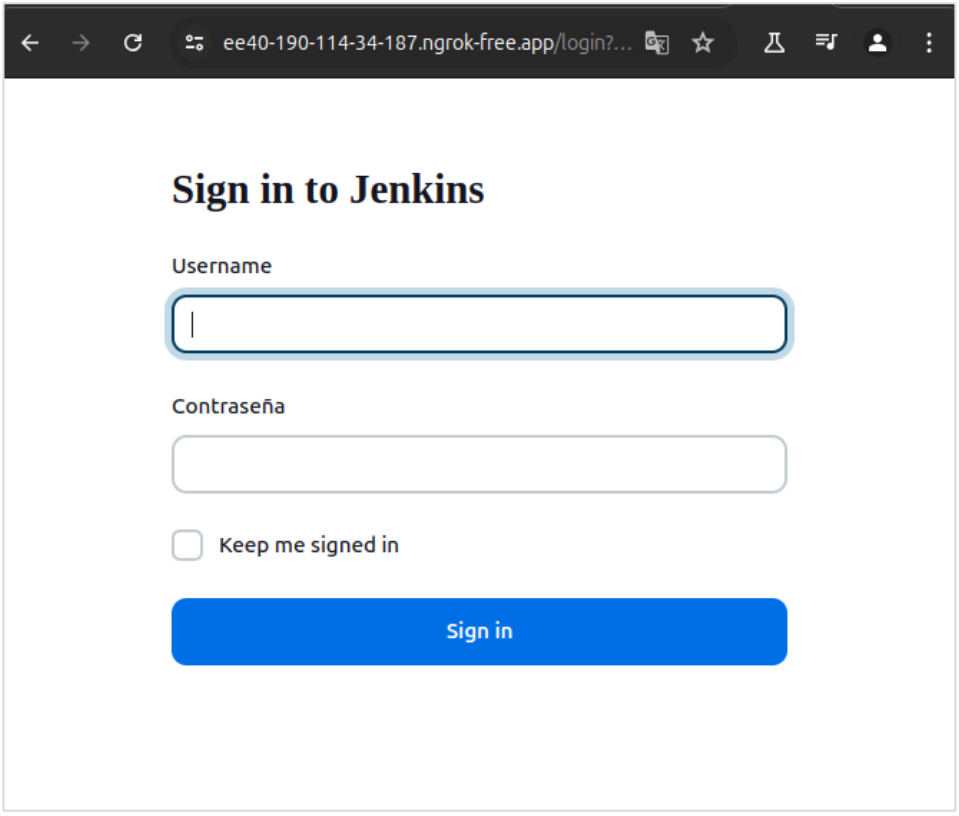
El funcionamiento interno es el siguiente, docker inicializa Jenkins llamando al Dockerfile que contiene sus especificaciones. Jenkins se inicia en el puerto 8080 y luego ngrok expone el puerto 8080 a la web.

Internamente ngrok, ejecuta el siguiente comando para exponer el puerto de Jenkins ngrok http http://localhost:8080

En la siguiente imagen, podemos ver la consola de administracion de ngrok, indicándonos que se encuentra online y exponiendo nuestro localhost a internet.



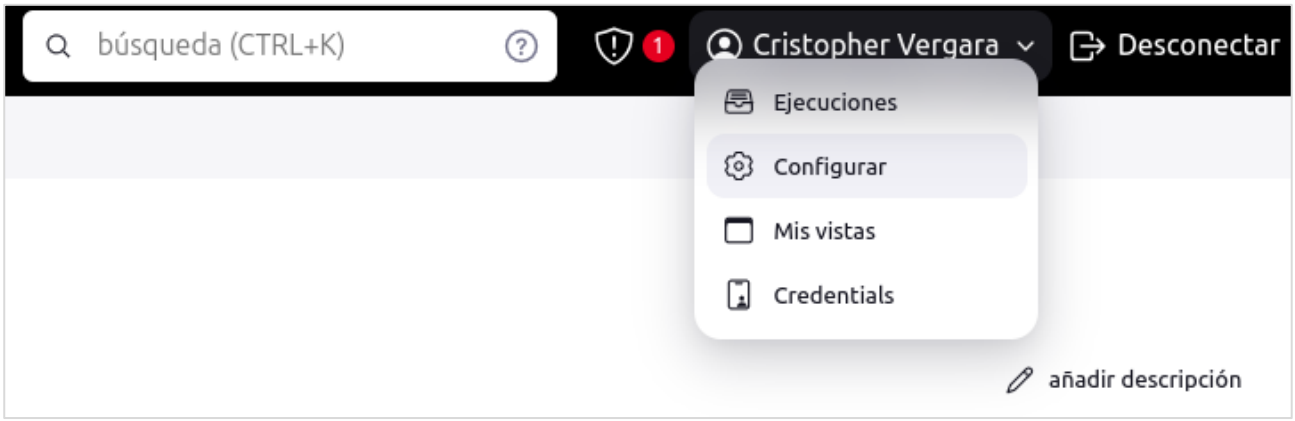
Usando la dirección pública presentada en la línea de comandos podemos acceder a nuestro Jenkins.



5.6. Generar un token en Jenkin para permitir conexiones entrantes

Ahora se requiere crear un API Token con el fin de permitir conexiones entrantes en nuestro Jenkins.

Para eso se entra al apartado de configuración.



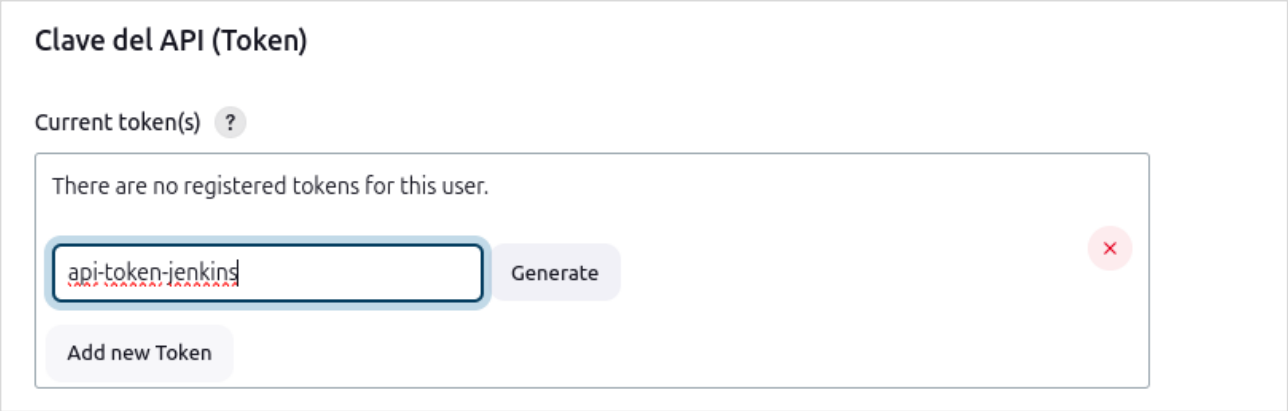
Se nombra el API token con un nombre y se presiona el botón de generate.

Se requiere guardar este Token para el paso siguiente.

5.7. Generación de un Jenkinsfile.

Un Jenkinsfile, es un archivo de configuración utilizado por Jenkins, para definir el flujo de trabajo de un pipeline como código. Permite describir, de manera declarativa o programática, todas las etapas del pipeline que Jenkins debe seguir para construir, probar y desplegar una aplicación

Antes de crear un pipeline en Jenkins, crearemos un Jenkinsfile en la raíz del proyecto con el fin gatillar algunas acciones




```groovy
pipeline {
    agent any


    stages {
        stage('Checkout') {
            steps {
                // Clona el repositorio desde GitHub
                git 'https://github.com/colombo1986/BASIC-CRUD-thymeleaf.git'
            }
        }

        stage('Build') {
            steps {
                // Compila el proyecto usando Maven
                sh 'mvn clean compile'
            }
        }

        stage('Test') {
            steps {
                // Ejecuta las pruebas unitarias
                sh 'mvn test'
            }
        }

```

```
        stage('Package') {
            steps {
                // Empaqueta la aplicación
                sh 'mvn package'
            }
        }

        stage('Deploy') {
            steps {
                // Despliega la aplicación (ajusta según tu entorno)
                sh 'echo "Deploying application..."'
            }
        }
    }


```

```

    post {
        always {
            // Publica los resultados de la prueba
            junit 'target/surefire-reports/*.xml'
        }
        success {
            // Notifica en caso de éxito
            echo 'Build and tests succeeded!'
        }
        failure {
            // Notifica en caso de fallo
            echo 'Build or tests failed.'
        }
    }
}
```

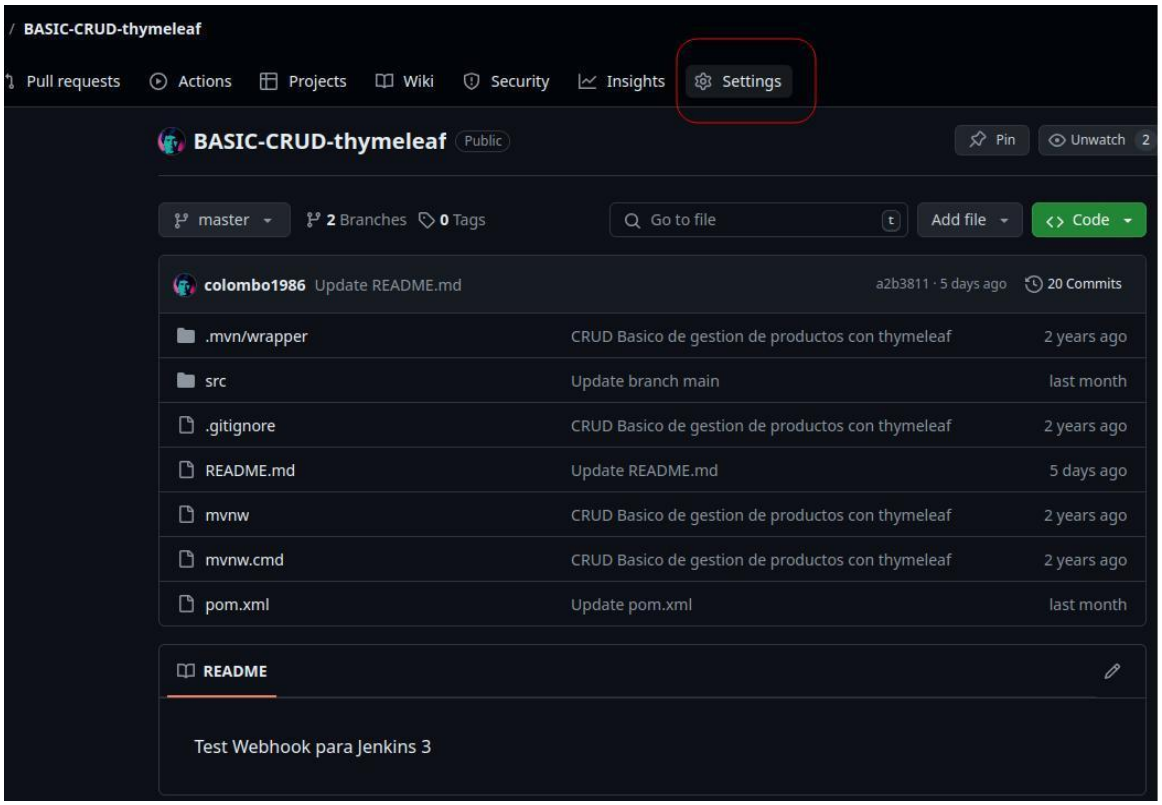
5.8. Configuración de GitHub.

Un webhook en GitHub, es una manera de notificar automáticamente a un servicio externo, cuando ocurren ciertos eventos en un repositorio. Los webhooks, permiten la integración continua entre GitHub y otras herramientas o plataformas, enviando datos en tiempo real cada vez que sucede un evento, como un push, pull request o creación de una issue.

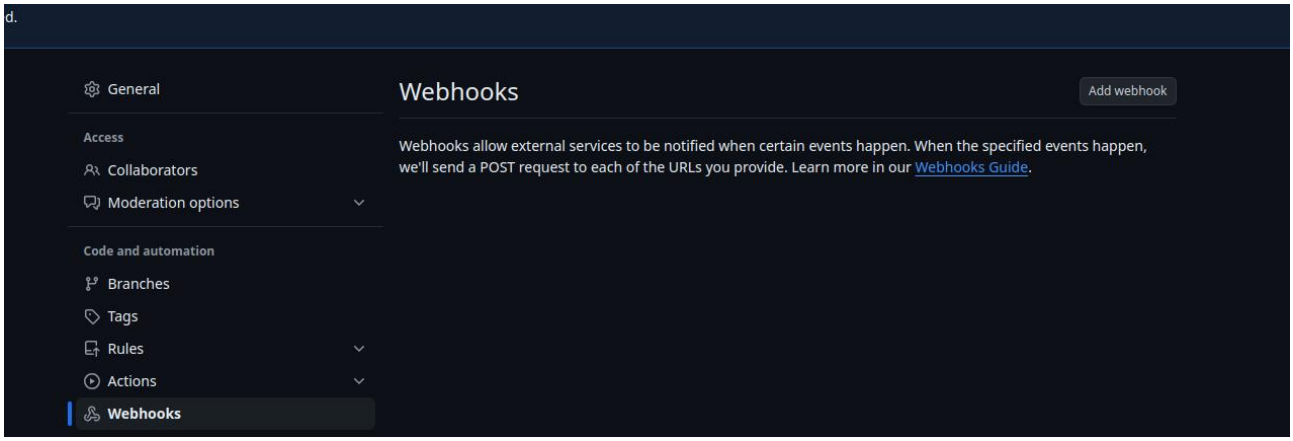
Lo que buscamos al crear un webhook, es que cada vez que efectuemos cambios en nuestro código y estos sean subido a GitHub, se gatille una acción de notificación que

pasará por ngrok y llegará a nuestro Jenkins para inicializar las acciones que definimos en nuestro Jenkinsfile.

Para generar un webhook en nuestro repositorio, debemos ingresar al apartado de settings.



Desde settings, se requiere entrar a Webhooks y presionar en el botón Add webhook.



Copiamos la dirección, proporcionada por ngrock para agregarla.

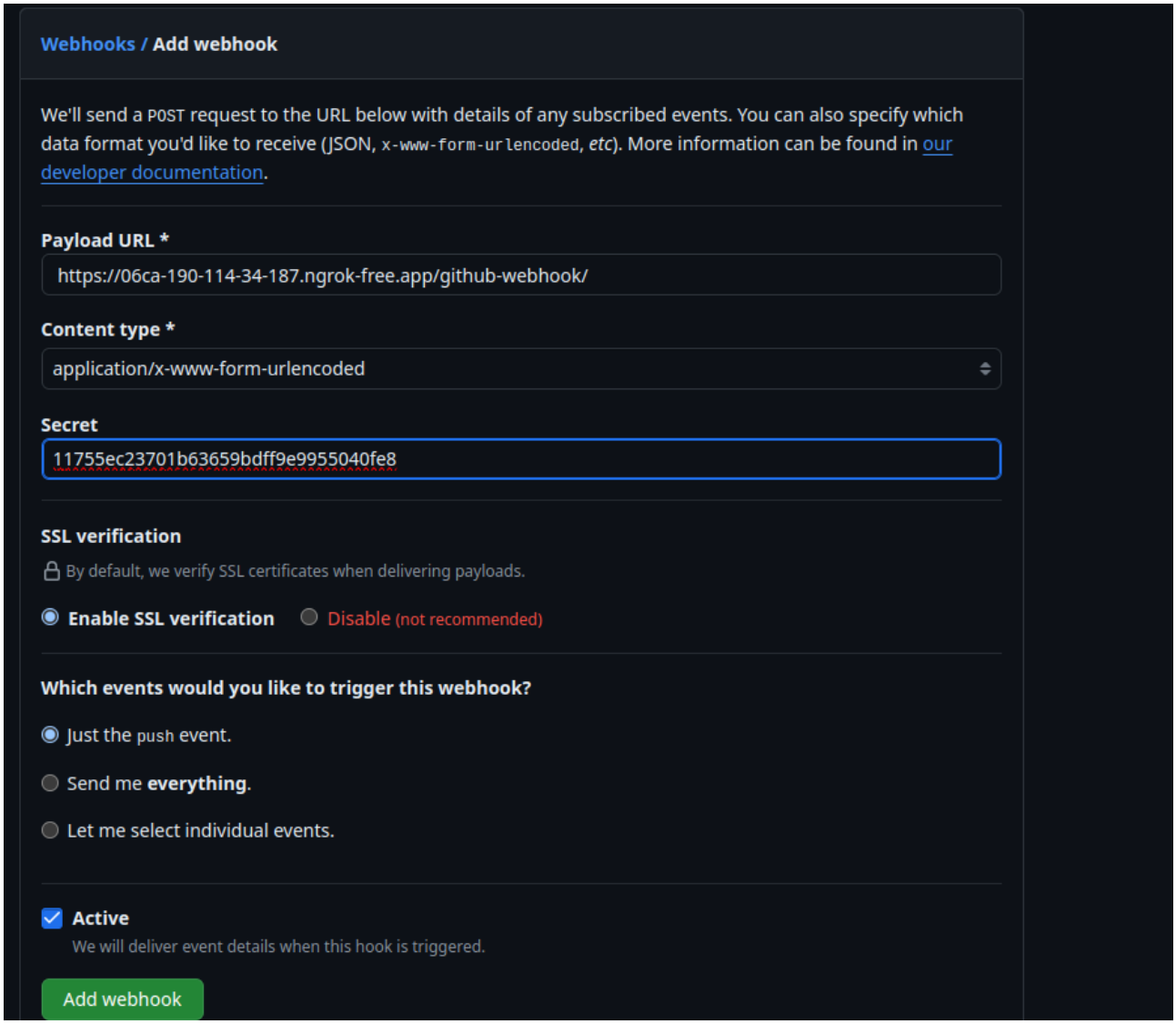
Al final de la url agregamos /github-webhook/ .

```
New guides https://ngrok.com/docs/guides/site-to-site-apis/

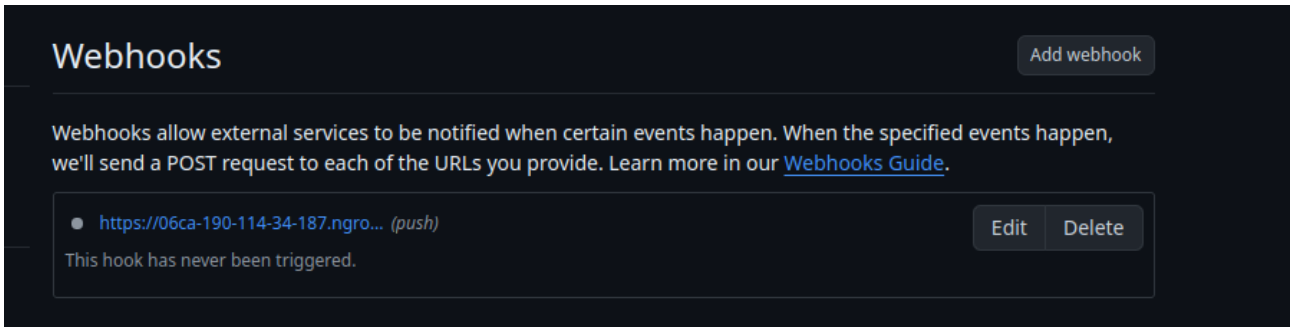
Session Status                online
Account                       Cristopher Vergara (Plan: Free)
Version                       3.13.0
Region                        South America (sa)
Latency                       57ms
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://ee40-190-114-34-187.ngrok-free.app -> http://localhost:8080

Connections                   ttl     opn     rt1     rt5     p50     p90
                              54      0       0.00    0.00    0.51    30.38
```

En el apartado de secret, va el API key que generamos en Jenkins en el paso **5.7.**



Una vez creado el Webhook, 0se ve como en la siguiente imagen:

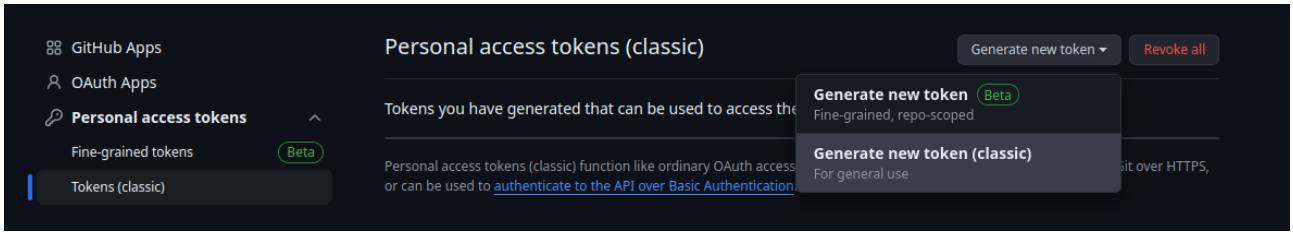


Con la creación del API Key de Jenkins y la agregación de este, junto con la url pública generada con ngrok en el Webhook de GitHub, se autentica la comunicación desde Jenkins hacia GitHub.

Ahora, se requiere autenticar desde GitHub hacia Jenkins, por lo que se debe generar otro Token desde Github.

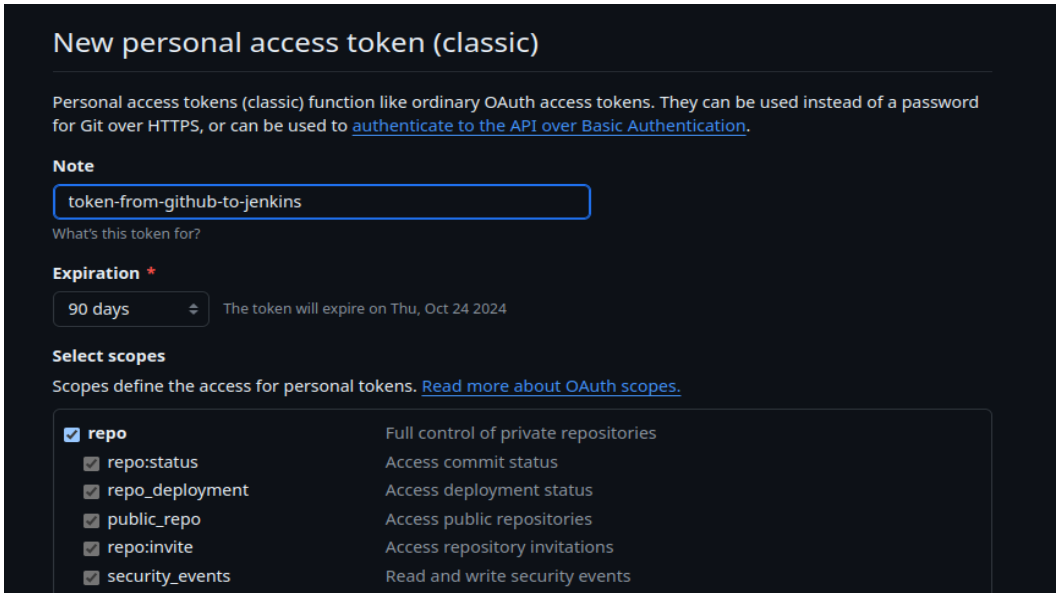
Para esto, ingresamos a la url https://github.com/settings/tokens

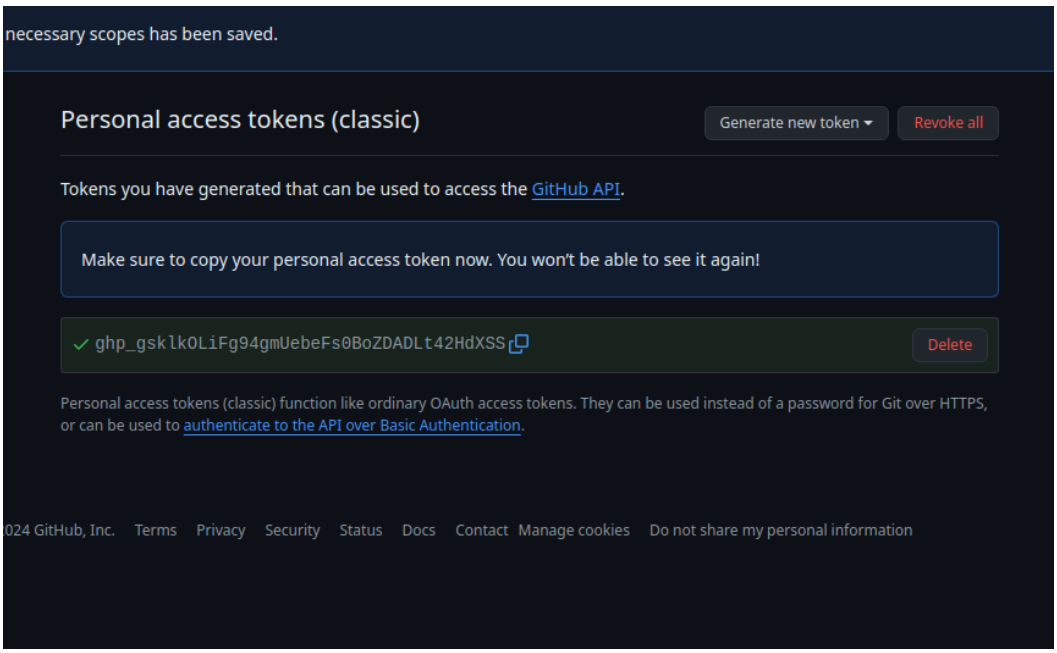
Ingresar a la opción para generar un Token clásico para uso general.



Le asignamos permisos sobre nuestros repositorios, junto con la duración y presionamos sobre el botón Generate token al final de la página.

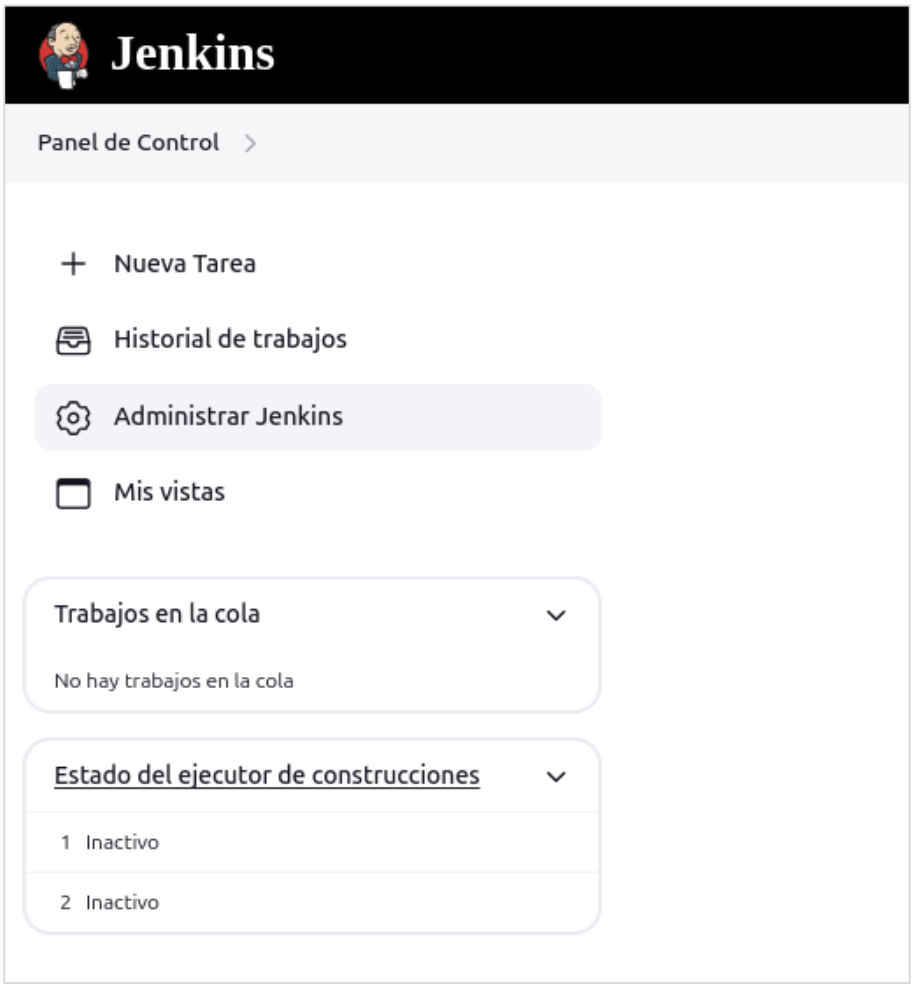
Se debe guardar este token, ya que no hay manera de recuperarlo después.



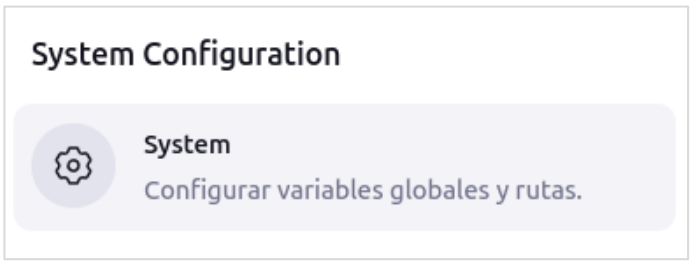


5.9. Configurar Jenkins.

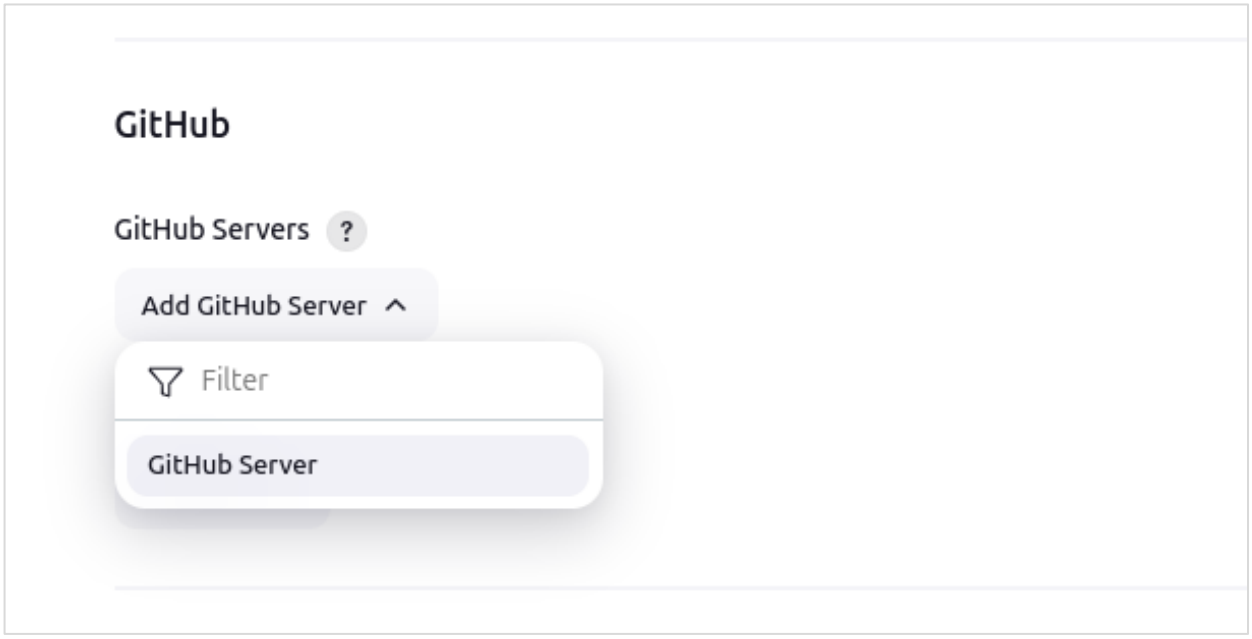
Ahora, ingresamos el token de GitHub en Jenkins, para eso vamos a Administrar Jenkins en el menú lateral en el apartado Administrar Jenkins



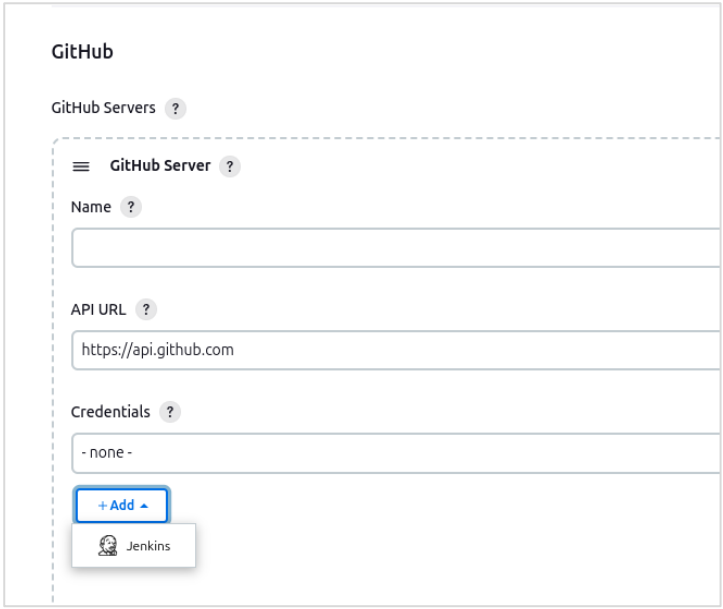
Luego, vamos a System Configuration



En System y nos posicionamos, sobre el apartado de GitHub Servers



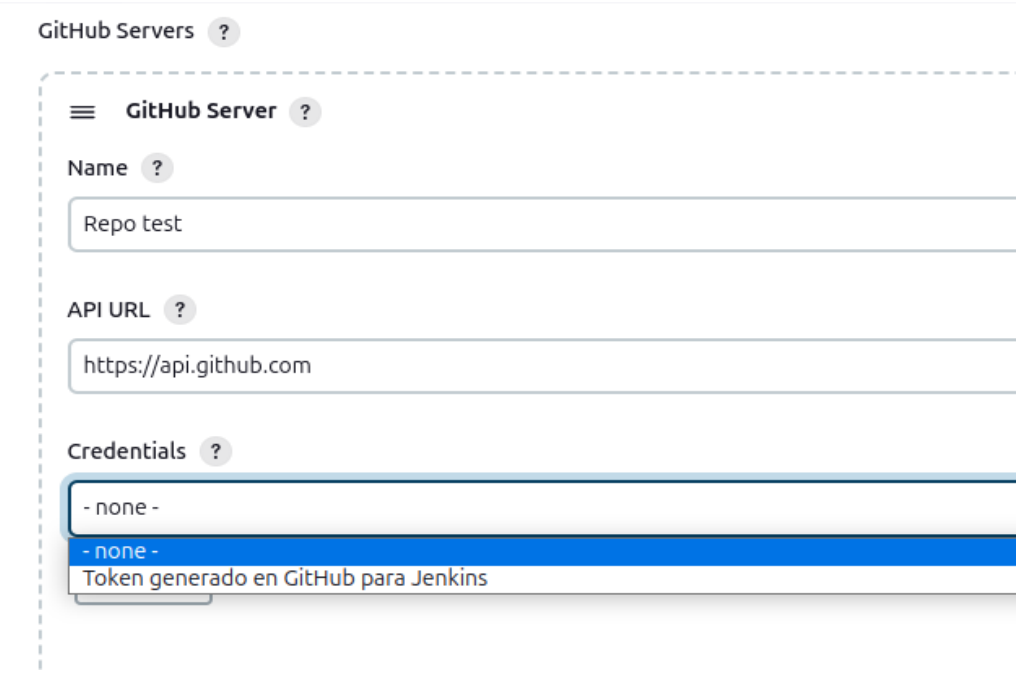
Vamos a agregar credenciales, con el botón +Add y luego sobre Jenkins



Acá, seleccionamos que el tipo sea secret text y en Secret pegamos el Token generado en GitHub, en el paso previo.



Ahora podemos ver el token, que agregamos como credencial para autenticar en GitHub

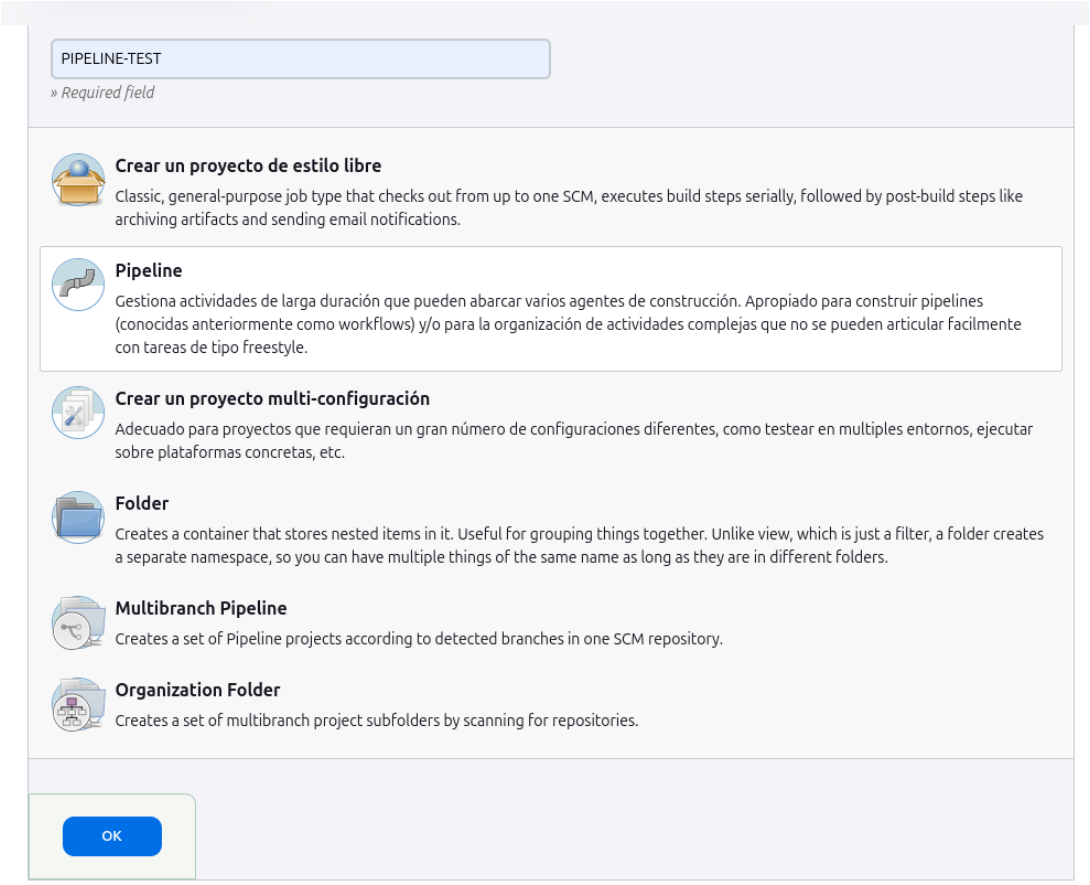


5.10. Crear un Pipeline en Jenkins.

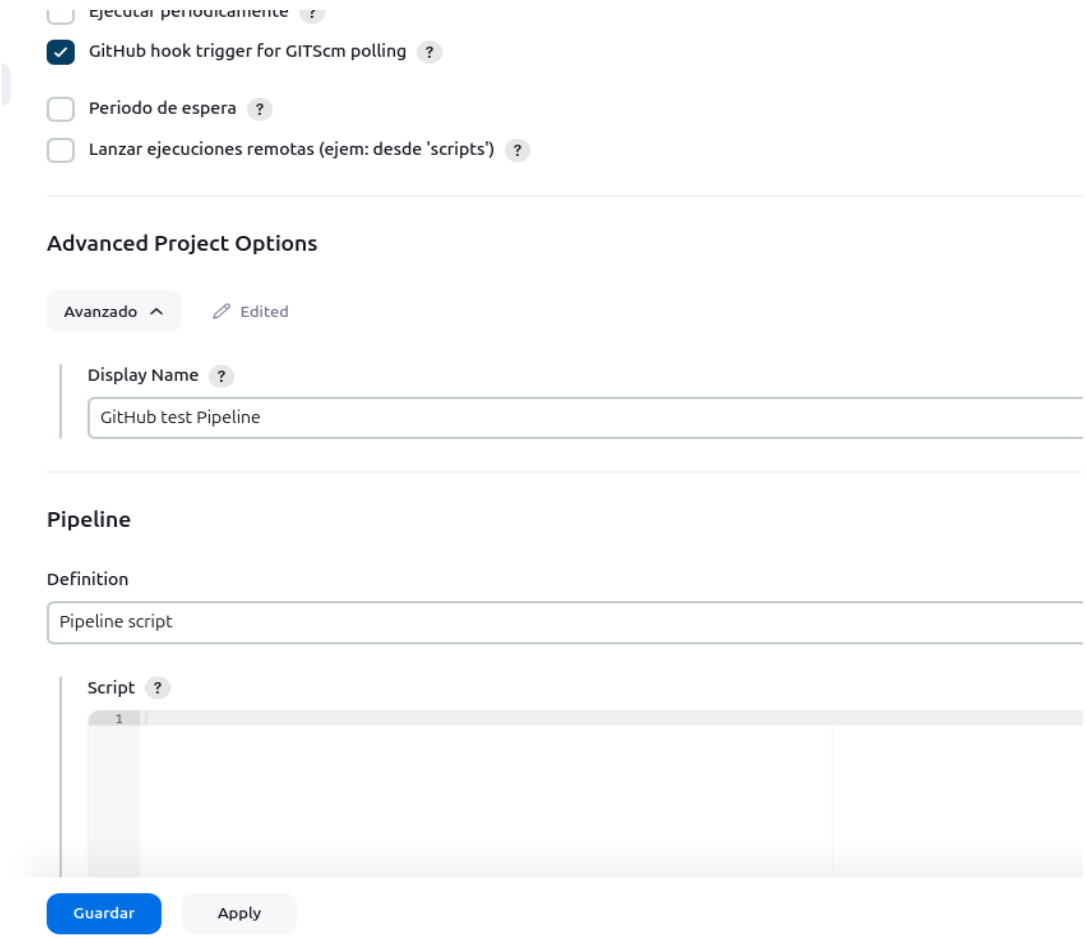
Nos situamos en la pantalla principal de Jenkins y luego en Create Job→ Create Pipeline



Le asignamos un nombre y presionamos en OK



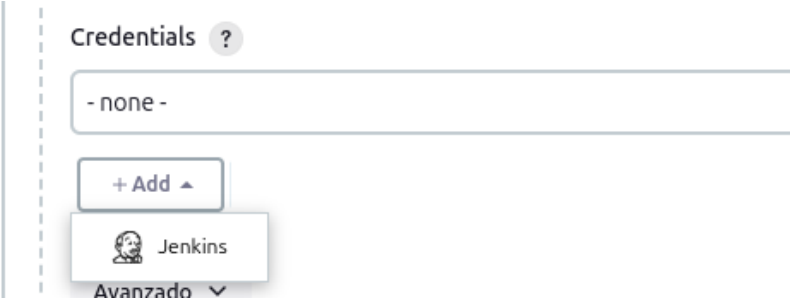
En el apartado Build Triggers, se debe marcar la casilla: GitHub hook trigger for GITScm. Esta opción permitirá que esta compilación se ejecute, después de que el Webhook envíe la solicitud POST.



Presionamos en guardar y vamos al menú de avanzado, donde especificamos la url de nuestro proyecto y el origen desde git



Abajo de credentials, se encuentra un botón para agregar nuestro Token de GitHub

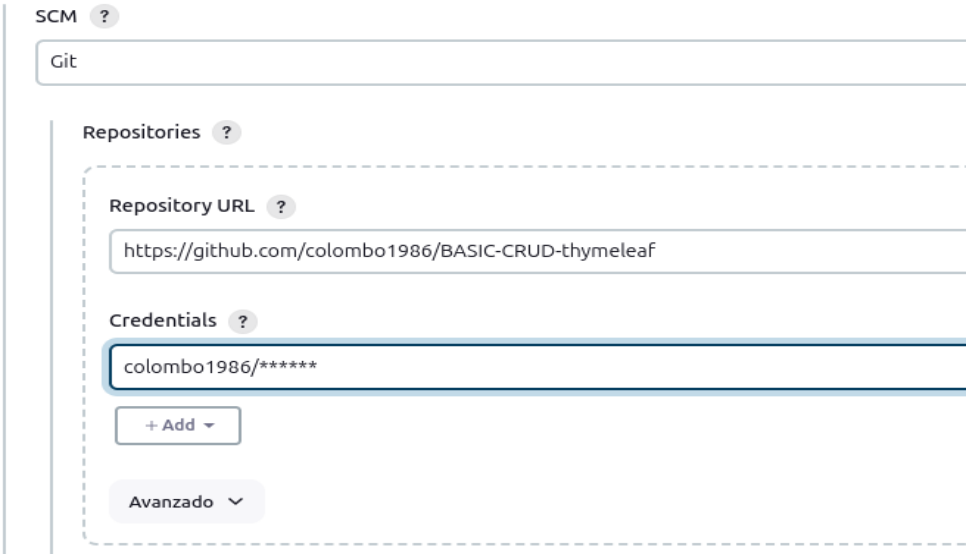


En el tipo, ingresamos usuario y password.

Ingresamos nuestro usuario y en el apartado de Password, volvemos a ingresar el API Token de GitHub, de este modo podremos ingresar a GitHub directamente desde Jenkins



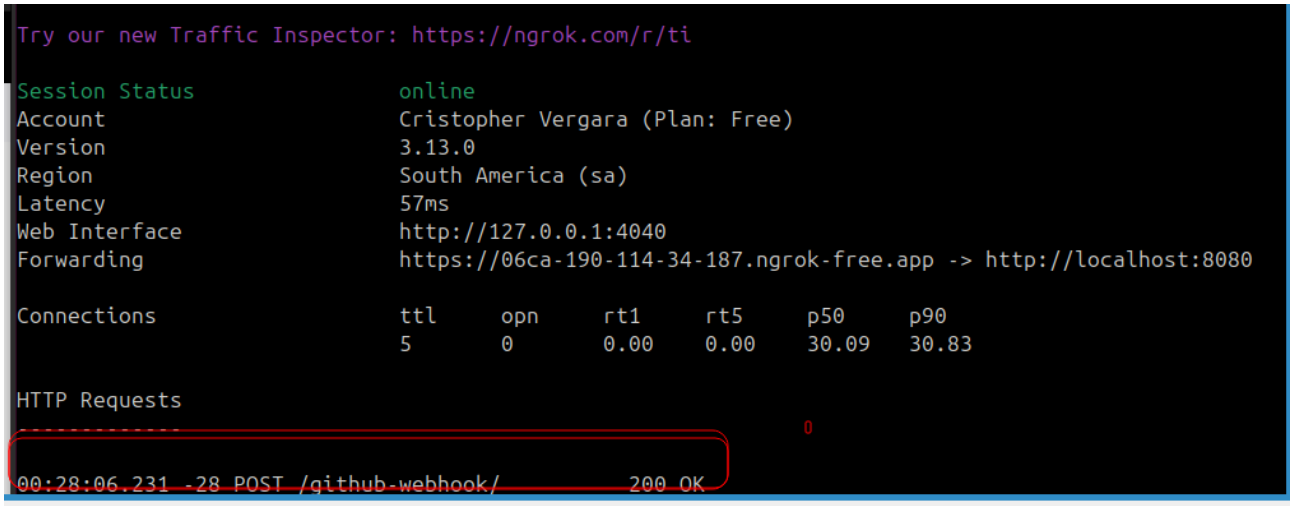
Ahora en el apartado de credentials, nos debería aparecer nuestro usuario de GitHub



5.11. Crear un Push en el repositorio de GitHub y accionar el Pipeline.

Enviamos un salto de línea sobre el JenkinsFile, con el fin de hacer un push en nuestro repositorio y verificar el Pipeline.

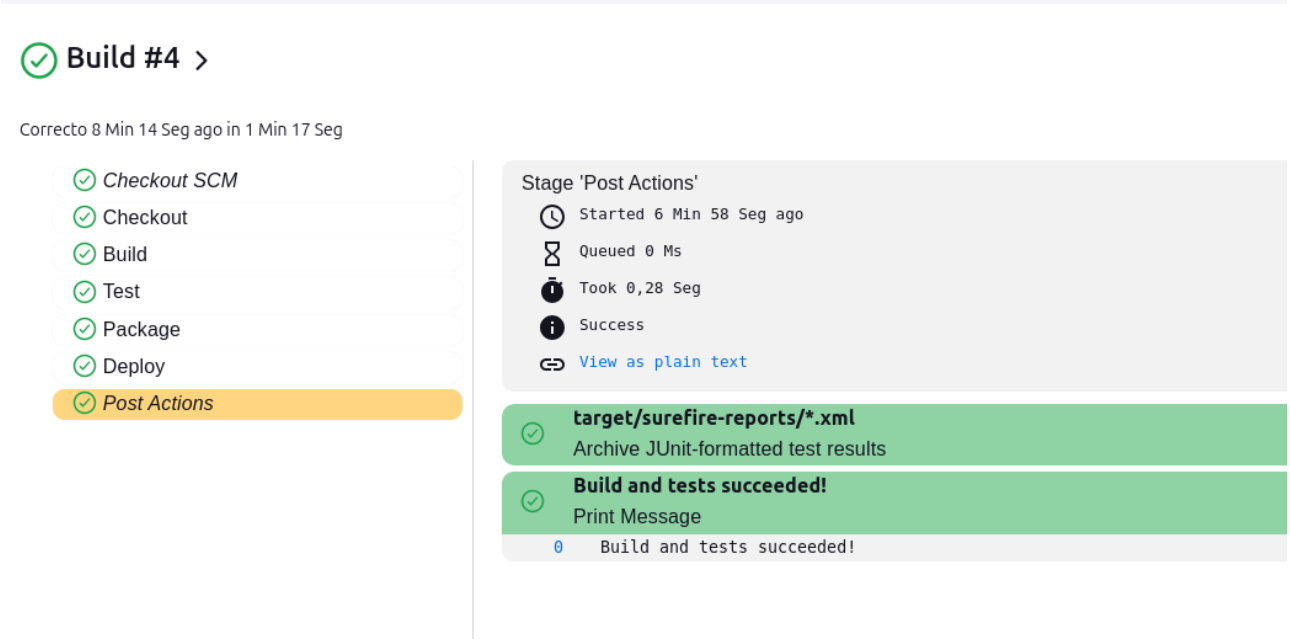
Vemos la consola de ngrok, para verificar la conexión entrante desde GitHub hacia Jenkins



Verificamos el Pipeline recién creado en Jenkins



Finalmente, verificamos la vista de Pipeline Console para verificar que todas las etapas salieron bien.



5.12. Integrar SonarQube

Antes de integrar esta herramienta, responderemos la siguiente pregunta:

¿Qué es SonarQube?

SonarQube es una plataforma de análisis de código, que permite evaluar la calidad del software mediante el análisis estático del código fuente. Detecta errores, vulnerabilidades, deuda técnica y malas prácticas, proporcionando métricas que ayudan a los desarrolladores y equipos de DevOps a mejorar la calidad y seguridad del software de forma continua.

SonarQube, es compatible con más de 20 lenguajes de programación, como Java, Python, JavaScript, C#, PHP, y más. Se integra fácilmente con herramientas de CI/CD como Jenkins, GitHub Actions, GitLab CI, Azure DevOps, entre otras, y genera informes de calidad con métricas detalladas.

¿Por qué es una buena práctica integrar SonarQube?

Control y Mejora de la Calidad del Código

- Detecta código duplicado, variables no utilizadas, métodos complejos, y malas prácticas.

- Genera informes sobre mantenibilidad del código y ayuda a mejorar la estructura y la calidad del proyecto.
- Asegura que las nuevas funcionalidades o cambios, no introduzcan degradaciones en el código existente.

Detección Temprana de Errores

- Al integrarse en un pipeline CI/CD, SonarQube analiza cada cambio realizado en el código y detecta errores de forma temprana.
- Detectar errores en las primeras etapas del desarrollo, evita que se conviertan en problemas críticos en producción.

Asegura la Seguridad del Código

- SonarQube, identifica vulnerabilidades y fallas de seguridad como SQL Injection o XSS.
- Ayuda a los desarrolladores a cumplir con estándares de seguridad, como OWASP Top 10.

Monitoreo y Reducción de Deuda Técnica

- La deuda técnica, es el esfuerzo necesario para corregir problemas de calidad en el código. SonarQube, mide la deuda técnica, proporcionando una estimación del trabajo necesario para mejorar el proyecto.
- Reducir la deuda técnica, garantiza que el código sea más fácil de mantener y escalar.

Transparencia y Colaboración

- Al integrar SonarQube, todo el equipo tiene acceso a los informes de calidad del código, fomentando una cultura de responsabilidad compartida.
- Mejora la colaboración entre equipos de desarrollo y DevOps, al automatizar el análisis de calidad en cada cambio del código.

Cumplimiento de Normativas y Estándares

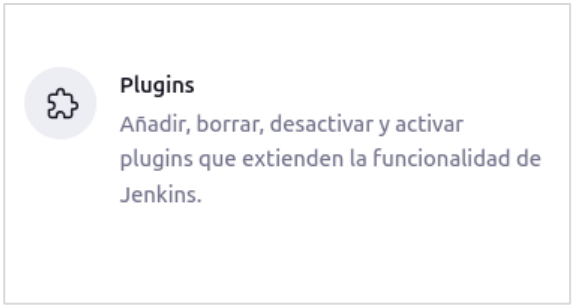
- SonarQube ayuda a cumplir con normativas internas y estándares internacionales, como OWASP, SANS, y reglas de calidad de código definidas por la organización.
- Mejora la gobernanza del software mediante métricas objetivas y procesos automatizados.

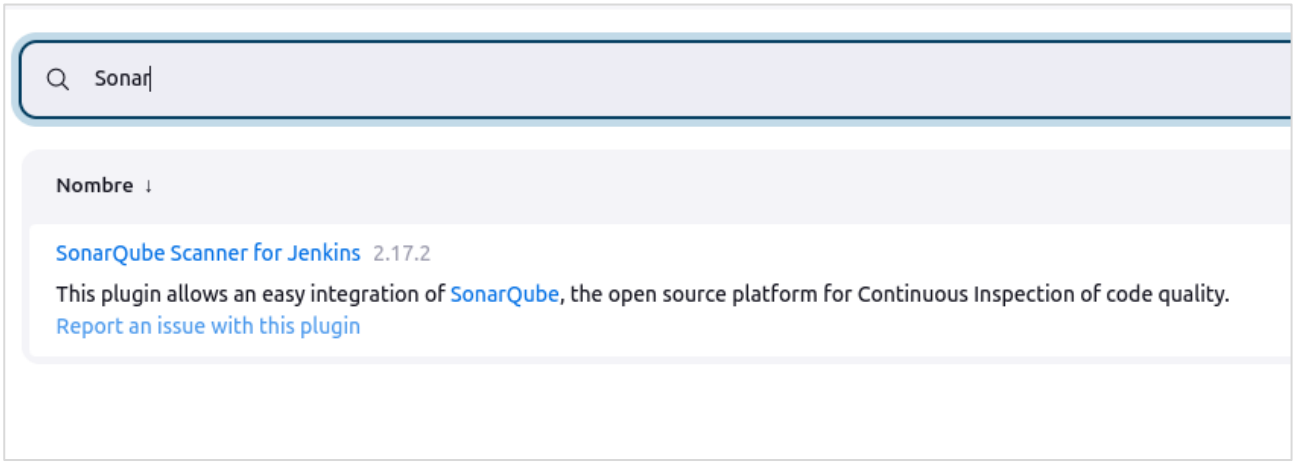
Integración Sencilla con CI/CD

- SonarQube, se integra con herramientas de automatización como Jenkins, GitLab CI, GitHub Actions, y otras.
- En un pipeline CI/CD, SonarQube puede analizar el código en cada push o pull request y evitar que el código se despliegue si no cumple los estándares de calidad.

Para agregar SonarQube en Jenkins debemos ir a:

Administrar Jenkins → Plugins →Instalar el Plugin de SonarQube en Jenkins





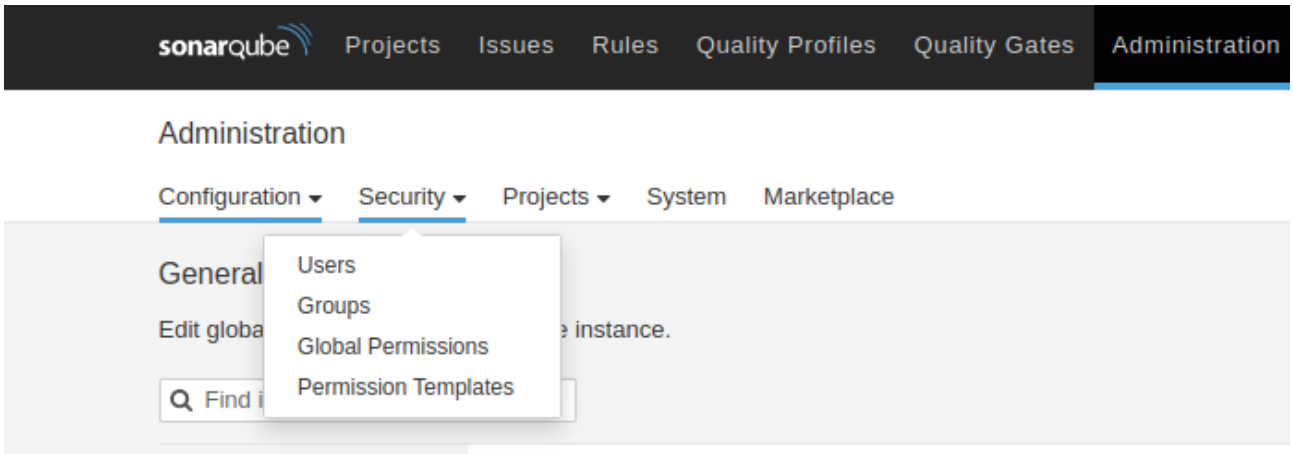
Junto con esto en nuestro Jenkinsfile agregaremos el servicio de SonarQube junto con una base de datos postgres necesaria para su funcionamiento

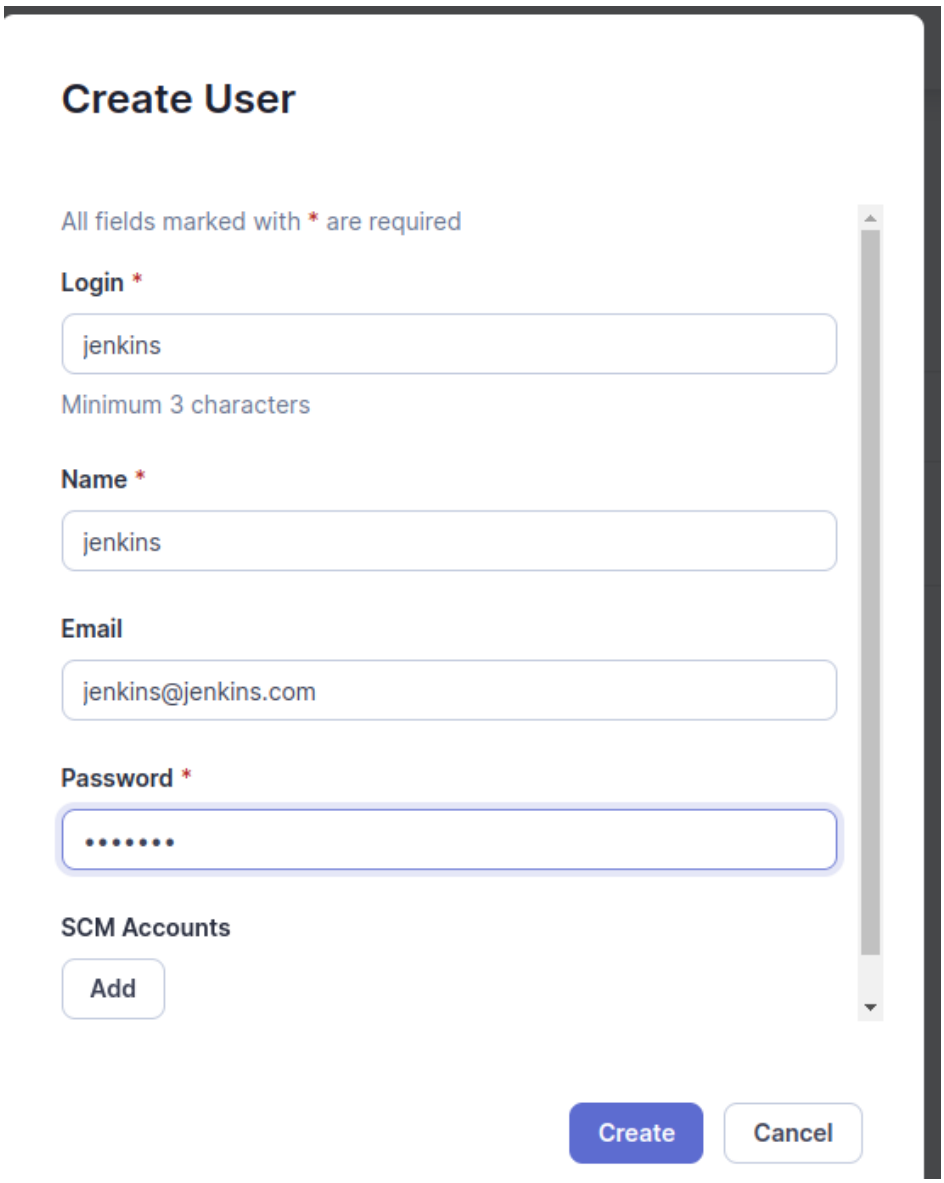
```
sonarqube:
  image: sonarqube:latest
  container_name: sonarqube
  volumes:
    - sonarqube_conf:/opt/sonarqube/conf
    - sonarqube_data:/opt/sonarqube/data
    - sonarqube_extensions:/opt/sonarqube/extensions
    - sonarqube_bundled-plugins:/opt/sonarqube/lib/bundled-plugins
  networks:
    - sonarnet
  ports:
    - '9000:9000'
  environment:
```

```
    - sonar.jdbc.username=sonar
    - sonar.jdbc.password=sonar
    - sonar.jdbc.url=jdbc:postgresql://db:5432/sonar

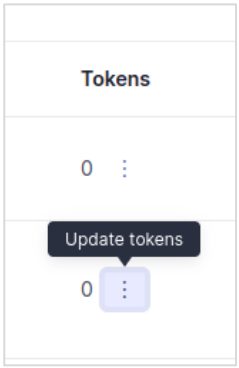
db:
  image: postgres
  container_name: db
  networks:
    - sonarnet
  environment:
    - POSTGRES_USER=sonar
    - POSTGRES_PASSWORD=sonar
  volumes:
    - postgresql:/var/lib/postgresql
    - postgresql_data:/var/lib/postgresql/data
```

Ahora generamos un usuario de SonarQube, para esto ingresamos a http://localhost:9000 y creamos un usuario nuevo.

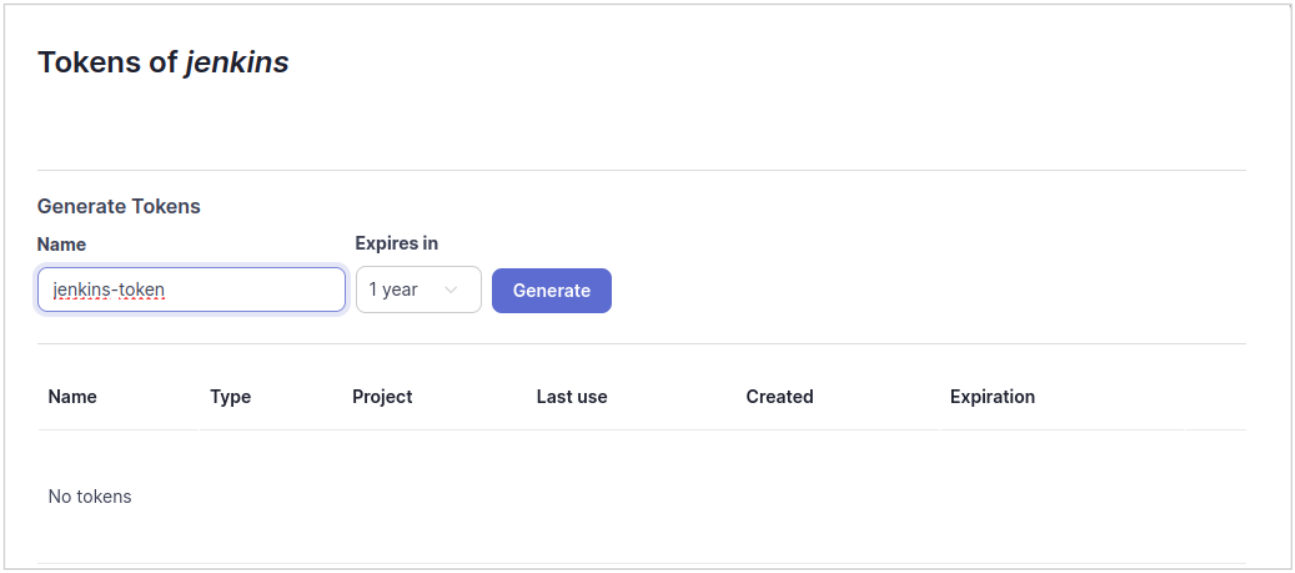




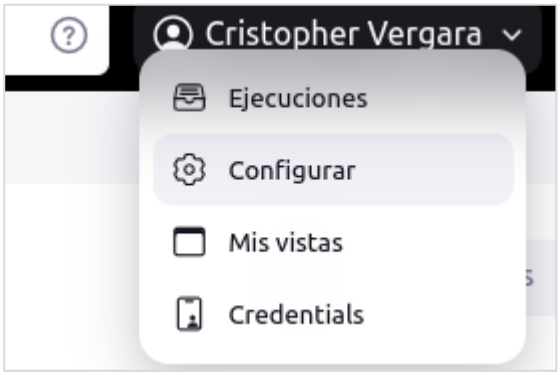
Generamos un token para el usuario recién creado para que este pueda autenticarse en Jenkins



Importante guardar el token, para su uso posterior en Jenkins

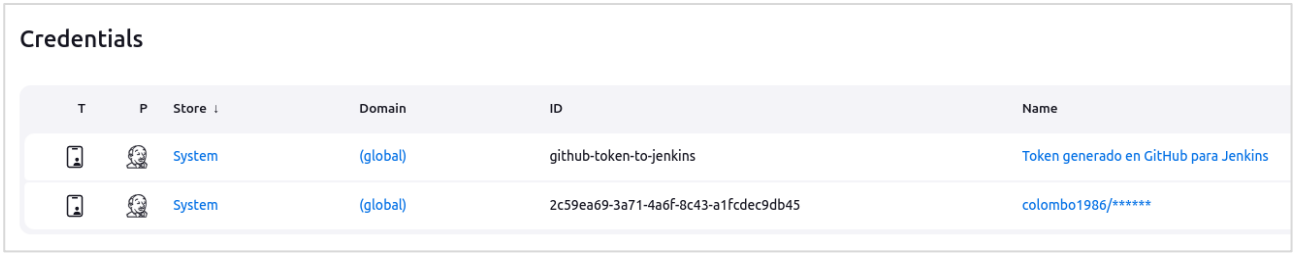


Ahora vamos a guardar el token de SonarQube en Jenkins, debemos ir al menú de Configurar→Credentials

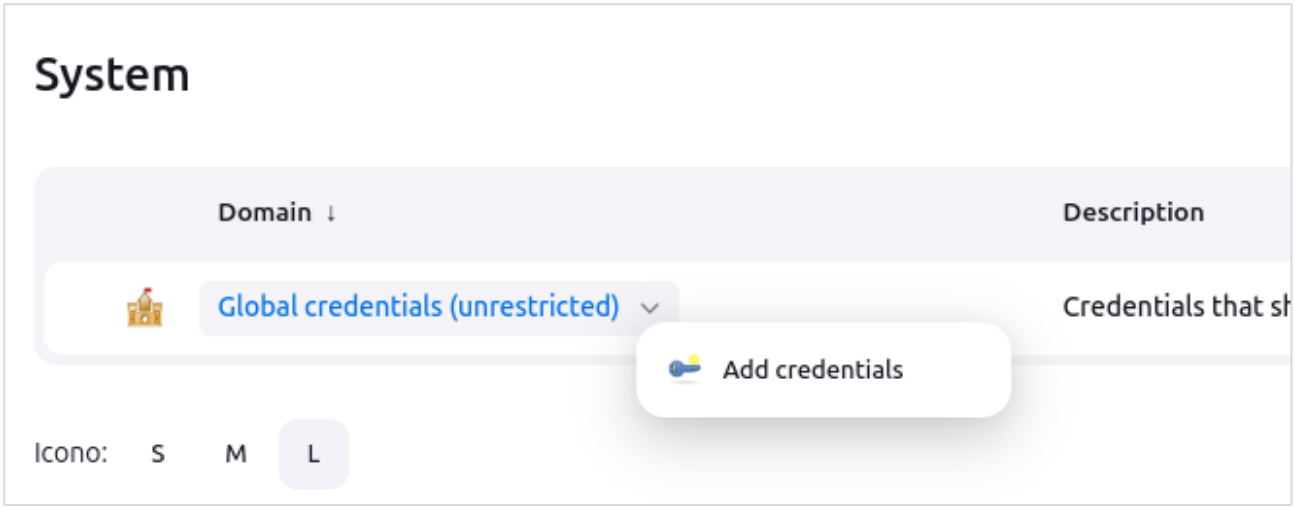




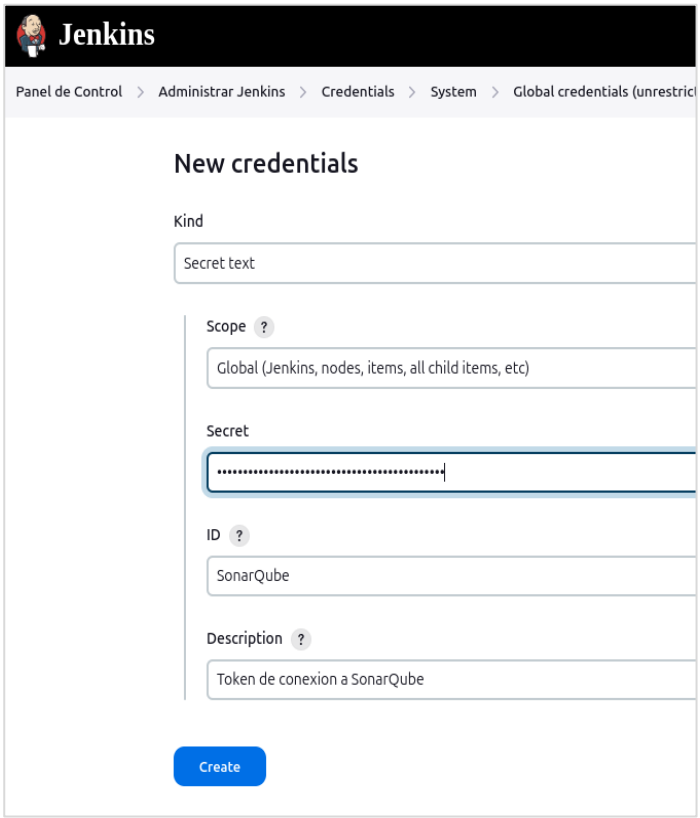
Credentials →System →Click en cualquiera



Se llegará a este menú y hacer click en Add credentials

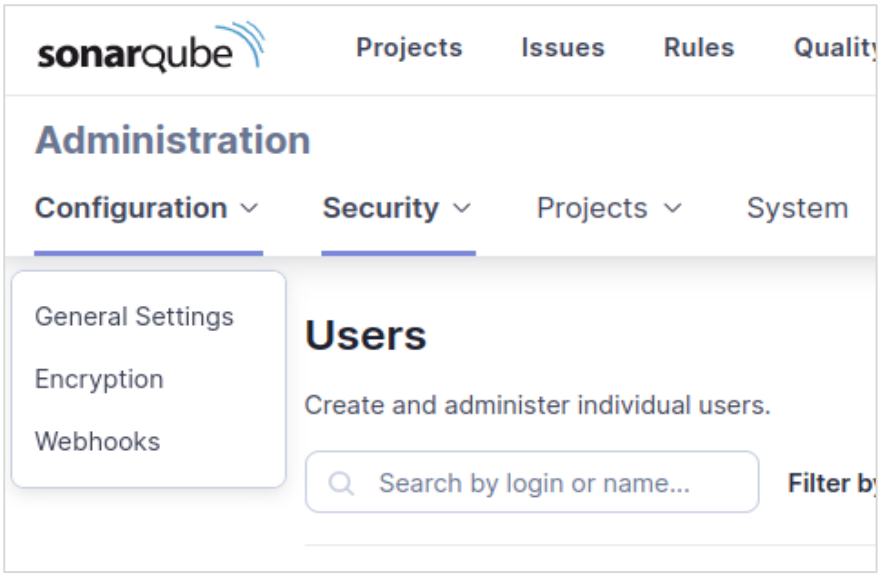


Agregar las credenciales de SonarQube como se ve en la imagen

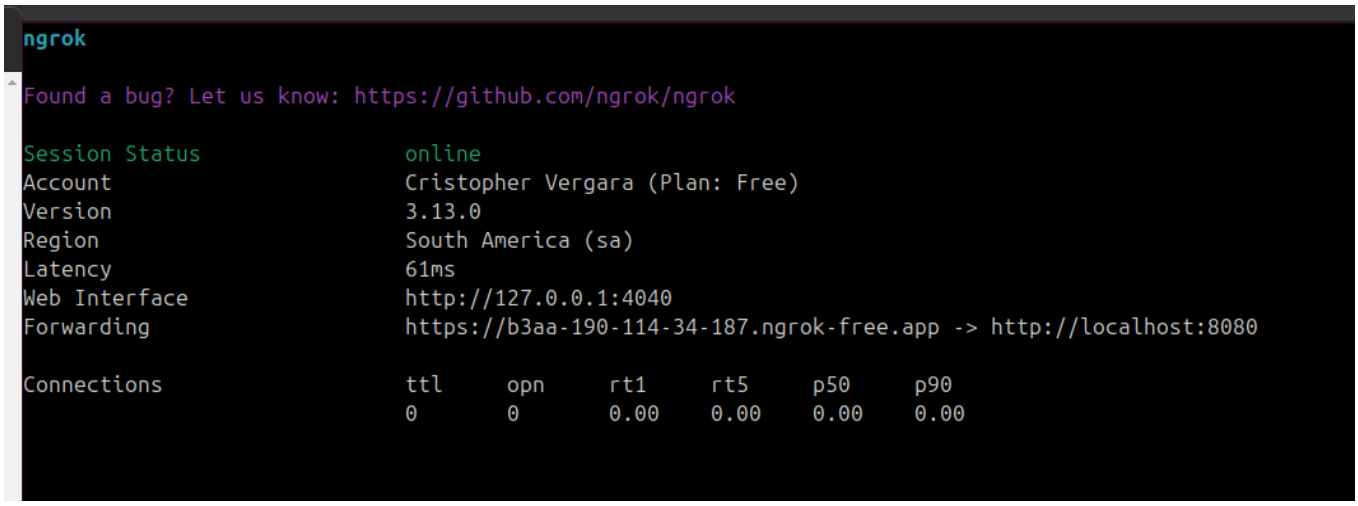


Volvemos a SonarQube, para crear un WebHook en Configuration→Webhooks

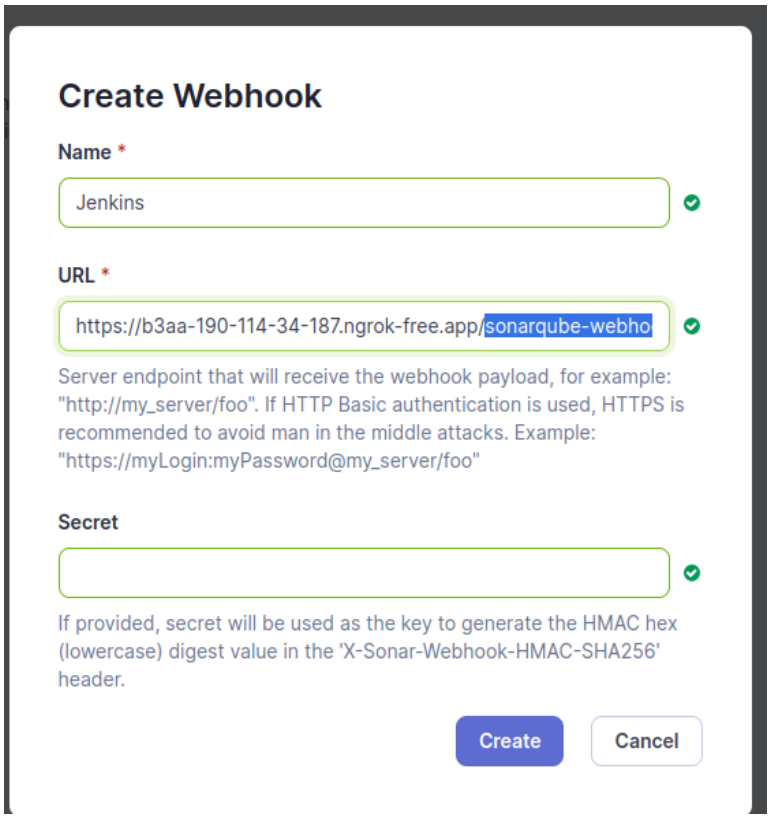
Esto es necesario, para que cuando termine de realizar el proceso de scan y envíe un aviso a Jenkins y puede continuar con el funcionamiento normal de nuestro pipeline, sino, la pipeline continua, continua de manera indefinida hasta que finaliza error de timeout.



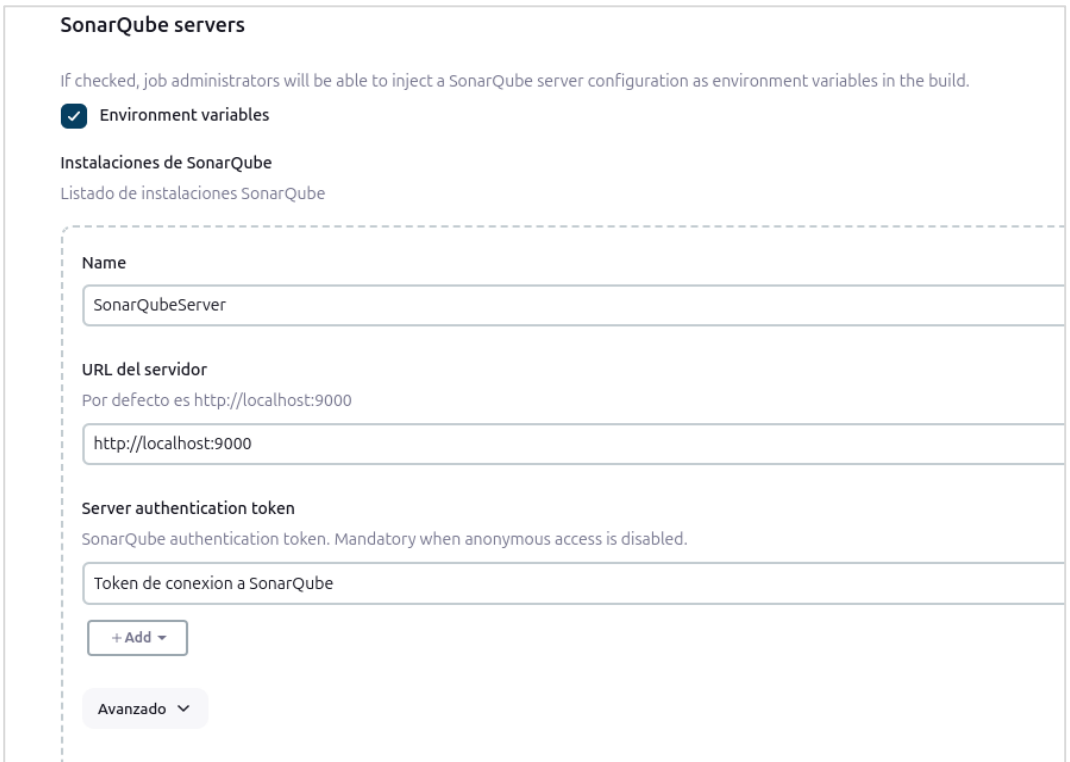
Para esto, usaremos la dirección proporcionada por ngrok.



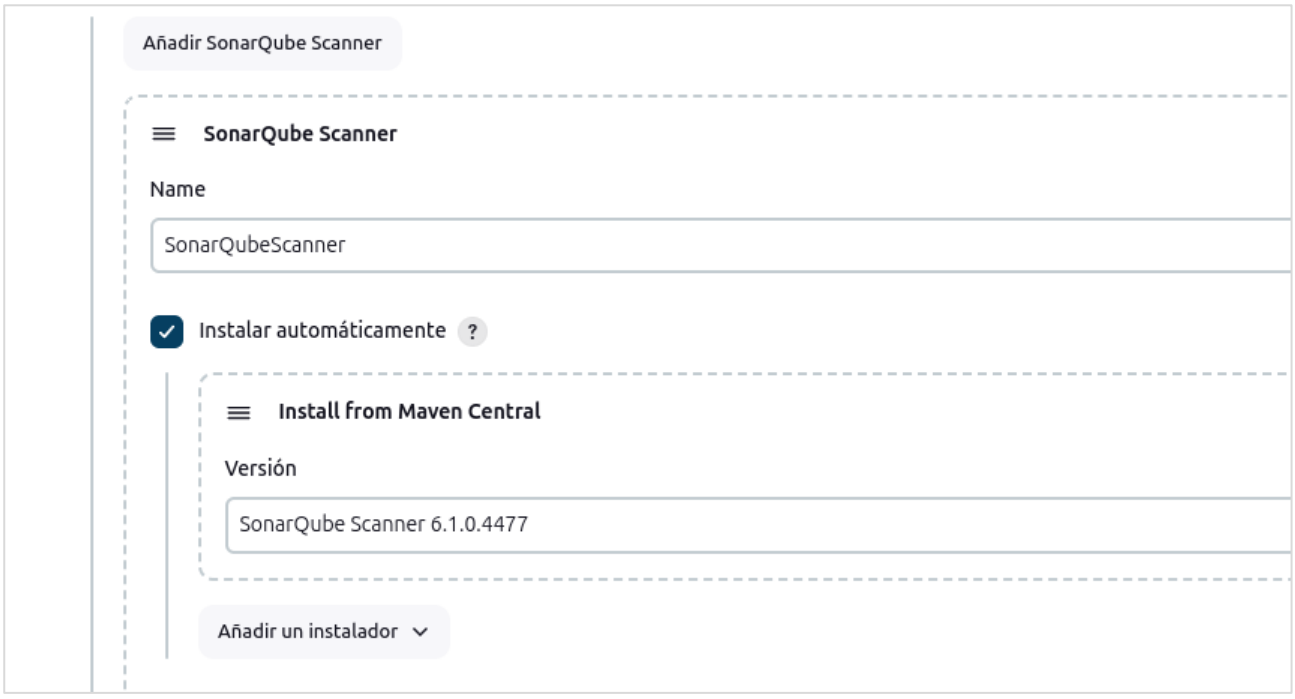
Al final de la url, se debe agregar /sonarqube-webhook/



Ahora volvemos a Jenkins, para terminar de configurar el plugin de SonarQube, de debe ir a Panel de Control →Administrar Jenkins→SonarQube servers



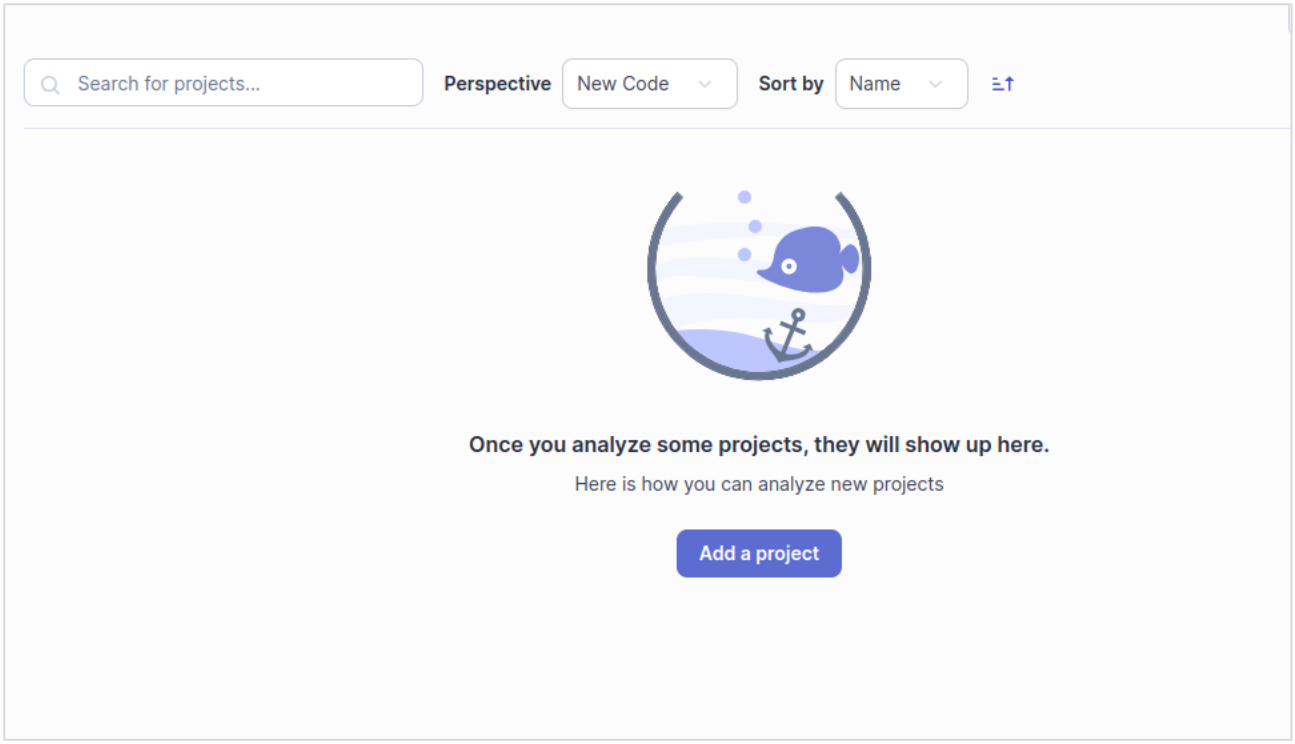
Luego nos vamos a Panel de Control→Administrar Jenkins→Tools y configuramos el escáner de SonarQube.



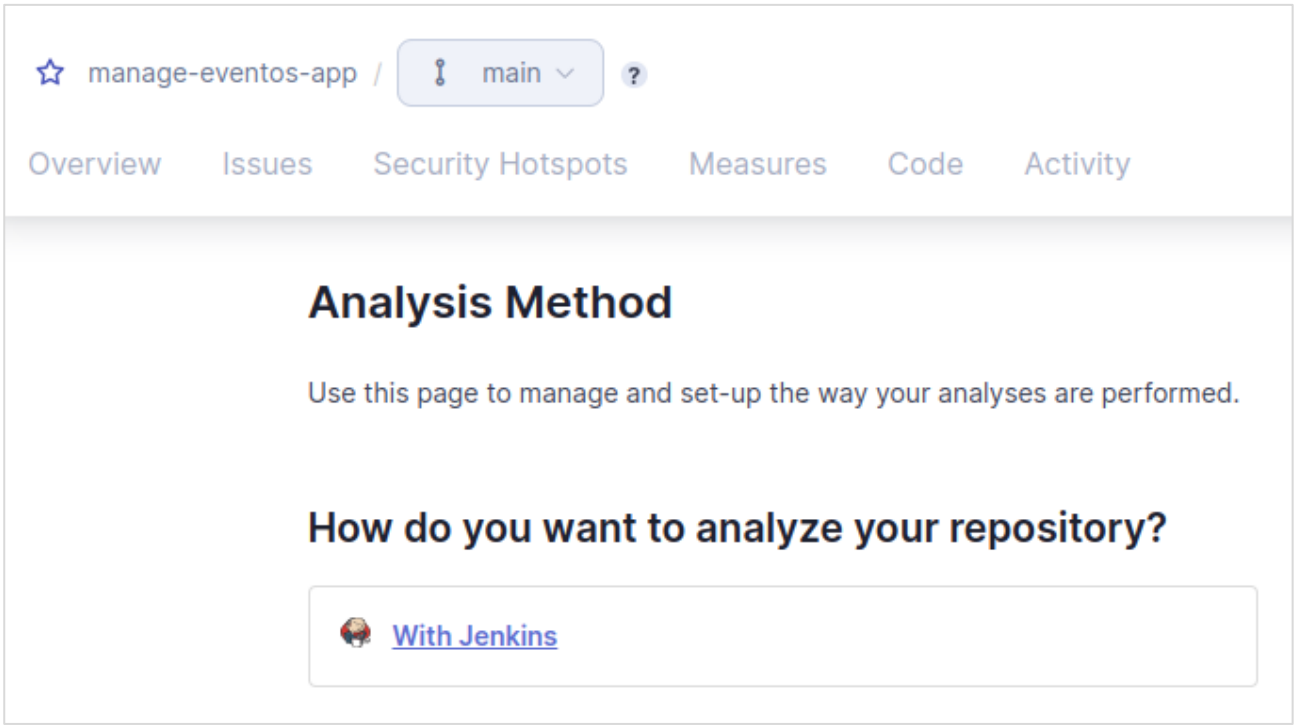
En el pom.xml de nuestro proyecto agregamos el plugin de SonarQube, para permitir el escáner.

```
73
74      <build>
75          <plugins>
76              <plugin>
77                  <groupId>org.springframework.boot</groupId>
78                  <artifactId>spring-boot-maven-plugin</artifactId>
79              </plugin>
80
81              <plugin>
82                  <groupId>org.sonarsource.scanner.maven</groupId>
83                  <artifactId>sonar-maven-plugin</artifactId>
84                  <version>3.9.1.2184</version>
85              </plugin>
86          </pl
87      </build>
88  </project>
89
```

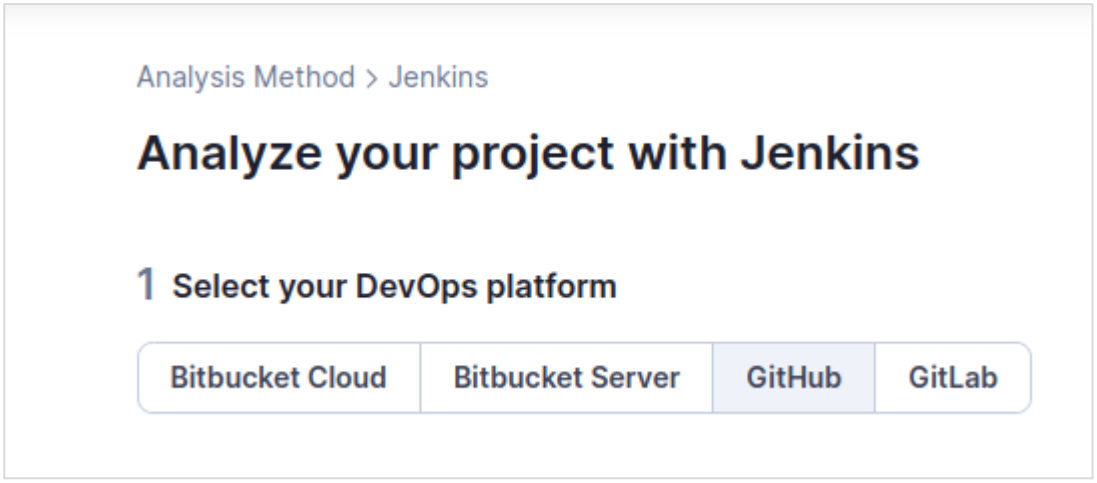
Complex type: **Plugin**
Description : The `<plugin>` element contains informations required for a plugin.
Version : 4.0.0+



Una vez creado el proyecto, seleccionamos como método de análisis a Jenkins



Luego seleccionamos la opción de GitHub.



Recibiremos una serie de instrucciones, pero lo que nos interesa es el punto donde se encuentra la opción de Maven

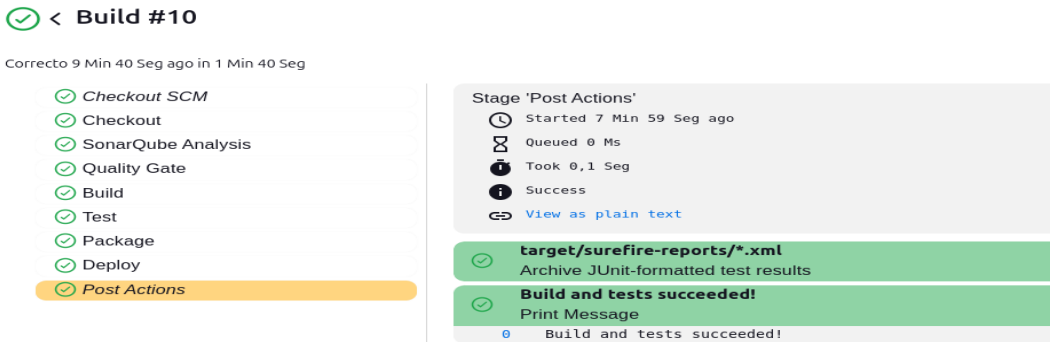
Esto es solo una guia, en el Jenkinsfile agregaremos unos elementos extras.

Estos son los stages nuevos en el JenkinsFile

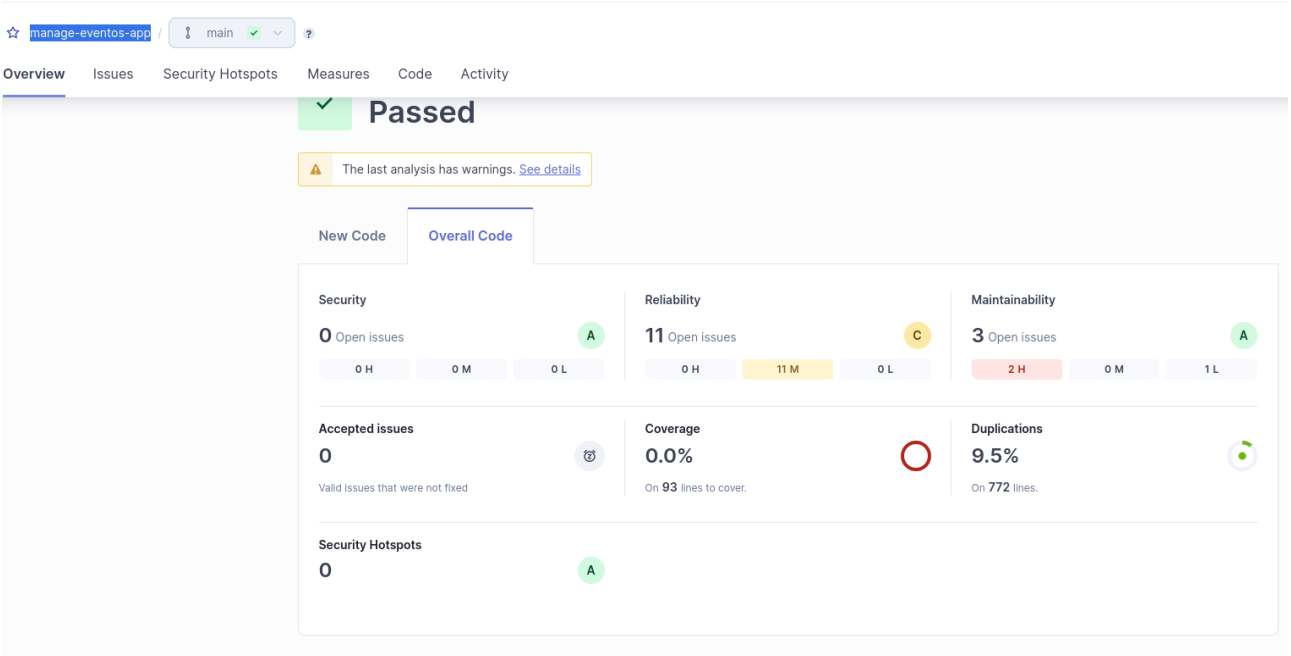
```
stage('SonarQube Analysis') {
  steps {
    script {
      def sonarRunner = tool name: 'SonarQubeScanner', type: 'hudson.plugins.sonar.SonarRunnerInstallation'
      withSonarQubeEnv('SonarQubeServer') {
        sh """
          ${sonarRunner}/bin/sonar-scanner \
          -Dsonar.projectKey=manage-eventos-app \
          -Dsonar.java.binaries=target/classes
        """
      }
    }
  }
}
```

```
stage('Quality Gate') {
  steps {
    timeout(time: 3, unit: 'MINUTES') {
      waitForQualityGate abortPipeline: true
    }
  }
}
```

Una vez agregados, enviamos un commit al repositorio de GitHub, para ejecutar el pipeline de Jenkins, donde veremos los dos nuevos stages de SonarQube.



En SonarQube, podemos ver el análisis de código efectuado.



5.13. Integrar notificaciones con Slack.

Slack es un programa de chat, utilizado por las empresas para comunicarse de forma interna y externa con sus colaboradores y socios. Su principal objetivo y función es, simplificar los flujos de comunicación.

Ventajas de Integrar Notificaciones con Slack en CI/CD

a. Visibilidad en Tiempo Real

- Cada vez que ocurre un evento importante (como una **construcción fallida** o un **despliegue exitoso**), los miembros del equipo reciben una **notificación instantánea**.
- Esto permite **reaccionar rápidamente** ante problemas críticos, mejorando la eficiencia y reduciendo el tiempo de inactividad.

b. Colaboración Mejorada

- Al notificar automáticamente en Slack, todos los miembros del equipo (desarrolladores, QA, DevOps) están al tanto del **estado del pipeline**.
- Las notificaciones fomentan la **colaboración inmediata** y la resolución conjunta de problemas, permitiendo que los equipos trabajen sincronizados.

c. Reducción de Tiempo en la Identificación de Problemas

- En caso de **errores en compilación o pruebas**, Slack puede notificar el fallo con **detalles específicos** (como los logs o cambios involucrados).
- Esto permite que el equipo actúe rápidamente y solucione problemas antes de que afecten el **proceso de despliegue** o el entorno productivo.

d. Mejora del Flujo de Trabajo y Automatización

- Puedes configurar Slack para que notifique sobre eventos clave como:
    - **Inicio y finalización de pipelines**.
    - **Errores en pruebas o compilación**.
    - **Despliegues exitosos en entornos** (staging, producción).
- Esto elimina la necesidad de consultar manualmente la interfaz de Jenkins o GitLab, haciendo el flujo más eficiente.

e. Alertas Proactivas y Seguimiento Continuo

- Las alertas en Slack permiten **anticipar problemas futuros**. Por ejemplo, si hay una caída continua en la calidad del código, se pueden enviar alertas automáticas para que los equipos actúen proactivamente.
- El equipo de DevOps puede seguir el progreso de los despliegues y **asegurar que no se pasen por alto errores importantes.**

f. Notificaciones Personalizadas

- Puedes configurar notificaciones **personalizadas por canal** en Slack:
  - **Un canal para fallos críticos** donde solo se reportan errores importantes.
  - **Un canal para despliegues** donde se notifican lanzamientos exitosos.
- Esto permite filtrar las notificaciones y evitar **sobrecarga de mensajes**, manteniendo el foco del equipo.

g. Historial de Eventos en Slack

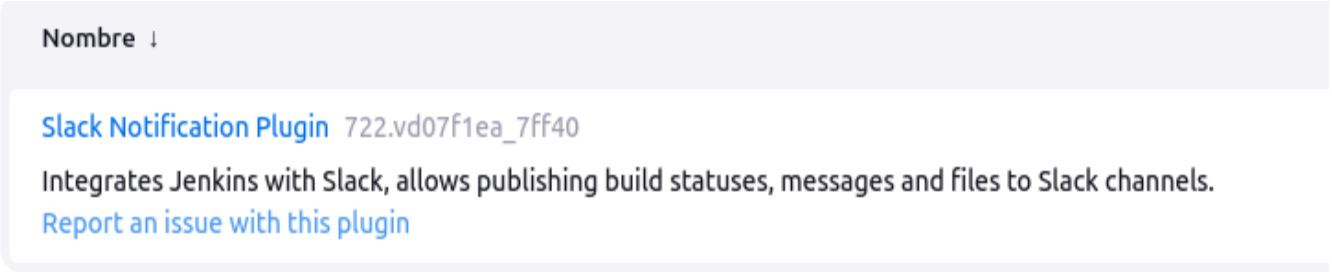
- Slack actúa como un **registro centralizado** donde los equipos pueden consultar el **historial de notificaciones** y ver qué ocurrió en el pipeline en momentos clave.
- Esto facilita la **trazabilidad** y ayuda a revisar incidentes pasados para optimizar el flujo de trabajo.

h. Integración Sencilla con Herramientas DevOps

- **Slack se integra fácilmente** con herramientas como **Jenkins, GitHub, GitLab, Azure DevOps** y otros servicios.
- Puedes configurar pipelines para enviar **notificaciones automáticas** usando **webhooks** o plugins específicos de Slack.

Para usar slack en nuestro ciclo de Devops debemos ir a Jenkins y luego:

Administrar Jenkins → Plugins →Instalar el Plugin de Slack



Luego crearemos una cuenta en Slack, en donde llegarán los mensajes que serán arrojados por nuestro pipeline de Jenkins más adelante.

slack

¡Te damos la bienvenida a tu nuevo espacio de trabajo!

Estos son los detalles de tu nuevo espacio de trabajo de Slack: **DevOps-Jenkins-MascaChicles**.

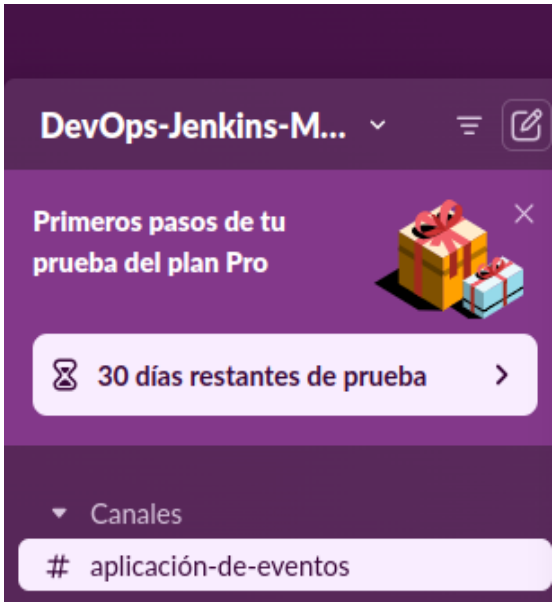
D

DevOps-Jenkins-MascaChicles

devopsjenkins-rqj7656.slack.com

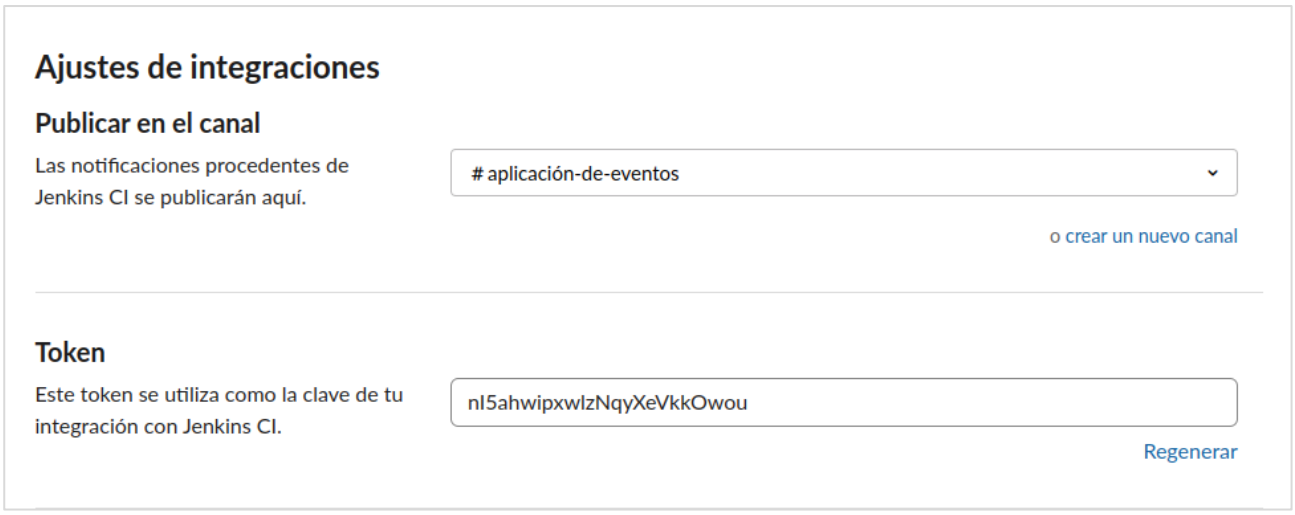
ABRIR SLACK

Generamos un canal para dejar las notificaciones generadas por Jenkins.

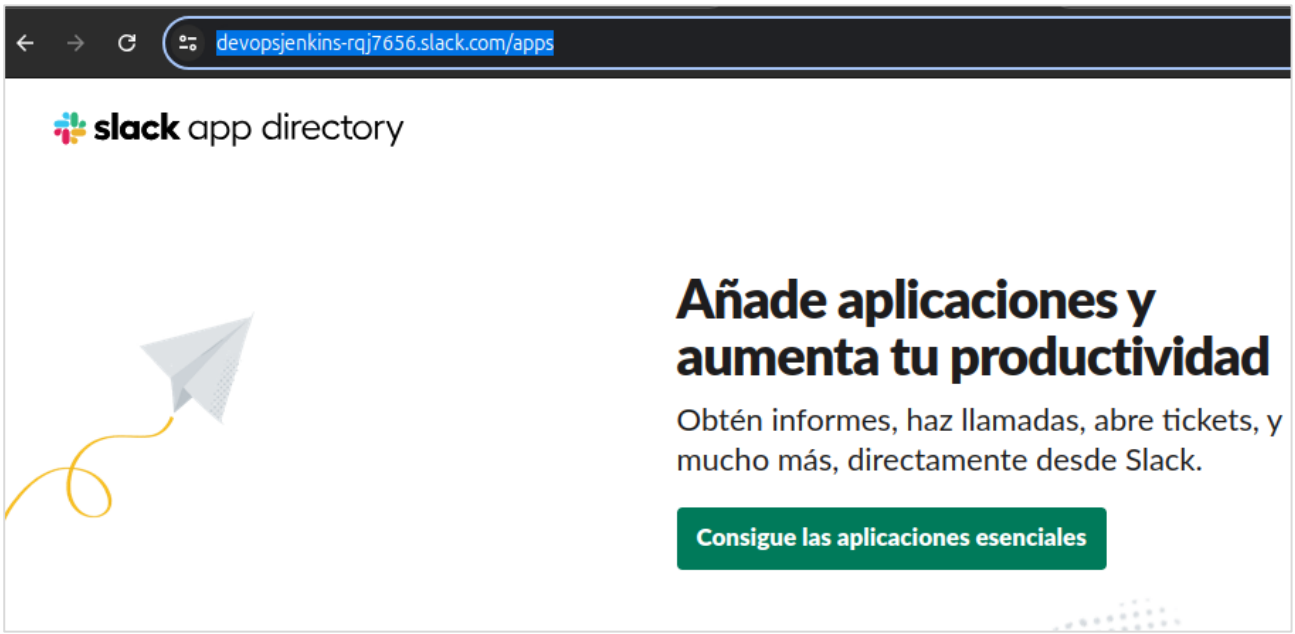


Más abajo en la misma página podremos ver el Token de acceso a Slack para poder integrarlo con otras aplicaciones.

Token : nI5ahwipxwlzNqyXeVkkOwou



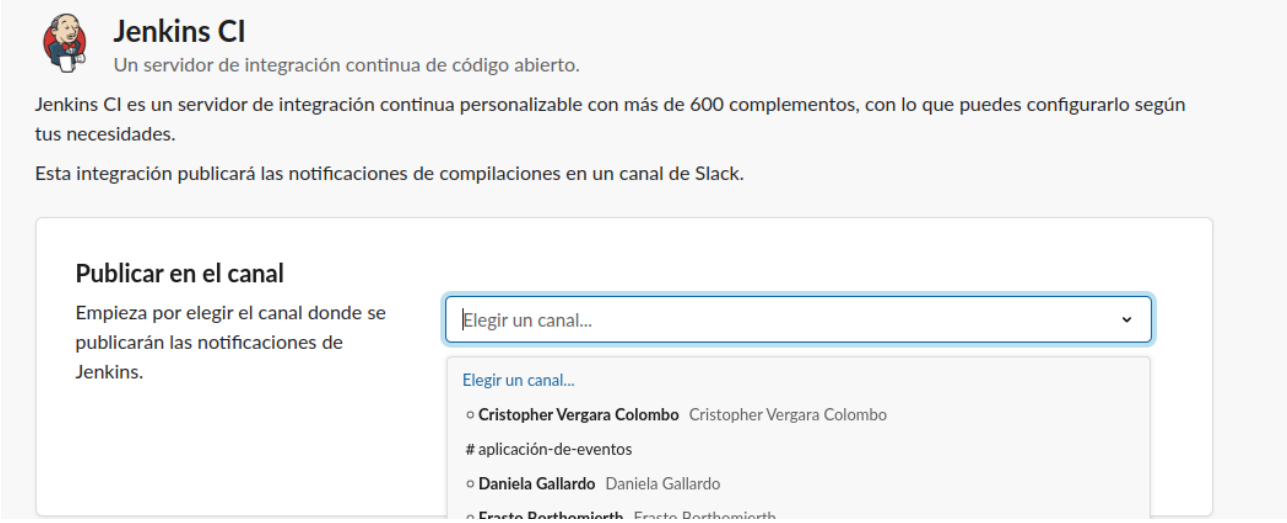
Ingresamos a la url, para descargar que nos provee slack para poder integrar aplicaciones externas y buscamos Jenkins.



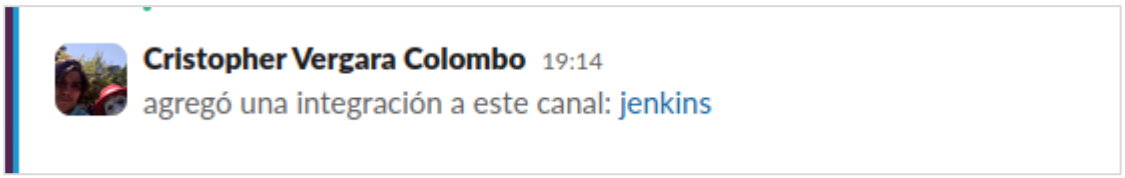
Buscamos Jenkins



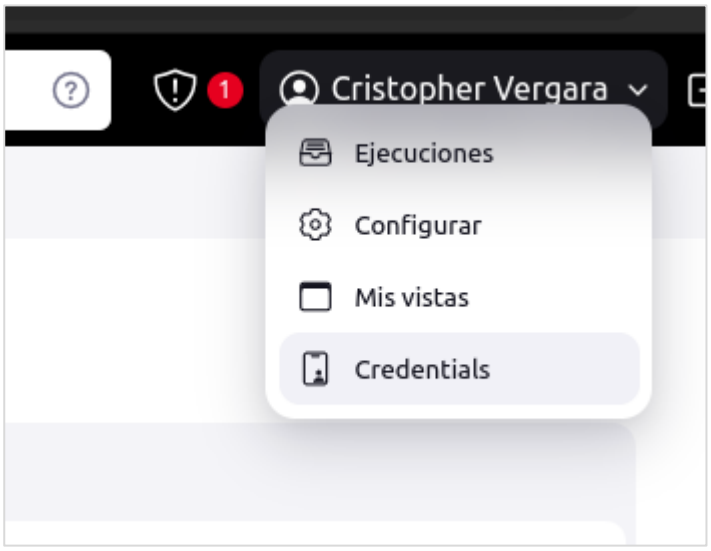
Elegimos el canal, donde aparecerá nuestra notificación



En slack recibiremos una notificación para confirmar la integración correcta del plugin de Jenkins



Ahora nos vamos a Jenkins y en panel de Control agregamos el Token generado por Jenkins en el apartado de Credentials

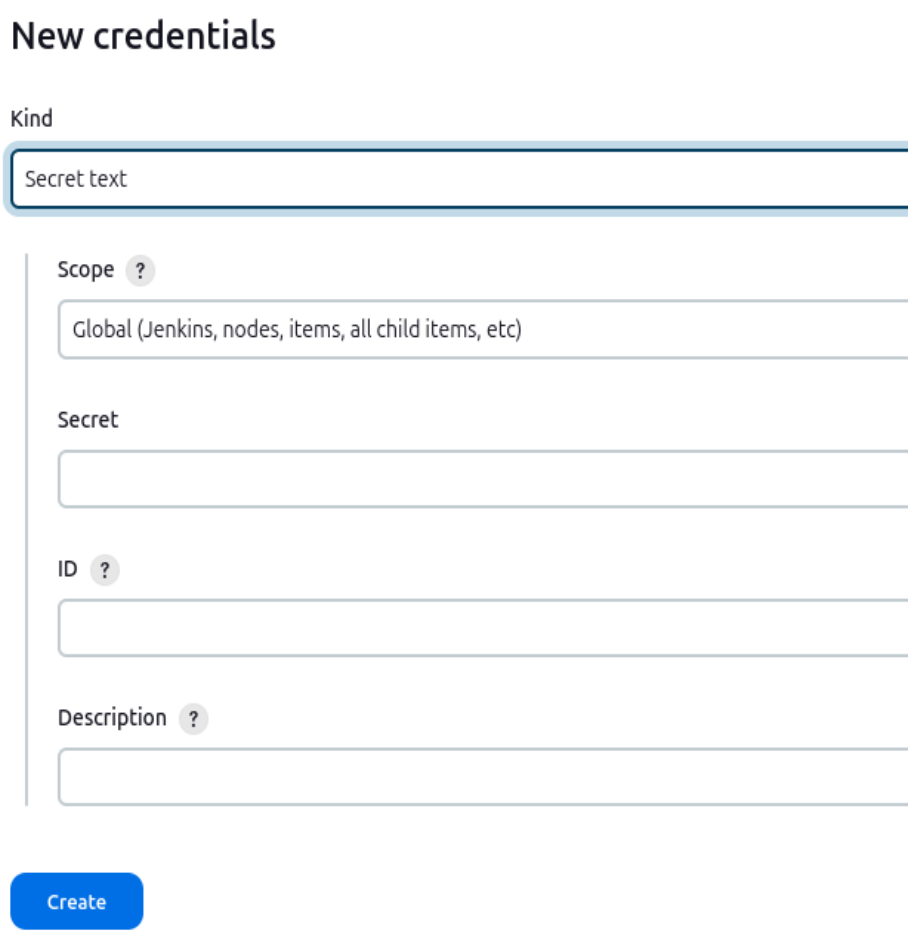


Creamos una nueva credencial con el Token que nos proporcionó slack previamente

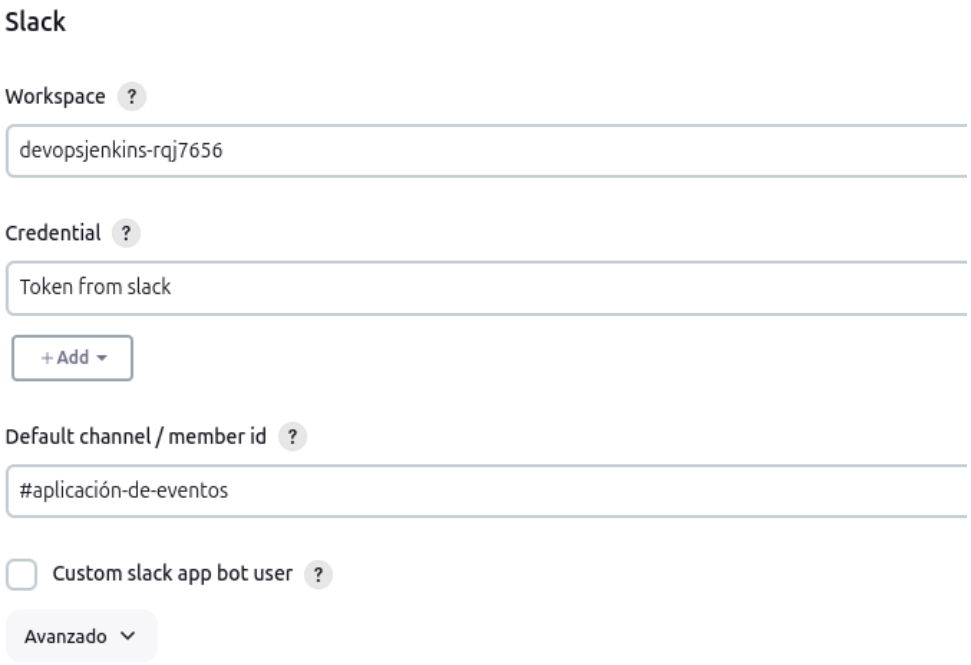
Kind: Secret text

Secret: Token de Jenkins

Id: Nombre descriptivo



Ahora nos vamos al Panel de Control→Administrar Jenkins y configuramos nuestro plugin de slack, con las credenciales que ingresamos en el paso previo.



Finalmente, en nuestro Jenkinsfile, agregamos como variables de ambiente las configuraciones propias para notificar a Slack.

```
pipeline {
    agent any

    environment {
        SLACK_CHANNEL = '#aplicacion-de-eventos'
        JOB_NAME = "${env.JOB_NAME}"
        BUILD_NUMBER = "${env.BUILD_NUMBER}"
        BUILD_URL = "${env.BUILD_URL}"
    }
    stages {
```

En el apartado de post de nuestro Jenkinsfile, agregamos los mensajes que se van a enviar una vez sea completado el pipeline de Jenkins.

```
post {
        success {
            slackSend(channel: env.SLACK_CHANNEL,
                      color: 'good',
                      message: "Job '${env.JOB_NAME} [${env.BUILD_NUMBER}]' succeeded. Check it out: ${env.BUILD_URL}",
                      tokenCredentialId: env.SLACK_CREDENTIAL_ID)
        }
        failure {
            slackSend(channel: env.SLACK_CHANNEL,
                      color: 'danger',
                      message: "Job '${env.JOB_NAME} [${env.BUILD_NUMBER}]' failed. Check it out: ${env.BUILD_URL}",
                      tokenCredentialId: env.SLACK_CREDENTIAL_ID)
        }
        unstable {
            slackSend(channel: env.SLACK_CHANNEL,
                      color: 'warning',
                      message: "Job '${env.JOB_NAME} [${env.BUILD_NUMBER}]' is unstable. Check it out: ${env.BUILD_URL}",
                      tokenCredentialId: env.SLACK_CREDENTIAL_ID)
        }
        always {
            slackSend(channel: env.SLACK_CHANNEL,
                      color: '#FFFF00',
                      message: "Job '${env.JOB_NAME} [${env.BUILD_NUMBER}]' completed. Check it out: ${env.BUILD_URL}",
                      tokenCredentialId: env.SLACK_CREDENTIAL_ID)
        }
```

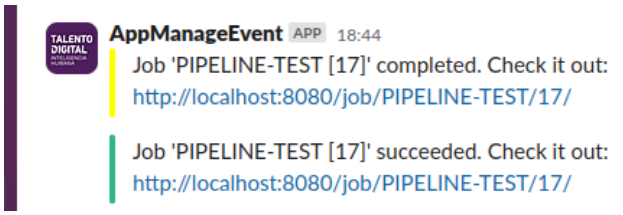
Enviamos un push a nuestro Github, para verificar que todo salga bien y verificamos Jenkins.

Nombre del Proyecto: PIPELINE-TEST
Construccion de proyecto con Webhook

Stage View

	Declarative: Checkout SCM	Checkout	SonarQube Analysis	Build	Test	Package	Deploy	Declarative: Post Actions
Average stage times: (Average full run time: ~1min 26s)	1s	1s	23s	9s	24s	22s	336ms	1s
#17 jul 31 18:42 1 commit	1s	1s	23s	9s	24s	22s	336ms	1s

En slack, recibiremos el mensaje confirmando que nuestro Pipeline terminó con éxito la tarea.



5.14. Integrar Nexus Sonatype

¿Qué es Nexus Sonatype?

Nexus Sonatype, es una herramienta de gestión de repositorios que permite almacenar, administrar y compartir artefactos generados durante el desarrollo del software. Los artefactos pueden ser librerías, dependencias o archivos compilados (como .jar, .war o .npm), que son necesarios para construir y desplegar aplicaciones.

Nexus ofrece soporte para múltiples tipos de repositorios, como:

- Maven (Java)
- npm (JavaScript/Node.js)
- Docker (imágenes de contenedores)
- PyPI (Python)
- NuGet (C#/.NET)

Se puede integrar en pipelines CI/CD, para asegurar un flujo seguro, eficiente y controlado del ciclo de vida del software.

¿Por qué es una buena práctica integrar Nexus Sonatype en un ciclo CI/CD?

a. Almacenamiento Centralizado de Artefactos

- Nexus actúa como un **repositorio central**, donde se almacenan los artefactos generados (por ejemplo, archivos .jar, .war, imágenes Docker), para diferentes entornos de desarrollo, pruebas y producción.
- Esto **evita la duplicación de artefactos** y facilita la gestión de versiones.

b. Optimización del Proceso de CI/CD

- Durante el pipeline CI/CD, Nexus almacena los artefactos generados por herramientas como **Maven o Gradle**. En lugar de reconstruir los artefactos en cada etapa, se **reutilizan desde el repositorio**, mejorando la eficiencia.

- Esto permite construir una vez y reutilizar el mismo artefacto en diferentes entornos (por ejemplo, dev, staging, producción).

c. Control de Versiones y Artefactos Históricos

- Nexus permite **versionar artefactos**, garantizando que cada versión del software esté disponible y se pueda recuperar en cualquier momento.
- Esto es útil para realizar **rollbacks** en caso de que una nueva versión presente problemas, asegurando **despliegues más seguros**.

d. Integración con Herramientas CI/CD

- Nexus Sonatype se integra fácilmente con **Jenkins, GitLab CI, GitHub Actions, Azure DevOps**, y otras plataformas CI/CD.
- Los artefactos se publican automáticamente en Nexus durante el pipeline, eliminando la necesidad de configuraciones manuales.

e. Seguridad y Escaneo de Dependencias

- Nexus ofrece **Nexus IQ**, que analiza las dependencias del proyecto en busca de **vulnerabilidades** conocidas.
- Esto garantiza que el software no dependa de librerías inseguras, permitiendo un **control proactivo de seguridad.**

f. Mejora en la Colaboración del Equipo

- Al contar con un **repositorio compartido**, los equipos de desarrollo y DevOps pueden acceder fácilmente a las mismas versiones de las dependencias y artefactos, evitando conflictos de versiones.
- Esto **facilita la colaboración** y asegura que todos trabajen con los mismos componentes.

g. Compatibilidad con Múltiples Tecnologías

- Nexus Sonatype soporta una gran variedad de **repositorios** y tipos de artefactos (Maven, npm, Docker, etc.), lo que lo convierte en una herramienta versátil para proyectos que involucran múltiples tecnologías.

h. Control de Acceso y Gobernanza

- Nexus permite definir **permisos y políticas de acceso** para que solo usuarios autorizados puedan acceder, publicar o descargar artefactos.
- Esto garantiza un control adecuado sobre **qué se despliega** y quién tiene acceso a los artefactos, mejorando la **gobernanza del software**.

i. Mejora del Rendimiento del Pipeline CI/CD

- Nexus actúa como un **cache local** de dependencias externas, como las obtenidas de **Maven Central** o **Docker Hub**. Esto acelera el pipeline porque las dependencias se descargan desde el propio Nexus en lugar de un servidor remoto.

j. Soporte para Rollbacks y Despliegues Seguros

- Al utilizar artefactos versionados y almacenados en Nexus, se pueden realizar **despliegues más seguros** y volver fácilmente a versiones anteriores si es necesario.
- Esto reduce los riesgos durante los despliegues y mejora la **estabilidad en producción**.

Para integrarlo como servidor de repositorios para nuestro código, lo agregamos en nuestro docker-compose

```
nexus:

  image: sonatype/nexus3:latest

  container_name: nexus

  ports:

    - "8081:8081"  # Puerto para acceder a Nexus

  volumes:

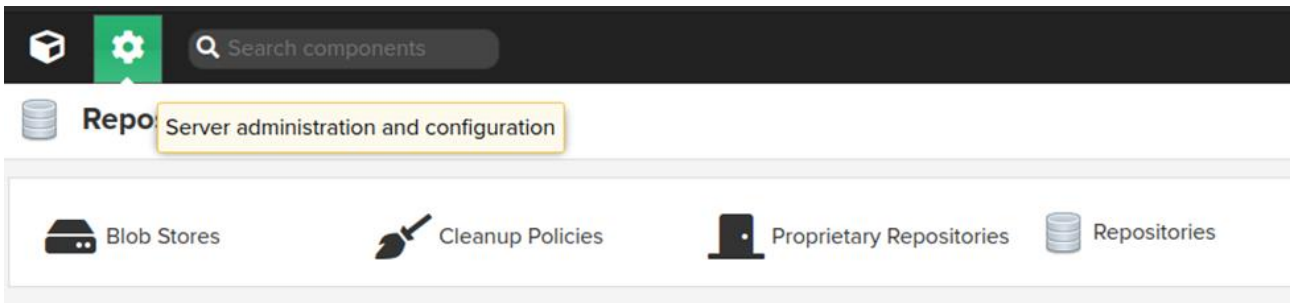
    - nexus-data:/nexus-data  # Persistir datos de Nexus

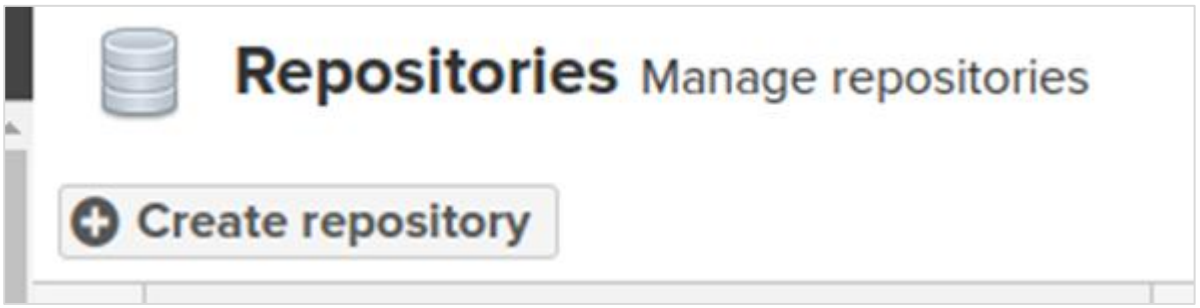
  environment:

    - NEXUS_CONTEXT=nexus
```

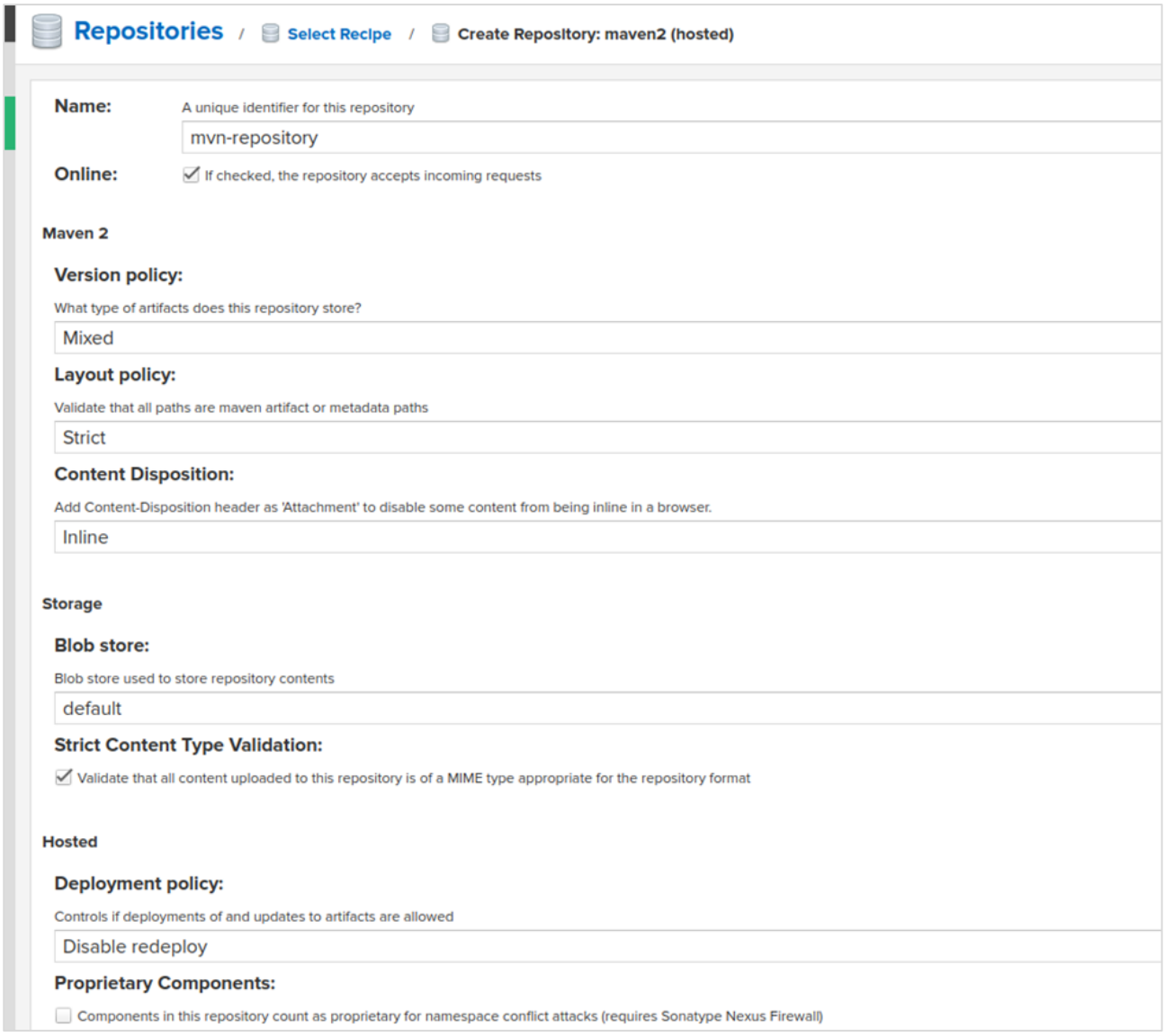
Crear un repositorio, para almacenar artefactos de software en Nexus OSS

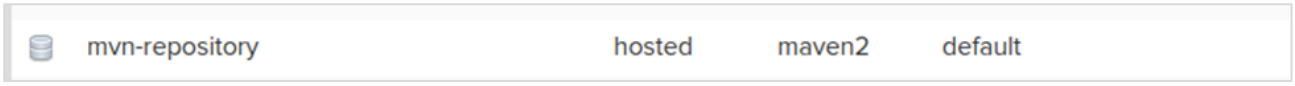
Inicio—>Server Administration and Configuration→Repositories→Create Repository



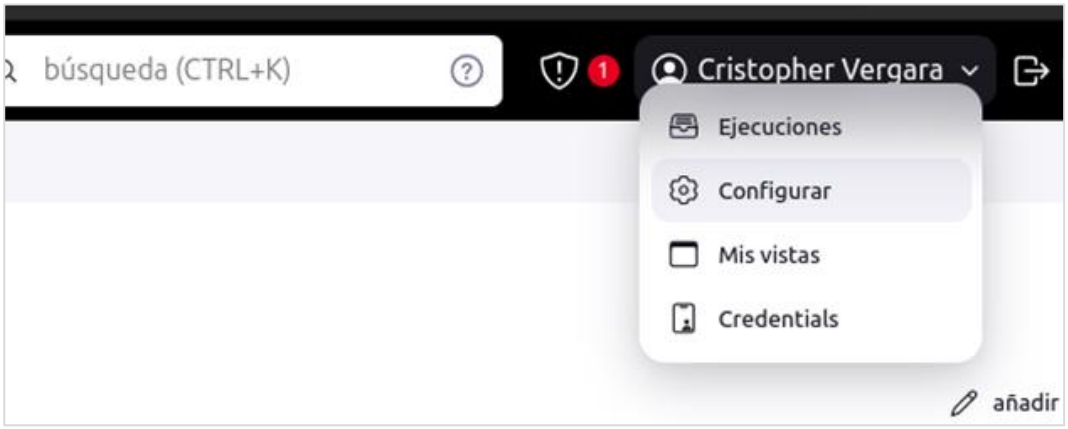


Creamos un repositorio, escogiendo la opción maven hosted y en version policy seleccionamos mixed

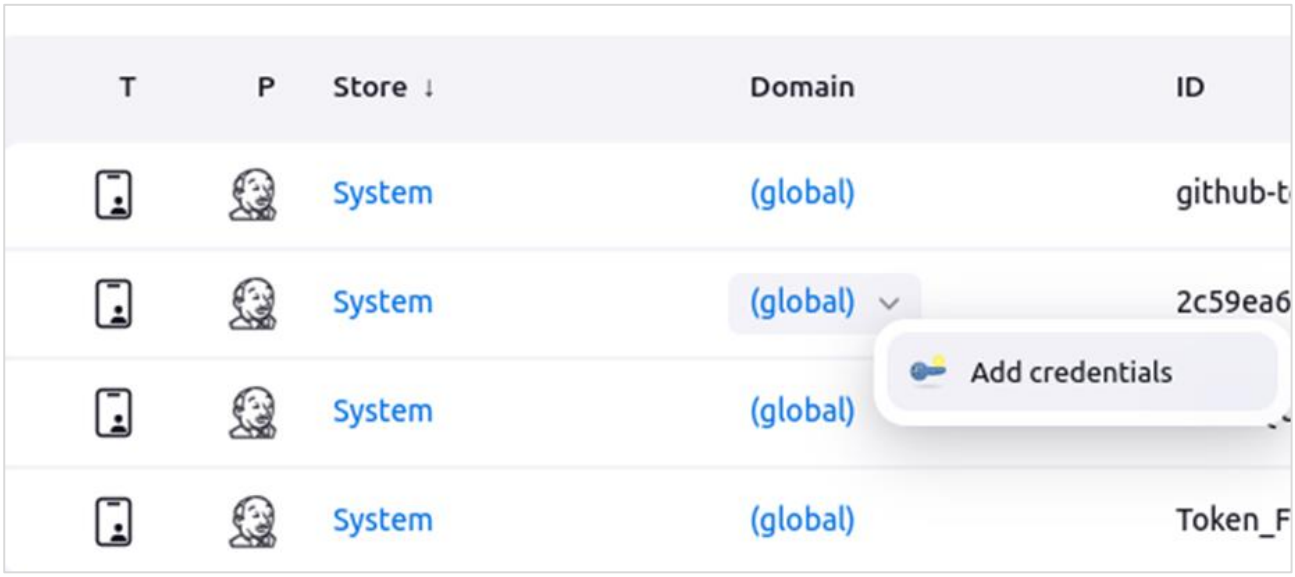




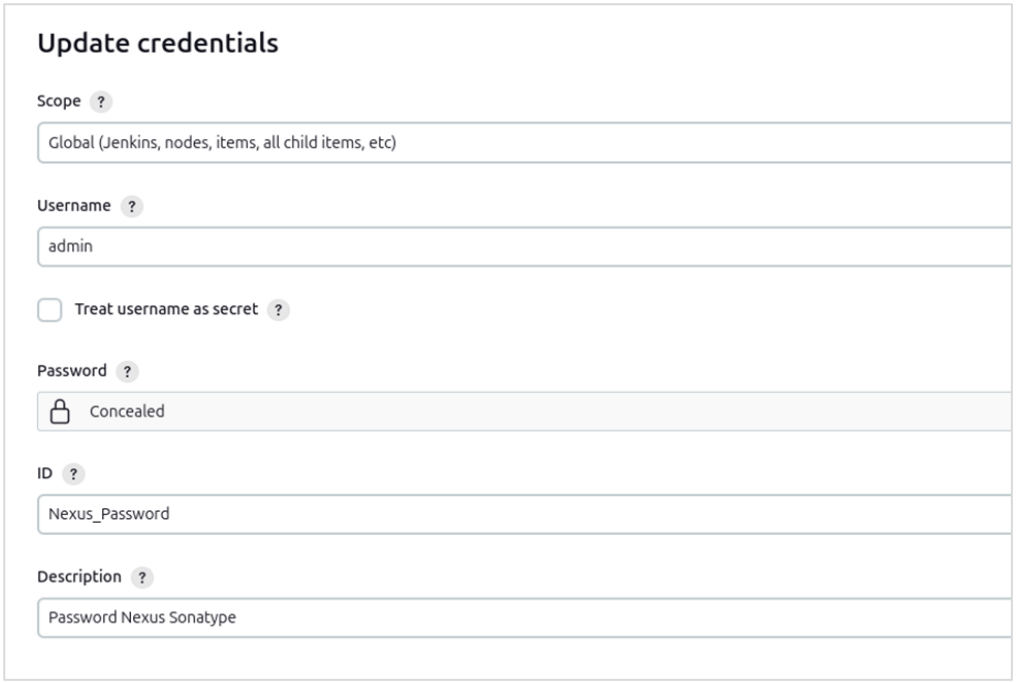
Luego nos vamos a Jenkins, para configurar las credenciales de Nexus Sonatype OSS

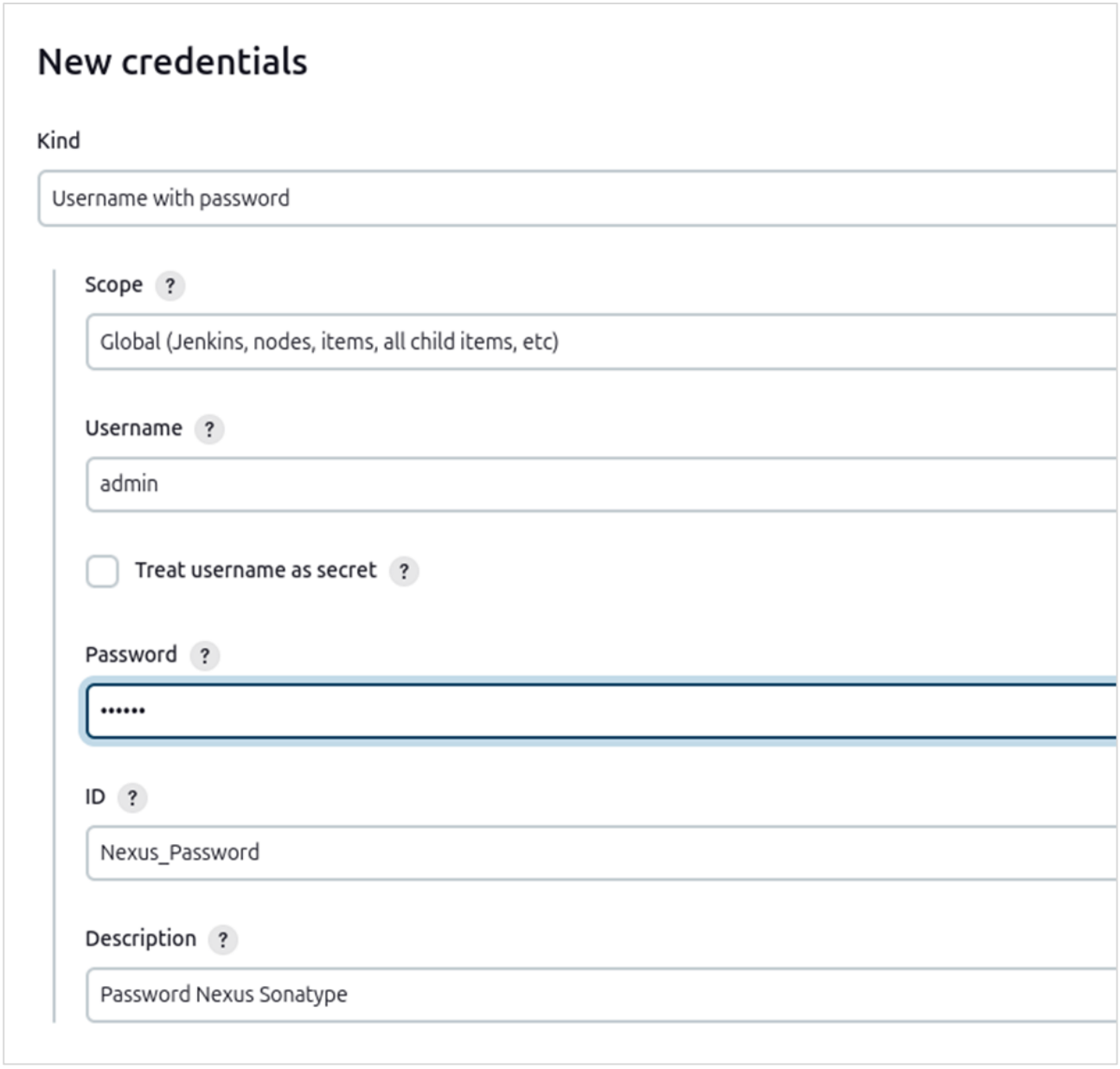


Nos vamos a Panel de Control—>Usuario—>Credenciales—>Add credentials



Seleccionamos de tipo Username with password y agregamos nuestras credenciales de Nexus


Update credentials
Scope
Global (Jenkins, nodes, items, all child items, etc)
Username
admin
Treat username as secret
Password
Concealed
ID
Nexus_Password
Description
Password Nexus Sonatype


New credentials
Kind
Username with password
Scope
Global (Jenkins, nodes, items, all child items, etc)
Username
admin
Treat username as secret
Password
ID
Nexus_Password
Description
Password Nexus Sonatype



En el pom.xml agregamos la url del repositorio de nexus, que en este caso está corriendo en ngrok:

```
<distributionManagement>
    <repository>
        <id>nexus</id>
        <url>https://3b33-190-5-37-40.ngrok-free.app/repository/events-mvn-app/</url>
    </repository>
</distributionManagement>
<dependencies>
```

En nuestro Jenkinsfile agregamos un nuevo stage para subir el archivo a Nexus

```
stage('Upload to Nexus') {
steps {
  script {

  pom = readMavenPom file: "pom.xml";
    // Find built artifact under target folder
  filesByGlob = findFiles(glob: "target/*.${pom.packaging}");
    // Print some info from the artifact found
  echo "${filesByGlob[0].name}${filesByGlob[0].path} ${filesByGlob[0].directory}" +
"${filesByGlob[0].length} ${filesByGlob[0].lastModified}"
 // Extract the path from the File found
  artifactPath = filesByGlob[0].path;
    // Assign to a boolean response verifying If the artifact name exists
  artifactExists = fileExists artifactPath;

  if(artifactExists) {
      echo "*** File: ${artifactPath}, group: ${pom.groupId}," +
```

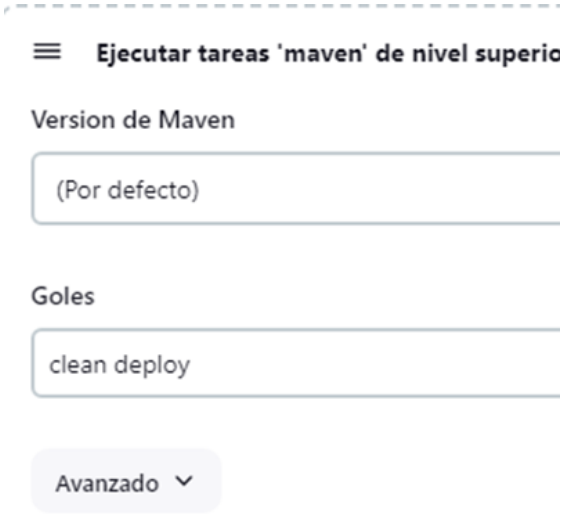
```
        " packaging: ${pom.packaging}, version ${pom.version}";
        nexusArtifactUploader(
            nexusVersion: NEXUS_VERSION,
            protocol: NEXUS_PROTOCOL,
            nexusUrl: NEXUS_URL,
            groupId: pom.groupId,
            version: ARTIFACT_VERSION,
            repository: NEXUS_REPOSITORY,
            credentialsId: NEXUS_CREDENTIAL_ID,



        artifacts: [
       // Artifact generated such as .jar, .ear and .war files.
            [artifactId: pom.artifactId,
            classifier: '',
            file: artifactPath,
            type: pom.packaging]
          ]
        );

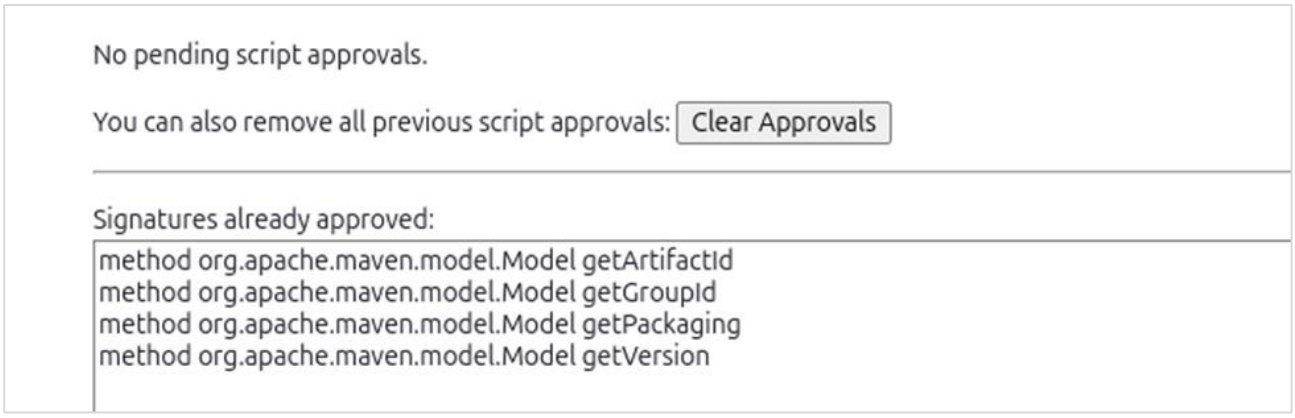
    } else {
        error "*** File: ${artifactPath}, could not be found";
    }
  }
}
}
```

Modificamos la goal de maven en el job de jenkins a “clean deploy”.



Debido a que ahora hay un script en nuestro Jenkinsfile, para verificar que el archivo se crea en Nexus Repository, es posible que se nos muestre un nuevo menu en Jenkins para aprobar el script y poder iniciar el pipeline.





Iniciamos el pipeline en Jenkins.

En los logs de la ejecución podemos ver que se inicia correctamente:

```
23:32:03 [INFO]
23:32:03 [INFO] --- maven-deploy-plugin:3.1.2:deploy (default-deploy) @ AppManageEvents ---
23:32:03 Uploading to nexus: https://3b33-190-5-37-40.ngrok-free.app/repository/events-mvn-app/cl/talentodigital/AppManageEvents/0.0.1-RELEASE/AppManageEvents-0.0.1-RELEASE.pom
23:32:04 Progress (1): 3.3 kB

Uploaded to nexus: https://3b33-190-5-37-40.ngrok-free.app/repository/events-mvn-app/cl/talentodigital/AppManageEvents/0.0.1-RELEASE/AppManageEvents-0.0.1-RELEASE.pom (3.3 kB at 2.9 kB/s)
23:32:04 Uploading to nexus: https://3b33-190-5-37-40.ngrok-free.app/repository/events-mvn-app/cl/talentodigital/AppManageEvents/0.0.1-RELEASE/AppManageEvents-0.0.1-RELEASE.jar
Progress (1): 1.0/62 MB
Progress (1): 1.6/62 MB
```

Termina el pipeline con success:

```
Uploaded to nexus: https://3b33-190-5-37-40.ngrok-free.app/repository/events-mvn-app/cl/talentodigital/AppManageEvents/0.0.1-RELEASE/AppManageEvents-0.0.1-RELEASE.jar (62 MB at 3.5 MB/s)
23:32:22 Downloading from nexus: https://3b33-190-5-37-40.ngrok-free.app/repository/events-mvn-app/cl/talentodigital/AppManageEvents/maven-metadata.xml
23:32:22 Progress (1): 328 B

Downloaded from nexus: https://3b33-190-5-37-40.ngrok-free.app/repository/events-mvn-app/cl/talentodigital/AppManageEvents/maven-metadata.xml (328 B at 1.1 kB/s)
23:32:22 Uploading to nexus: https://3b33-190-5-37-40.ngrok-free.app/repository/events-mvn-app/cl/talentodigital/AppManageEvents/maven-metadata.xml
23:32:23 Progress (1): 328 B

Uploaded to nexus: https://3b33-190-5-37-40.ngrok-free.app/repository/events-mvn-app/cl/talentodigital/AppManageEvents/maven-metadata.xml (328 B at 392 B/s)
23:32:23 [INFO] ------------------------------------------------------------------------
23:32:23 [INFO] BUILD SUCCESS
23:32:23 [INFO] ------------------------------------------------------------------------
23:32:23 [INFO] Total time:  31.114 s
23:32:23 [INFO] Finished at: 2024-07-25T23:32:23-04:00
23:32:23 [INFO] ------------------------------------------------------------------------
```

Artefacto en repositorio:



6. Monitoreo

6.1 Elasticsearch-Kibana-Logstash (ELK)

ELK es un popular stack de monitoreo compuesto por Elasticsearch, Logstash y Kibana. Elasticsearch, se encarga del almacenamiento y búsqueda de datos, Logstash gestiona la recolección y procesamiento de logs, y Kibana proporciona una interfaz gráfica para visualizar y analizar los datos en tiempo real. Es ampliamente utilizado para centralizar, monitorear y analizar grandes volúmenes de logs en aplicaciones distribuidas.

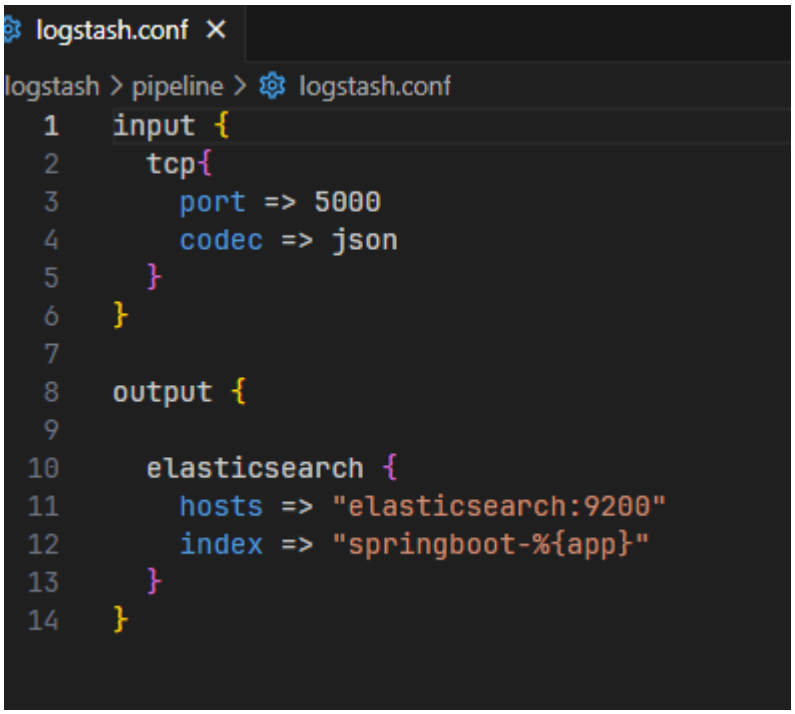
Para implementarlo en nuestro proyecto primero necesitamos definir algunos recursos y configuraciones que levantaremos en contenedores docker:

6.1.1 Definir contenedores en docker-compose.yml

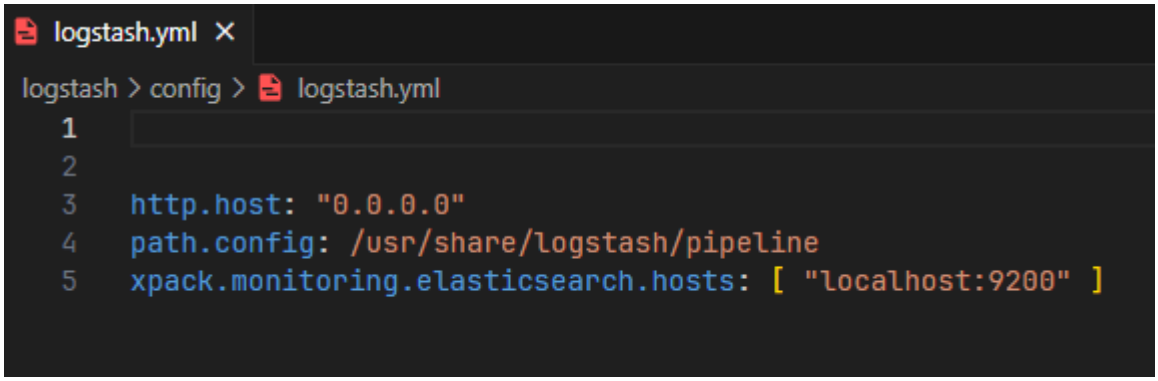


6.1.2 Definir configuraciones de logstash:

Logstash.conf: Archivo de configuración de Logstash. En input, se especifica que Logstash recibirá datos a través de TCP en el puerto 5000 y los procesará como formato **JSON**. En la sección output, los datos se envían a un clúster de Elasticsearch en el host elasticsearch:9200, almacenándolos en un índice con el prefijo **springboot**.



logstash.yml: Archivo que configura el host HTTP de Logstash como 0.0.0.0 para aceptar conexiones desde cualquier dirección IP. La ruta de configuración para los pipelines se establece en /usr/share/logstash/pipeline, y se especifica que Logstash monitorea un host de Elasticsearch ubicado en localhost:9200.



*Ambas ubicadas dentro de la carpeta ./logstash al mismo nivel que nuestro docker compose

6.1.3 Agregar dependencia de logstash en pom.xml

```xml
<dependency>
    <groupId>net.logstash.logback</groupId>
    <artifactId>logstash-logback-encoder</artifactId>
    <version>7.2</version>
</dependency>
```

6.1.4 Crear y configurar archivo logback-spring.xml en carpeta de resources.

Logback-spring.xml: Configura el logging de una aplicación Spring Boot utilizando Logback. Incluye la configuración base de Spring Boot y define un appender llamado logstash que envía los logs a localhost:5000 utilizando el formato JSON. Dentro del encoder se especifican varios proveedores de información para los logs, como el MDC, el nombre del logger, el nivel de log, el mensaje y más. Los logs a nivel info o superior se envían a este appender.

README.md | pom.xml (AppManageEvents) | logback-spring.xml | application.yml | AppManag

```xml
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
    <include resource="org/springframework/boot/logging/logback/base.xml"/>
    <appender name="logstash" class="net.logstash.logback.appender.LogstashTcpSocketAppender">
        <destination>localhost:5000</destination>
        <encoder class="net.logstash.logback.encoder.LoggingEventCompositeJsonEncoder">
            <providers>
                <mdc />
                <context />
                <logLevel />
                <loggerName />
                <pattern>
                    <pattern>
                        {
                        "app": "test-log"
                        }
                    </pattern>
                </pattern>
                <threadName />
                <message />
                <logstashMarkers />
                <stackTrace />
            </providers>
        </encoder>
    </appender>
    <root level="info">
        <appender-ref ref="logstash" />
    </root>
</configuration>
```

6.1.5 Verificar/Agregar logs en controladores REST.

Verificamos que nuestra app, tenga implementados algunos logs en sus controladores:

```java
public class EventoControlador {

    private static final Logger logger = LoggerFactory.getLogger(EventoControlador.class);  13 usages

    @Autowired  4 usages
    private EventoServicio eventoServicio;

    @Autowired  2 usages
    private SalonServicio salonServicio;

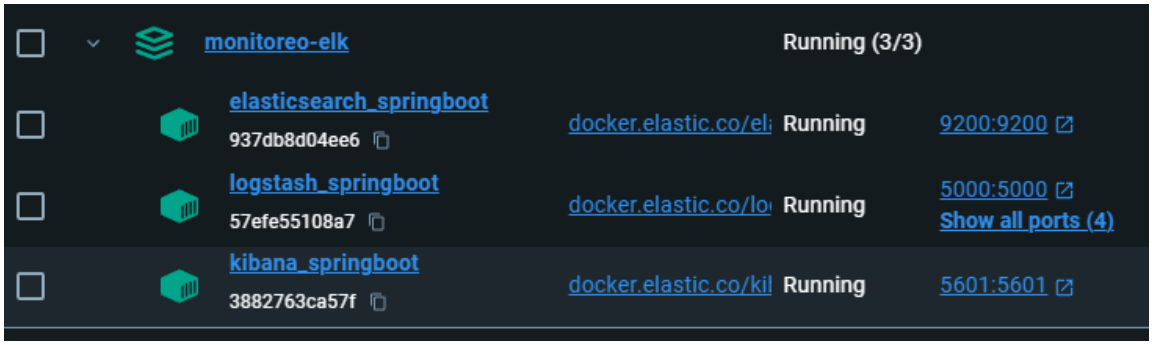
    @Operation(summary = "Obtiene la lista de eventos en la pagina de inicio y filtra por palabra clave")
    @ApiResponse(responseCode = "200", description = "HTTP STATUS 200")
    @RequestMapping(value ="/" ,method = RequestMethod.GET)
    public String verPaginaDeInicio(Model modelo, @Param("palabraClave") String palabraClave){

        List<Evento> listarEventos = eventoServicio.listAll(palabraClave);

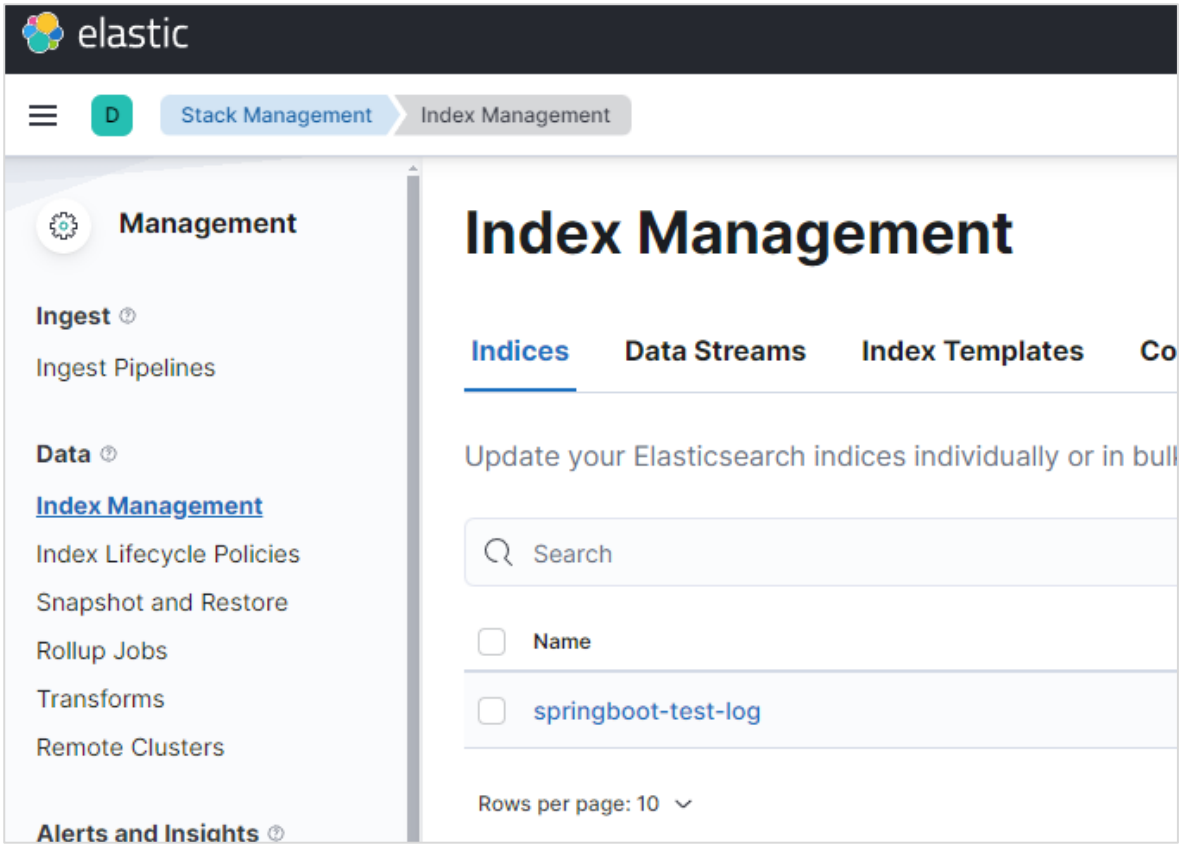
        modelo.addAttribute( attributeName: "palabraClave", palabraClave);
        modelo.addAttribute( attributeName: "listarEventos", listarEventos);
        logger.info("Main page login");

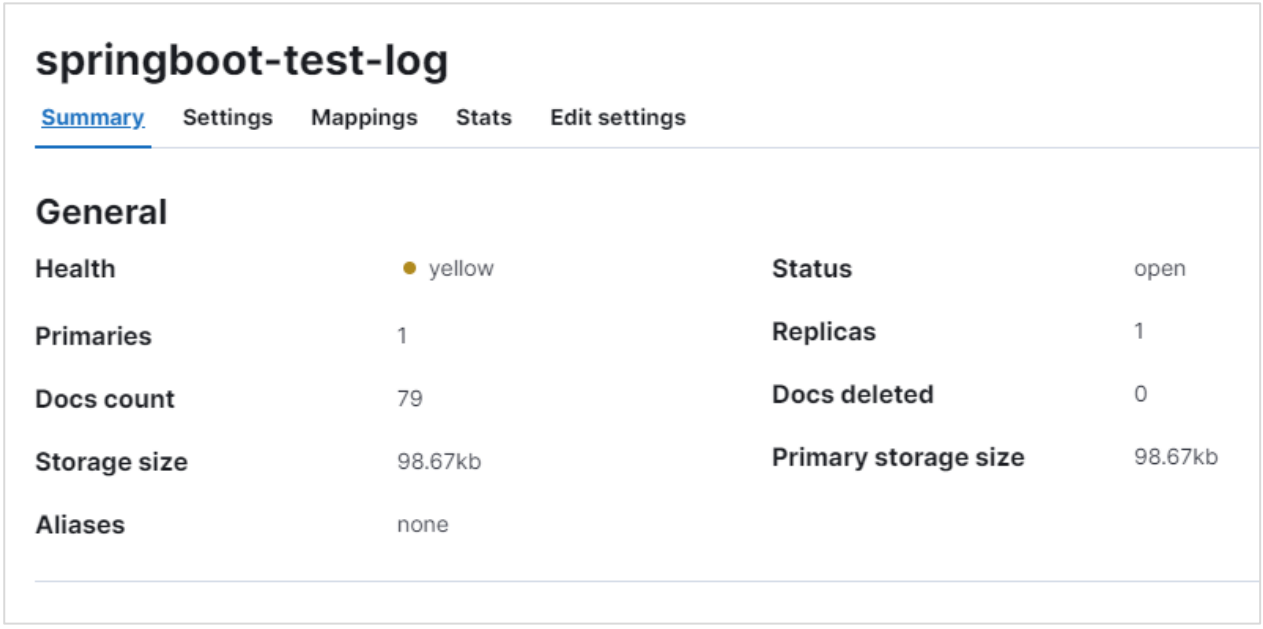
        return "index" ;
    }
```

6.1.6 Docker-compose up y verificar creacion del index.

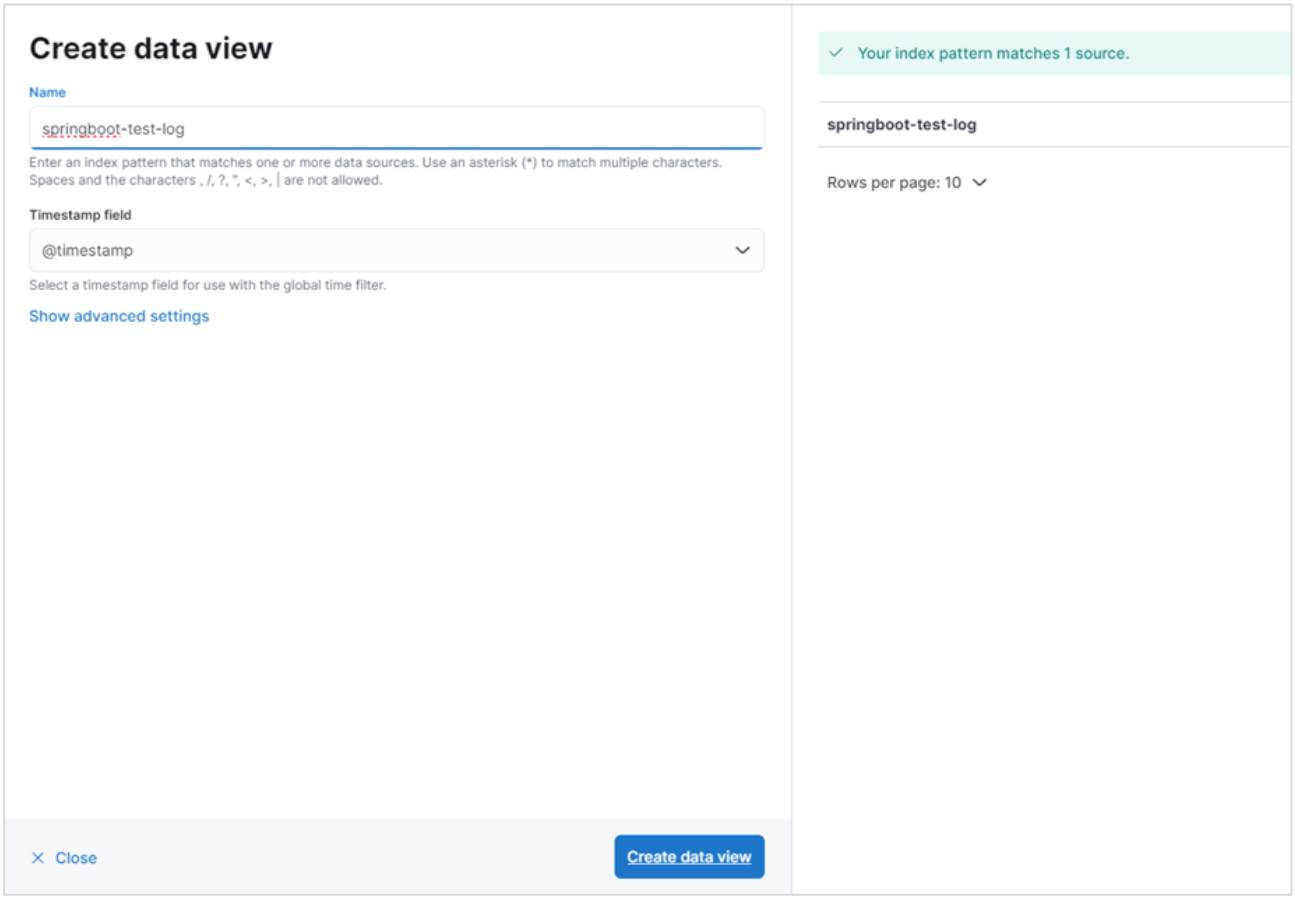


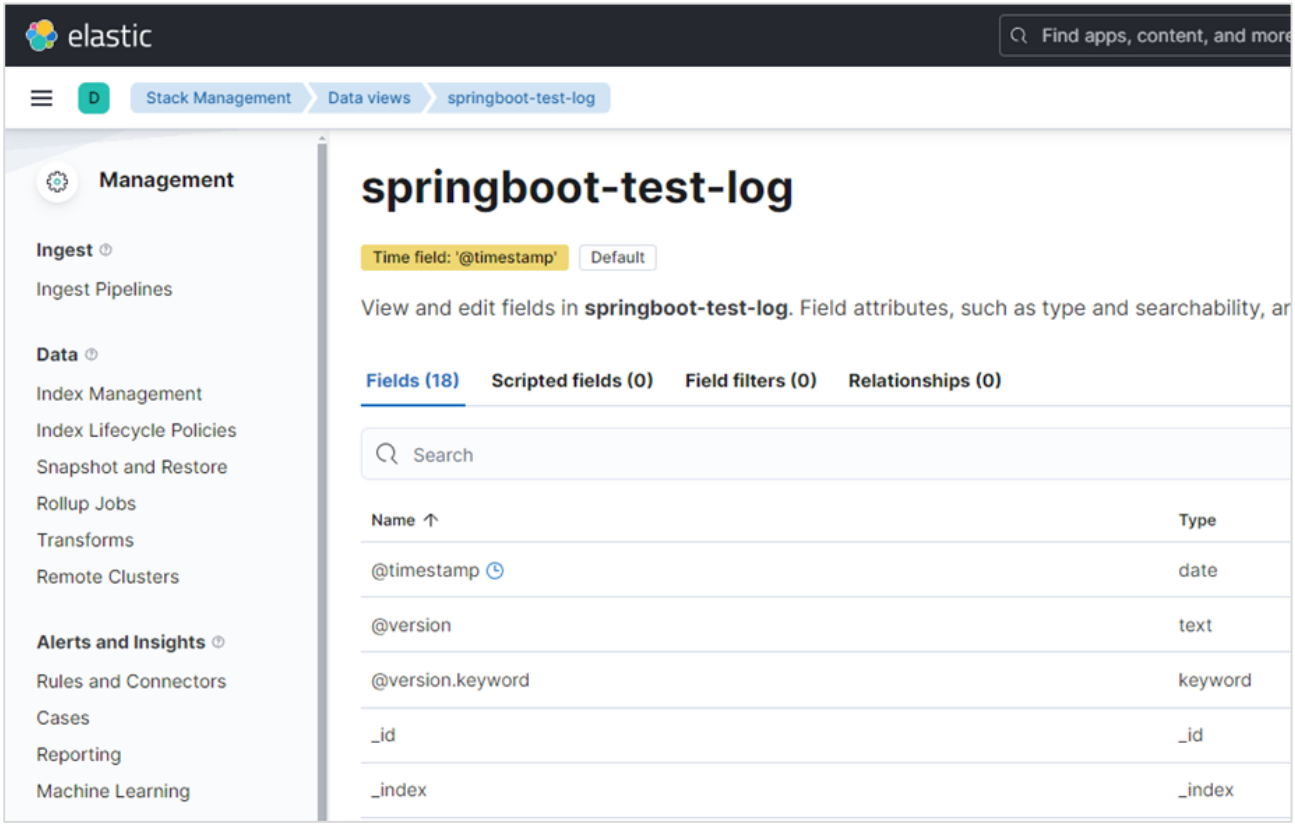
Levantamos la aplicación y generamos algunas peticiones de prueba. A continuación, en Kibana, accediendo a su puerto correspondiente, podemos encontrar el índice de Elasticsearch que definimos previamente en la sección de ***Index Management*** dentro de ***Stack Management***.


elastic
D
Stack Management
Index Management
Management
Ingest
Ingest Pipelines
Data
Index Management
Index Lifecycle Policies
Snapshot and Restore
Rollup Jobs
Transforms
Remote Clusters
Alerts and Insights
Index Management
Indices
Data Streams
Index Templates
Co
Update your Elasticsearch indices individually or in bull
Search
Name
springboot-test-log
Rows per page: 10


springboot-test-log
Summary
Settings
Mappings
Stats
Edit settings
General
Health
yellow
Status
open
Primaries
1
Replicas
1
Docs count
79
Docs deleted
0
Storage size
98.67kb
Primary storage size
98.67kb
Aliases
none

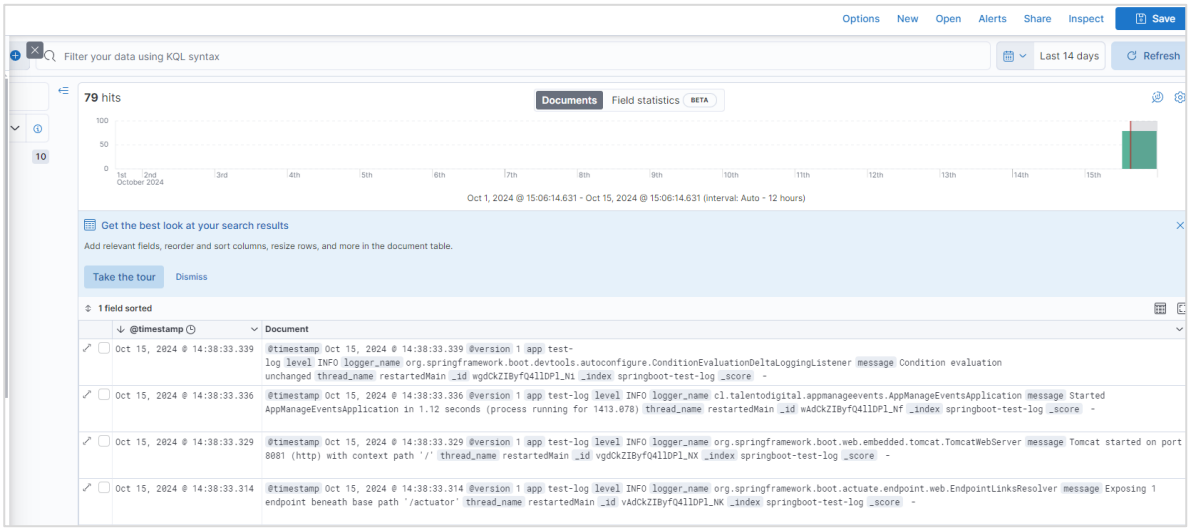
6.1.7 Creación de Dataview de Kibana.





6.1.8 Resultado final y visualización de Logs.

En **Analytics -> Discover** se mostrará gráficamente el resultado de la implementación de monitoreo.



6.2 Graylog,Fluentd,Grafana (GFG).

Configuración inicial del stack GFG con Docker compose:

Configuración inicial del docker-compose.yml; Procedimos a definir los servicios en el archivo docker-compose.yml, para levantar los componentes de Graylog, Fluentd y Grafana. Especificamos la configuración necesaria, para que estos contenedores se comunicaran correctamente entre ellos y con Elasticsearch, que sería el encargado de almacenar los logs :

```
services:
  jenkins:
    image: jenkins/jenkins:lts
    volumes:
      - jenkins_data:/var/jenkins_home
    networks:
      - sonarnet
    ports:
      - '8080:8080'
      - '50000:50000'

  sonarqube:
    image: sonarqube:10.6-community
    volumes:
      - sonarqube_conf:/opt/sonarqube/conf
      - sonarqube_data:/opt/sonarqube/data
      - sonarqube_extensions:/opt/sonarqube/extensions
      - sonarqube_bundled-plugins:/opt/sonarqube/lib/bundled-plugins
    networks:
      - sonarnet
    ports:
      - '9000:9000'
    environment:
      - SONAR_JDBC_USERNAME=sonar
      - SONAR_JDBC_PASSWORD=sonar
      - SONAR_JDBC_URL=jdbc:postgresql://db:5432/sonar
    depends_on:
      - db

  db:
    image: postgres:13
    networks:
      - sonarnet
    environment:
      - POSTGRES_USER=sonar
      - POSTGRES_PASSWORD=sonar
      - POSTGRES_DB=sonar
    volumes:
      - postgresql:/var/lib/postgresql
      - postgresql_data:/var/lib/postgresql/data

  nexus:
    image: sonatype/nexus3
    volumes:
      - nexus-data:/nexus-data
    networks:
      - sonarnet
    ports:
      - '8081:8081'

  mongodb:
    image: mongo:4.4
    networks:
      - sonarnet
    volumes:
      - mongo_data:/data/db
    restart: always
```

```
elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:7.10.2
  environment:
    - discovery.type=single-node
    - ES_JAVA_OPTS=-Xms512m -Xmx512m
  networks:
    - sonarnet
  volumes:
    - elasticsearch_data:/usr/share/elasticsearch/data
  ports:
    - '9200:9200'
  restart: always

graylog:
  image: graylog/graylog:4.2
  networks:
    - sonarnet
  depends_on:
    - mongodb
    - elasticsearch
  environment:
    - GRAYLOG_PASSWORD_SECRET=Gp4d5bGndgXlZxIvpgsGmjUzZZN+l8izRGJw+YNRExs=
    - GRAYLOG_ROOT_PASSWORD_SHA2=38f6177fe1c4eab3707b75de9f15f2f5b22bc157de84c0b5b5fcc9d619cf84bf
    - GRAYLOG_HTTP_EXTERNAL_URI=http://127.0.0.1:9002/
    - GRAYLOG_MONGODB_URI=mongodb://mongodb:27017/graylog
    - GRAYLOG_ELASTICSEARCH_HOSTS=http://elasticsearch:9200
  volumes:
    - graylog_data:/usr/share/graylog/data
  ports:
    - '9002:9000'
  restart: always

fluentd:
  image: fluent/fluentd-kubernetes-daemonset:v1.14-debian-elasticsearch7-1
  volumes:
    - ./fluentd/conf:/fluentd/etc
  ports:
    - '24224:24224'
    - '24224:24224/udp'
  environment:
    - FLUENT_ELASTICSEARCH_HOST=elasticsearch
    - FLUENT_ELASTICSEARCH_PORT=9200
  networks:
    - sonarnet
  depends_on:
    - elasticsearch
```

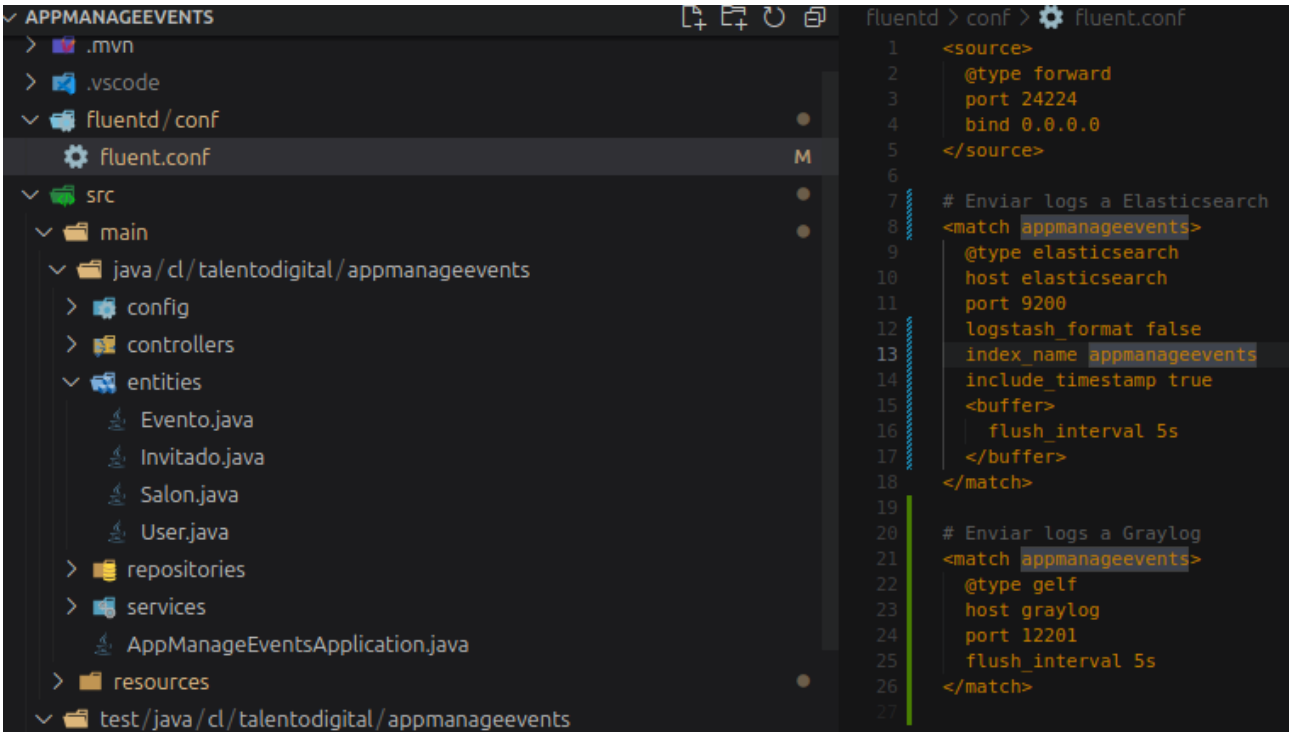
```yaml
  grafana:
    image: grafana/grafana:8.5.5
    networks:
      - sonarnet
    depends_on:
      - graylog
      - elasticsearch
    ports:
      - '3001:3000'
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=admin
    volumes:
      - grafana_data:/var/lib/grafana
    restart: always

networks:
  sonarnet:
    driver: bridge

volumes:
  sonarqube_conf:
  sonarqube_data:
  sonarqube_extensions:
  sonarqube_bundled-plugins:
  postgresql:
  postgresql_data:
  jenkins_data:
  nexus-data:
  mongo_data:
  elasticsearch_data:
  graylog_data:
  grafana_data:
```

Configuración de Elasticsearch: En esta parte, aseguramos que Elasticsearch estuviera disponible para almacenar los logs enviados desde Fluentd. Utilizamos la versión 7.9.2 y especificamos los puertos necesarios. También nos aseguramos de que Fluentd y Grafana pudieran acceder correctamente.

Configuración de Fluentd (fluent.conf): Configuramos Fluentd para enviar logs a Elasticsearch con este archivo de configuración. Aseguramos que se incluyera el @timestamp para que Grafana pudiera indexar los datos correctamente.



Configuración de logback en la aplicación Spring Boot (logback-spring.xml): En nuestra aplicación Spring Boot, añadimos un appender para que los logs se enviaran a Fluentd utilizando FluentAppender. Esto lo hicimos modificando el archivo logback-spring.xml:

src > main > resources > </> logback-spring.xml

```xml
<configuration>
    <!-- Appender para enviar logs a Fluentd via Socket -->
    <appender name="FLUENTD" class="ch.qos.logback.classic.net.SocketAppender">
        <remoteHost>localhost</remoteHost> <!-- Cambia a 'fluentd' si está en la red de Docker -->
        <port>24224</port>
        <reconnectionDelay>10000</reconnectionDelay>
        <encoder class="net.logstash.logback.encoder.LogstashEncoder" />
    </appender>

    <!-- Appender para la consola -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
            <pattern>%d{yyyy-MM-dd HH:mm:ss} %-5level %logger{36} - %msg%n</pattern>
        </encoder>
    </appender>

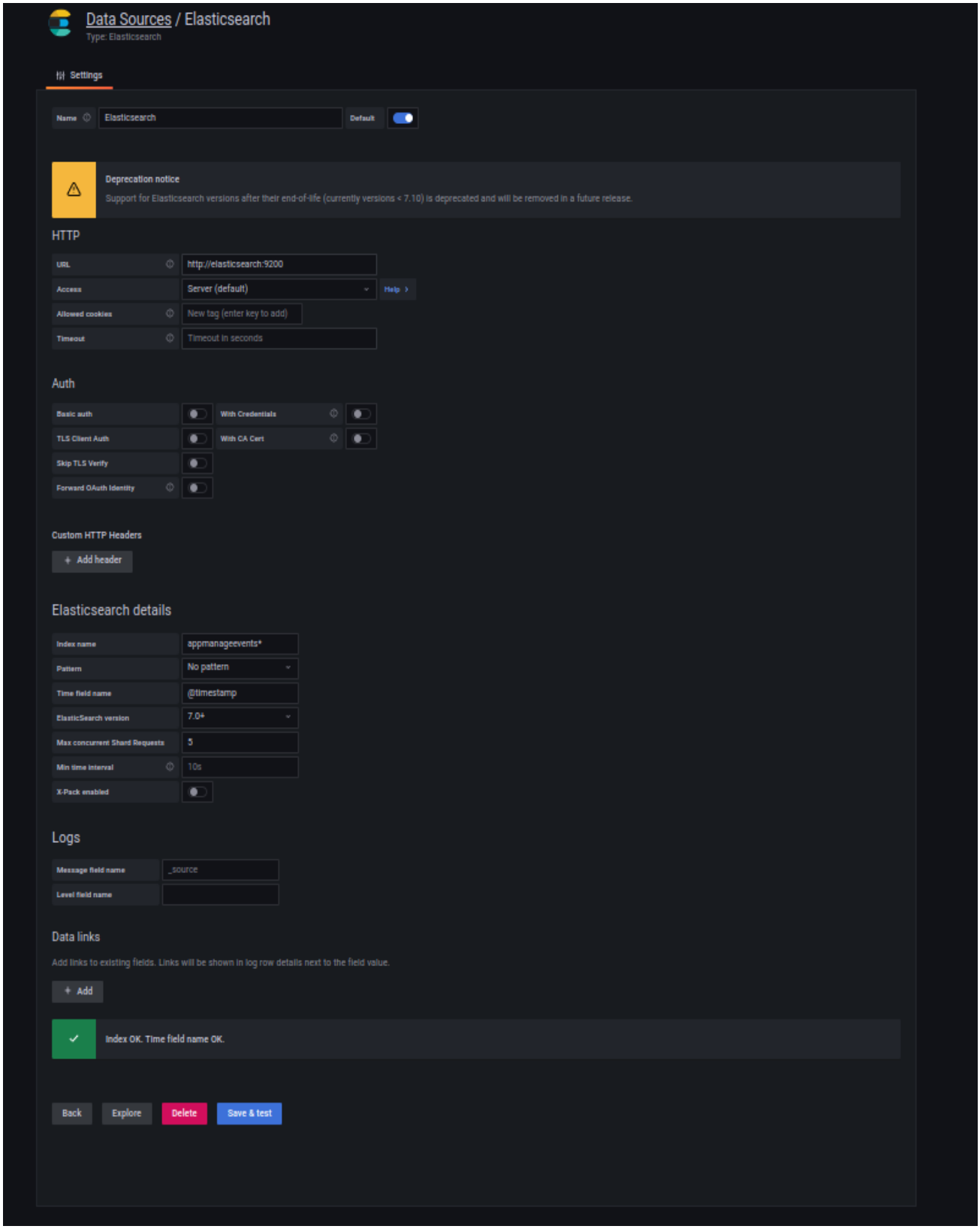
    <!-- Nivel de log y appenders utilizados por defecto -->
    <root level="INFO">
        <appender-ref ref="FLUENTD" />
        <appender-ref ref="CONSOLE" />
    </root>
</configuration>
```

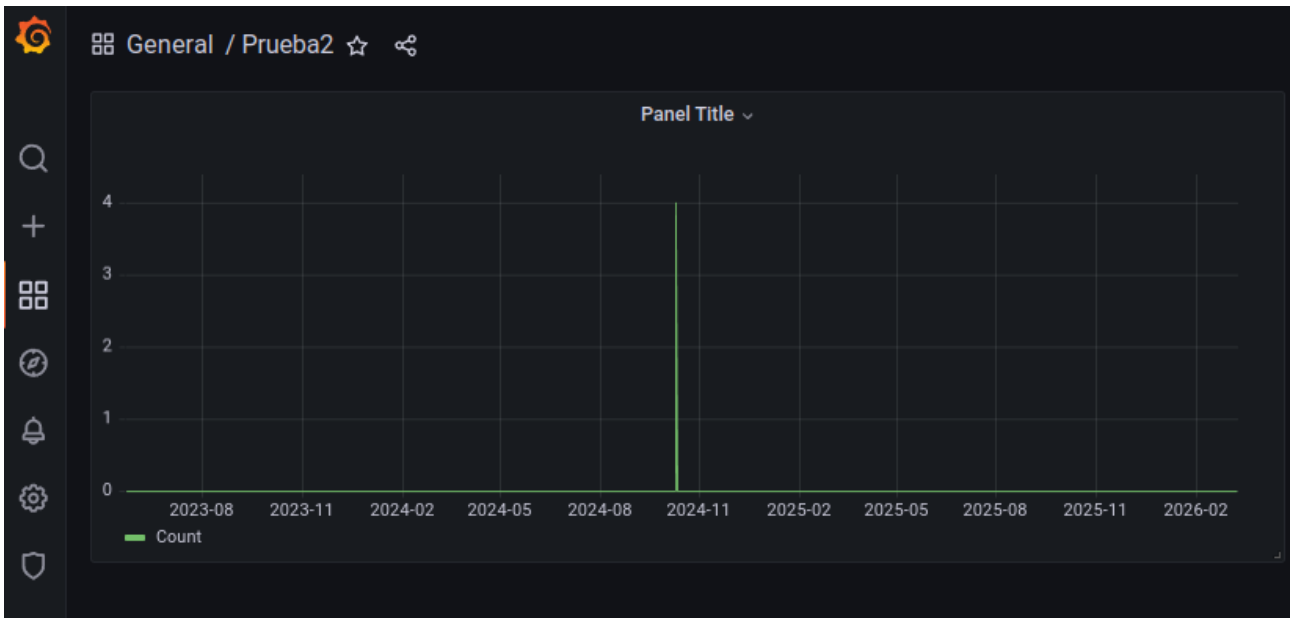
Validación del Stack:

Primeramente vemos que todos los docker-compose esten up :

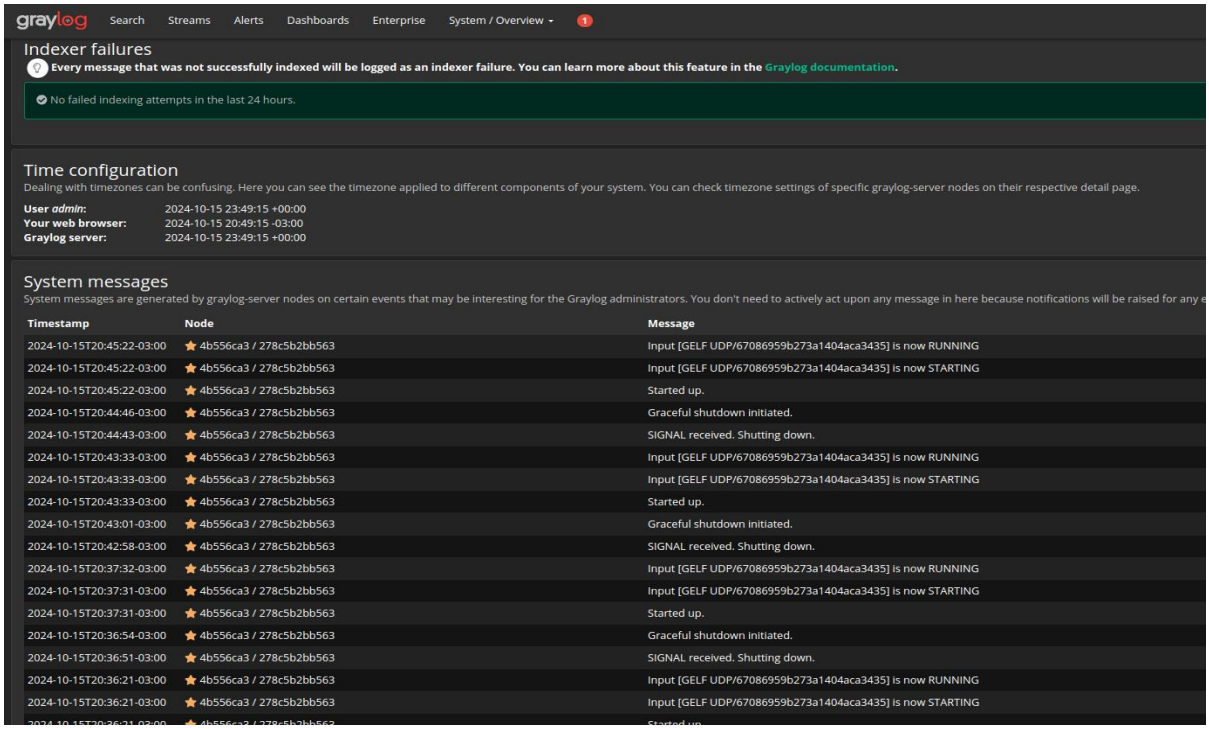
```
seiler@jesus-seiler:~/Escritorio/AppManageEvents$ docker-compose ps
            Name                          Command                State                                                
```

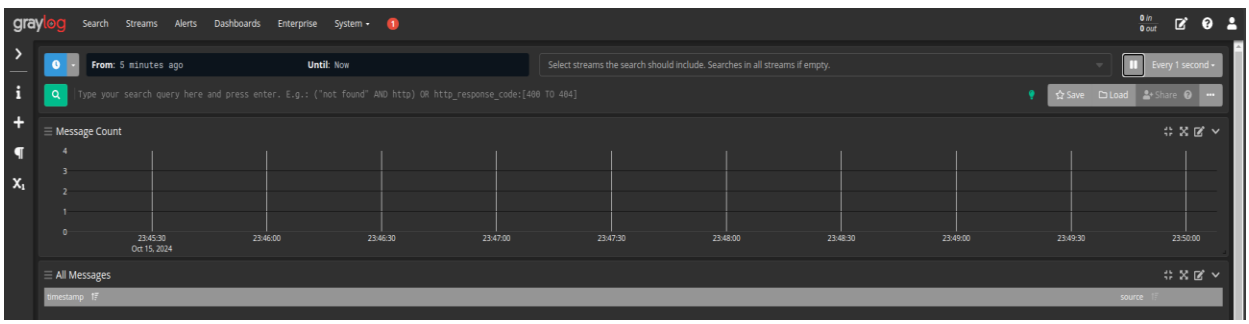
- Accedimos a **Grafana** y añadimos a Elasticsearch como fuente de datos, seleccionando el campo @timestamp para gestionar la data temporal.





- En **Graylog**, también configuramos dashboards para visualizar los logs de nuestra aplicación en tiempo real.





7. Conclusión

El proyecto tiene como objetivo desarrollar e implementar una aplicación web para la gestión y arriendo de salones destinados a eventos de diferentes tipos, el enfoque principal radica en garantizar la escalabilidad y resiliencia del sistema, utilizando tecnologías robustas como Java y Spring Boot, en combinación con Jenkins para la integración continua y el despliegue (CI/CD) y GitHub para el control de versiones, esta elección tecnológica no solo permite automatizar los procesos clave, sino que también garantiza la estabilidad y calidad de la aplicación a lo largo del tiempo.

En la fase inicial del proyecto, la colaboración estrecha con los stakeholders es fundamental para definir de manera precisa los requerimientos funcionales y no funcionales, estableciendo una base sólida para el desarrollo posterior, este enfoque asegura que la aplicación cumpla con las expectativas de los usuarios finales y se alinee con los objetivos estratégicos de la organización.

El proyecto adopta un enfoque DevOps, apoyado en prácticas como Git Flow y la orquestación de pipelines en Jenkins, esto permite gestionar de manera eficiente las ramas de desarrollo y producción, facilitando la integración continua de nuevas funcionalidades y correcciones, gracias a Jenkins, se automatizan pruebas unitarias, análisis de código estático y el despliegue en diferentes entornos, minimizando los riesgos de errores en producción y acelerando los ciclos de desarrollo, reforzando la agilidad y eficiencia a lo largo del proceso, lo que resulta en una mayor capacidad de respuesta ante cambios y mejoras.

Para asegurar el monitoreo y la observabilidad de la aplicación, se implementan herramientas avanzadas como el stack ELK (Elasticsearch, Logstash y Kibana) y el stack GFG (Graylog, Fluentd y Grafana), estas herramientas permiten un análisis en tiempo real del comportamiento del sistema, ayudando a detectar problemas de rendimiento o fallos antes de que impacten en la experiencia del usuario y el enfoque proactivo garantiza que el sistema funcione de manera óptima y que cualquier incidencia sea resuelta rápidamente.

En conjunto, el proyecto sigue un enfoque ágil basado en Scrum, lo que facilita una entrega continua de valor y una mejora constante, al mantener la satisfacción del cliente como objetivo central, se asegura que el desarrollo de la aplicación se ajuste a las necesidades del negocio y evolucione con el tiempo, ofreciendo una solución escalable y sostenible para la gestión de salones.